

Computational Grammar Theory

Structure, Operation, and the Cost of Resolution

Abed Kadaan

Computational Substrate Scientist and Principal Researcher

Working edition 0.1, October 2026

A research manuscript in progress. Not peer reviewed; proofs marked “proved here” are self-checked and await independent review.

Contents

PART I

What Is a Computational Structure?

Ordinary data structures, treated from first principles: elements, relations, operations, representations and costs — before any new notation.

1 Structures, Representations, and Operations

Elements, relations, state, operations and machine models — and the three questions this book never lets collapse into one.

MIXED

2 Arrays: Addresses You Can Compute

The array as the first and purest example of a grammar whose words are arithmetic — and what that costs when the array must grow, shift or become multidimensional.

MIXED

3 Lists, Stacks, Queues, and Deques

Pointer-based sequences, and the first structure whose language of defined operation sequences is not regular.

MIXED

4 Hash Tables, Sets, Maps, and Tries

When names are not dense numbers — hashing computes an address anyway, at the price of collisions; tries turn keys into words read by an automaton.

MIXED

5 Trees, Heaps, Search Trees, and Forests

Tree addresses as words, the heap numeration and exactly when it gives constant-time navigation, search trees, and forests compressed by sharing.

MIXED

6 Graphs and Paths

Graph representations, walks versus simple paths, and the relational semantics that makes "a path is an edge followed by a path" a theorem rather than a slogan.

MIXED

PART II

Grammar as a Computational Object

The formal core: operational grammars, three semantic layers, and the admissibility languages that constrain operations.

7 Operational Grammars

The formal object of Computational Grammar Theory — moves, relational semantics, admissibility languages, adequate representations, cost profiles — and the five mechanisms through which grammar can affect cost.

MIXED

8 Admissibility Languages and the Product Construction

How an admissibility language is compiled into an automaton, how a structure is searched under it, and exactly how much a constraint can help or hurt.

MIXED

PART III

Classical Algorithms Reconstructed

Baseline algorithms with proofs, implementations and counted costs, each mapped to its grammar — including the many cases where grammar adds nothing.

9 Searching and Sorting Reconstructed

Linear and binary search, four comparison sorts and counting sort — with invariants, counted costs, the comparison lower bound, and the one way around it.

MIXED

10 Graph Algorithms Reconstructed

Traversal, ordering, components, shortest paths, spanning trees and closure — and the theorem that evaluating a path grammar is computing a closure, with cost depending on the grammar's shape.

MIXED

11 Range Queries, Dynamic Programming, and Strings

Fenwick and segment trees as arithmetic addressing, sparse tables as precomputation, dynamic programming as shared derivations, and string matching as automata.

MIXED

PART IV

Cost, Information, and the Limits of Grammar

Preprocessing, information lower bounds, compiled operations, and what grammars cannot do to complexity classes.

12 Cost Profiles, Preprocessing, and Information

When does precomputing pay? What is the least information a "grammar of all answers" must contain? And what happens when the structure changes?

MIXED

13 Compiling Operations: When Grammar Becomes Arithmetic

A move word of any length can be compiled into a constant-size operation on some structures and provably not on others. This chapter finds the line.

PROVED HERE

14 The Limits: Advice, Restriction, and P versus NP

What a grammar can and cannot do to complexity classes — stated as precise challenges with their proof obligations, and answered.

MIXED

PART V

PageRank, Search, and Information Retrieval

A long-form case study: the mathematics of PageRank from first principles, and what a grammatical view does and does not add.

15 PageRank from First Principles

The random surfer, stochastic matrices, dangling pages, damping, existence and uniqueness of the ranking, power iteration with certified error, solvers, sparse representations, updates and personalization.

MIXED

16 PageRank Through the Lens of Grammar

The random surfer as a probabilistic operational grammar, rankings constrained by admissibility languages, block structure as a structural grammar — and an honest list of what the grammatical view does not improve.

MIXED

PART VI

Modern Computational Systems

Explicit models of computation graphs, retrieval, vector search and agent workflows — and careful limits on what grammar can claim about them.

17 Computation Graphs, Retrieval, and Agent Systems

Where computational grammars apply to modern AI systems — computation graphs, attention masks, retrieval pipelines, vector search and agent workflows — and where they do not.

MIXED

PART VII

The Laboratory, the SDK, and Grammar as a Service

The executable companion: how to use, extend and serve computational grammars.

18 The CGT SDK

A tutorial for the executable companion to this book — structures, grammars, the three layers, cost models, comparisons, experiment records and extension points.

MIXED

19 Grammar as a Service

A contract for requesting operations on versioned grammars, with layered rejection, quotas, determinism, provenance and cache validity — and an argument about what should run where.

PROPOSED

PART VIII

Conclusions

What has actually been established, what has not, and where the research goes next.

20 What Computational Grammar Theory Has Established

The confirmed contributions, the negative results, the open questions and the research agenda — without inflating any of them.

MIXED

Appendices

Appendix A Notation and Conventions

The symbols, cost conventions and evidence labels used throughout the book.

DEFINITION

Appendix B Reproducing This Book

How to rerun every experiment, test, example and build step, and what reproducibility means for each.

DEFINITION

Structures, Representations, and Operations

LEARNING OBJECTIVES

After this chapter you should be able to:

- describe a data structure as a set of elements with relations, distinguished from any particular **representation** of it in memory;
- explain what it means for an operation to be *partial*, and why “undefined” is a legitimate answer;
- state the word-RAM cost model and say precisely what a claim such as “this takes $\mathcal{O}(1)$ time” asserts and does not assert;
- separate, for any computational claim, the questions of **syntax** (is the request well formed?), **semantics** (what does it denote?) and **execution** (what does it cost?).

1.1 The question this book asks

Every programmer learns early that an array lets you read element i “in constant time”, while a linked list makes you walk to it. The usual explanation is mechanical: the array’s elements sit side by side in memory, so the machine can *compute* where element i lives; the list’s elements are scattered, so the machine must *follow* pointers. Both explanations are true. But notice what they are really about. They are about the relationship between a **description of an operation** — “element i ” — and the **machinery** that carries it out.

This book is about that relationship, taken seriously and made precise. Its research program was formulated by Abed Kadaan, who arrived at it from work on programmable addresses: addresses that do more than locate a resource, because they also identify a state, select a transformation, and record a history of transitions. In such a system an address behaves less like a label and more like a *sentence* in a small language, and the language has a grammar. The question that follows is general:

Can the grammar of a computational structure — the formal rules saying which operations are admissible, how they compose, and what they mean — become part of the machinery by which the structure is operated on, and not merely a description of it?

The motivation is not evidence, and this book does not treat it as such. The question has to be answered the way any question in computer science is answered: by precise definitions, by algorithms and their analysis, by proofs, by reproducible experiments, and — just as importantly — by **counterexamples** that mark where an idea stops working. Some of the answers turn out to be positive and new; many turn out to be known results seen from a new angle; several turn out to be negative. All three kinds of answer appear in this book, labelled as such.

1.2 Elements, relations, and states

We begin with the most ordinary vocabulary.

DEFINITION 1.1 (Computational structure)

CGT-DEF-001

A **computational structure** $\mathfrak{A} = (A; R_1, \dots, R_k)$ consists of a finite set A of **elements** and finitely many **relations** $R_i \subseteq A^{r_i}$ over them, possibly together with labels on elements and distinguished elements (a root, a source, a current position). Its **size** n is $|A|$ (plus the number of related tuples when that is larger).

The definition is deliberately bland. It is the standard notion of a finite relational structure from logic, and nothing in it is new. Its value is that it lets us talk about very different data structures in the same words.

EXAMPLE 1.1 (Three structures in one vocabulary)

1. A **playlist** of five songs is the set $A = \{s_0, \dots, s_4\}$ with the successor relation $next = \{(s_0, s_1), (s_1, s_2), (s_2, s_3), (s_3, s_4)\}$ and the distinguished element s_0 (the first song).
2. A **family tree** is a set of people with the relation $parent$ and a distinguished root.
3. A **road map** is a set of junctions with a labelled relation $road_\ell(u, v)$ meaning “a road of kind ℓ leads from u to v ”.

The *state* of a structure is everything an operation can inspect or change: which tuples are present, the labels, the distinguished elements. A program that manipulates a structure moves it from state to state.

1.3 Operations are partial

An **operation** inspects the current state, possibly changes it, and possibly returns a value. Reading the third song of the playlist returns a value and leaves the state alone; inserting a song changes the state. Crucially, many operations are **partial**: reading the seventh song of a five-song playlist is not an operation that “returns an error value”; it is an operation that is *not defined* in this state.

This distinction matters throughout the book, so we fix it now. When an operation is undefined in a state we will say that its **denotation is empty**. In the formal treatment of Chapter 7, each operation a denotes a relation $\llbracket a \rrbracket \subseteq A \times A$ between states (or elements), and “undefined at x ” means $\llbracket a \rrbracket(x) = \emptyset$. Programming languages report this with exceptions, sentinel values or crashes; the mathematics simply records that there is no successor state.

1.4 Representations

A structure is a mathematical object. A **representation** is a concrete layout of that object in a machine’s memory: a bit string, an arrangement of memory cells and pointers. The same structure has many representations, and *the representation, not the structure, determines what operations cost*.

EXAMPLE 1.2 (One sequence, two layouts)

The sequence (10, 20, 30, 40) can be laid out

- **contiguously**, as four consecutive memory cells starting at some base address b : the value 30 sits at address $b + 2$;
- **as a linked list**, as four two-word records (value, next) placed anywhere in memory, each holding the address of the following record, with a separate *head* cell holding the address of the first.

Both represent the same abstract sequence. In the first, the position of element i is *computable* from i by one addition. In the second, the position of element i is known only after following i pointers.

The same pattern recurs for every structure in Part I. A graph can be stored as an $n \times n$ adjacency matrix or as adjacency lists; a binary tree as linked records or as an array in “heap order”; a set as a sorted array, a hash table or a balanced tree. In each case one representation

makes some operations cheap and others expensive, and a second representation reverses the trade. The comparison is only meaningful once we say *how cost is counted*.

1.5 Cost needs a machine model

Throughout the book costs are stated in an explicit machine model.

DEFINITION 1.2 (Word RAM)

The **word RAM** has an unbounded array of memory cells, each holding a w -bit word with $w \geq \log_2 n$. In one step it can read or write a cell at a computed address, or perform an arithmetic, comparison or bitwise operation on one or two words. An operation on a b -bit integer costs $\lceil b/w \rceil$ word steps.

Three conventions follow, and the book holds itself to them strictly.

1. “ $\mathcal{O}(1)$ ” is a **claim about a model**. It means: a number of word-RAM steps bounded by a constant *independent of the parameters that grow*, and those parameters must be named. An array access is $\mathcal{O}(1)$ as n grows because one address computation suffices; it is *not* $\mathcal{O}(1)$ if the index itself has n bits.
2. **Build, storage, query, update and output are separate costs**. A query that takes $\mathcal{O}(1)$ time after $\Theta(n^2)$ preprocessing is not “an $\mathcal{O}(1)$ algorithm”. Chapter 12 develops this into a full accounting discipline (the *cost profile*).
3. **We count, we do not time**. The book’s software counts elementary steps — reads, writes, comparisons, pointer hops, hash probes — under a stated cost model. Wall-clock time measured in an interpreted language mostly measures the interpreter; a flat curve of timings over a small range is not evidence of constant time.

REMARK

The word RAM idealizes real machines: it ignores caches, memory hierarchies and parallelism. Where these matter (PageRank on large graphs, in Part V) we say so explicitly and do not let a word-RAM bound stand in for a claim about real hardware.

1.6 Syntax, semantics, execution

We can now state the organizing principle of the book. Any request made of a structure raises three different questions, and conflating them is the most common source of exaggerated claims about “fast” representations.

DEFINITION 1.3 (The three layers)

CGT-DEF-003

For a request (an *operation expression*) addressed to a structure:

1. **Syntax** asks whether the expression is well formed and admissible — whether it uses known operations, with the right number of arguments, in an allowed order.
2. **Semantics** asks what the expression *denotes* in the current state: which values it returns and which state it leads to, or that it is undefined.
3. **Execution** asks what it costs to compute that denotation in a given representation and machine model.

The three questions are independent: every combination of answers occurs (CGT-PROP-002).

EXAMPLE 1.3 (The layers come apart)

- *Well formed but undefined.* `get(7)` on a five-element array is a perfectly good expression; it denotes nothing.
- *Defined but expensive.* `get(4999)` on a linked list of 5,000 elements is defined, and costs about 5,000 pointer hops.
- *Cheap to retrieve, expensive to obtain.* “Is there a path from u to v ?” is answered by one bit lookup if a reachability table has been precomputed — but that table may need $n^2/4$ bits for some graphs, whatever encoding is used (Chapter 12).

The SDK that accompanies this book makes the three layers concrete. Executing an expression returns a result that reports each layer separately, and checks that the representation’s answer agrees with a reference semantics:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk.structures import ArrayStructure, LinkedListStructure

for S in (ArrayStructure, LinkedListStructure):
    g = S([10, 20, 30, 40, 50]).grammar()
    for expr in ["get(3)", "get(7)", "fLy(1)"]:
        r = g.execute(expr)
        layer = ("syntax error" if not r.syntax.ok else
```

```
"undefined" if not r.defined else f"value {r.value}")
print(f"${S.name:12s} {expr:7s} → {layer:13s} cost {r.cost.snapshot()}")
```

OUTPUT (CHECKED)

```
array      get(3) → value 40      cost {'arith': 1, 'compare': 1, 'read': 1}
array      get(7) → undefined     cost {'compare': 1}
array      fly(1) → syntax error  cost {}
linked_list get(3) → value 40      cost {'compare': 1, 'pointer': 3, 'read': 2}
linked_list get(7) → undefined     cost {'compare': 1}
linked_list fly(1) → syntax error  cost {}
```

Read the output carefully. The *same* expression `get(3)` has the same meaning (the value 40) on both structures; what differs is the execution layer — one address computation versus three pointer hops. The expression `get(7)` passes the syntax layer and is rejected by the semantic layer after a single bounds check. The expression `fly(1)` never reaches semantics: there is no such operation.

1.7 A grammar, informally

A sequence of operations such as `get(2); set(0, 9); get(0)` is a sentence in a small language. Its alphabet is the structure’s set of operations; its grammar says which sentences are admissible; its meaning is the composition of the meanings of the individual operations. For an array that grammar is almost trivial: every sequence of `get`, `set`, `len` and the rest is admissible, and indices out of range make a sentence undefined. For other structures the grammar has real content:

- for a **stack**, a sentence that pops more than it has pushed is undefined — and the set of *defined* sentences is not even a regular language (Chapter 3);
- for a **binary tree**, the sentences over the moves $\{L, R, U\}$ (“left child”, “right child”, “up”) are exactly the addresses and walks of the tree (Chapter 5);
- for a **graph with labelled edges**, a regular expression over labels describes which *paths* are admissible, and resolving it costs at most a factor $|Q|$ more than ordinary search, where $|Q|$ is the size of the expression’s automaton — a factor that is tight (Chapter 8).

Part II turns this informal picture into definitions. Before that, Part I examines the ordinary structures one at a time, because the definitions must fit them, not the other way round.

1.8 How evidence is labelled in this book

Every formal statement carries a badge saying what kind of claim it is:

- **DEFINITION** introduces an object; it is not a discovery.
- **ESTABLISHED** marks a known result from the literature, with a citation.
- **ADAPTED** marks known ideas restated or packaged in this book's terms.
- **PROVED HERE** marks a result whose proof appears in this book. Such proofs have been checked by their authors and, where possible, by exhaustive computation on small cases, but **they have not been independently reviewed**.
- **EMPIRICAL** marks a reproducible measurement — evidence, never proof.
- **CONJECTURE** and **REJECTED** mark open and failed hypotheses. Rejected hypotheses are kept: knowing where an idea fails is part of knowing the idea.
- **PROPOSED** marks a model or design introduced here whose novelty has not been established.

Identifiers such as [CGT-THM-006](#) link each statement to the research ledger, the repository's permanent record of definitions, hypotheses, experiments, results, counterexamples and decisions.

EXERCISE 1.1

Write the road map of §1.2 for three junctions u, v, x with a *motorway* from u to v and a *lane* from v to x and from x to u as a structure $(A; road_{motorway}, road_{lane})$. Which pairs are in the composition $road_{motorway} ; road_{lane}$ (“a motorway, then a lane”)?

► Solution to [Exercise 1.1](#)

EXERCISE 1.2

For each statement, say which layer — syntax, semantics or execution — it is about. (a) “pop on an empty stack is undefined.” (b) “Binary search needs at most $\lceil \log_2(n+1) \rceil$ comparisons.” © “insert(2) is rejected because insert takes two arguments.” (d) “There is a path from u to v whose labels spell abab.”

► Solution to [Exercise 1.2](#)

EXERCISE 1.3

A colleague reports that reading a value from their “grammar-indexed store” takes the same 40 nanoseconds for stores of 1,000, 10,000 and 100,000 entries, and concludes that reads are $\mathcal{O}(1)$. List three things you would need to know before accepting the conclusion.

EXERCISE 1.4

Give an example of an operation on a familiar structure that is cheap in one representation and expensive in another, other than indexed access. State both costs in the word-RAM model.

SUMMARY

- A **structure** is a set of elements with relations; a **representation** is a layout in memory; costs belong to representations.
 - Operations are **partial**: “undefined” is a semantic answer, not an execution failure.
 - Costs are stated in the **word-RAM** model, counted rather than timed, and split into build, storage, query, update and output.
 - Every request raises three independent questions — **syntax, semantics, execution** — and this book files every claim under exactly one of them.
 - Formal statements carry evidence badges; “proved here” means self-checked, not independently reviewed.
-

Arrays: Addresses You Can Compute

LEARNING OBJECTIVES

- State the address formula of an array and explain why it makes indexed access independent of the index.
- Write the array's operation grammar: its moves, their admissibility conditions, their denotations and their costs.
- Generalize the address formula to multidimensional arrays (row-major numeration) and explain density.
- Prove the exact amortized cost of a doubling dynamic array under an explicit cost model.
- Compare arrays and linked lists on identical workloads, and say which costs are intrinsic to the request and which to the representation.

2.1 Contiguous memory and the address formula

An **array** of n elements, each occupying s memory cells, is stored in ns consecutive cells starting at a **base address** b . Element i (counting from 0) begins at

$$\text{addr}(i) = b + i \cdot s \quad (2.1)$$

and nowhere else. Equation (2.1) is the whole secret of the array. To reach element i the machine does not search, and it does not follow anything: it *computes* where the element is, with one multiplication and one addition — or, when s is a power of two, a shift and an addition. On the word RAM of Chapter 1 this is a constant number of steps for every $i < n$, because i , b and s each fit in a machine word.

It is worth pausing on what makes this possible. Three properties hold at once:

1. **Every element has a name that is a number.** The index i identifies element i uniquely.
2. **The names are dense.** The indices $0, 1, \dots, n - 1$ are used with no gaps, so a block of exactly n cells suffices.

3. **The map from names to locations is arithmetic.** It is a single affine function, computable in $\mathcal{O}(1)$ word operations.

Each property can fail separately, and much of this book is about what happens when it does. If names are not numbers (strings, say), we need hashing (Chapter 4). If names are numbers but sparse — a few thousand keys drawn from $\{0, \dots, 2^{64} - 1\}$ — a block indexed by them would be astronomically large. If locations are not an arithmetic function of names, we must store and follow pointers (Chapter 3). The array is the case where all three hold, and that is why it is the right place to begin.

DEFINITION 2.1 (Array structure)

CGT-DEF-001

An **array** of length n is the structure whose elements are the positions $\{0, \dots, n - 1\}$, each carrying a value, with the successor relation $i \mapsto i + 1$. Its **contiguous representation** stores the value at position i in the s cells starting at $\text{addr}(i) = b + is$.

2.2 The array's grammar

An operation on an array is written as a *call*: a name and arguments. Here is the move signature of the dynamic array used throughout the book (it is exactly the signature of `ArrayStructure` in the SDK):

MOVE	ADMISSIBLE WHEN	DENOTATION (ON THE ABSTRACT SEQUENCE S)	COST IN THE CONTIGUOUS REPRESENTATION
<code>get(i)</code>	$0 \leq i < n$	returns s_i	1 comparison, 1 address computation, 1 read
<code>set(i, x)</code>	$0 \leq i < n$	replaces s_i by x	1 comparison, 1 address computation, 1 write
<code>len</code>	always	returns n	1 read of the header
<code>append(x)</code>	always	$s \cdot x$	1 write, plus a reallocation when full (§§2.4)
<code>pop</code>	$n > 0$	removes and returns s_{n-1}	$\mathcal{O}(1)$
<code>insert(i, x)</code>	$0 \leq i \leq n$	inserts x before position i	shifts $n - i$ elements

MOVE	ADMISSIBLE WHEN	DENOTATION (ON THE ABSTRACT SEQUENCE S)	COST IN THE CONTIGUOUS REPRESENTATION
<code>delete(i)</code>	$0 \leq i < n$	removes and returns s_i	shifts $n - i - 1$ elements

Three columns, three layers. The *admissibility* column is part of the semantics: `get(9)` on a five-element array is well formed but denotes nothing. The *denotation* column says what each move means on the **abstract** sequence, independently of any memory layout. The *cost* column belongs to one particular representation; the linked list of Chapter 3 has the same first two columns and a different third.

A sequence of calls is an operation expression, and its meaning is the composition of the meanings of its calls: `set(0, 9); get(0)` returns 9 whatever the array held before, provided it was non-empty. This compositional reading is formalized in Chapter 7. For now it is enough to see it executed, with the cost of each call counted:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk.structures import ArrayStructure

g = ArrayStructure([5, 3, 8, 1]).grammar()
r = g.execute("set(0, 9); get(0); insert(1, 7); get(4); delete(0)", trace=True)
print("values:", r.values)
print("final :", g.structure.abstract())
print("cost  :", r.cost.snapshot(), "total", r.cost.total)
print("adequate (representation agrees with semantics):", r.adequate)
```

OUTPUT (CHECKED)

```
values: [None, 9, None, 1, 9]
final : [7, 3, 8, 1]
cost  : {'alloc': 8, 'arith': 4, 'compare': 5, 'read': 14, 'write': 13} total 44
adequate (representation agrees with semantics): True
```

The `insert` filled the array to capacity and forced a reallocation (`alloc 8`), and both `insert` and `delete` shifted elements: the reads and writes beyond those of `get` and `set` are the shifts. The last line is a check the SDK performs on every execution: the values computed by the memory representation are compared with the values computed by a separate, purely mathematical reference semantics on the abstract sequence. Agreement is called **adequacy**, and every structure in the SDK is tested for it on thousands of random operation sequences.

REMARK

Out-of-range indices are where the semantic layer earns its keep. A language such as C would let `get(9)` read whatever memory follows the array — the representation has *a* behaviour even where the abstract operation has *no* meaning. Keeping a reference semantics separate from the representation is what lets us say precisely that such a behaviour is not a value of the operation.

2.3 Multidimensional arrays and dense numerations

A $d_1 \times d_2$ grid of values is stored by choosing a *numeration* of its positions: a bijection from coordinate pairs to $\{0, \dots, d_1 d_2 - 1\}$, followed by the address formula. The usual choice is **row-major** order:

$$\nu(x_1, x_2) = x_1 d_2 + x_2, \quad \text{and in general} \quad \nu(x_1, \dots, x_k) = \sum_{i=1}^k x_i \prod_{j>i} d_j. \quad (2.2)$$

PROPOSITION 2.2 (Row-major numeration is a dense arithmetic addressing)

ESTABLISHED

For dimensions $d_1, \dots, d_k \geq 1$, the map ν of (2.2) is a bijection from $\prod_i [0, d_i)$ onto $[0, N)$, $N = \prod_i d_i$. Moving one step along axis i adds or subtracts the **stride** $\sigma_i = \prod_{j>i} d_j$, so each move costs $\mathcal{O}(1)$ word operations, and the step is defined exactly when the coordinate stays in $[0, d_i)$.

Proof.

ν is the mixed-radix representation of integers with digits $x_i \in [0, d_i)$ and radices $d_1, \dots, d_k$; every integer in $[0, N)$ has exactly one such representation. Changing x_i by ± 1 changes ν by $\pm \sigma_i$. Definedness is the coordinate condition, which is checked by comparing x_i with 0 and $d_i - 1$ — or, if only ν is stored, by computing $x_i = \lfloor \nu / \sigma_i \rfloor \bmod d_i$, still $\mathcal{O}(1)$ word operations for fixed k . ■

The **density** of a numeration — the number of elements divided by the size of the range of numerals — is exactly 1 here. That is the property that lets a block of exactly N cells hold the grid. In Chapter 5 the same idea applied to binary trees gives density 1 for complete trees and density $2^{-\Theta(n)}$ for path-shaped ones, which is the whole difference between a heap and an unusable array. In Chapter 13 we go one step further: a long *sequence* of grid moves can be **compiled** into one translation together with a bounding box, after which it is applied to any start point in constant time, however long the sequence was ([CGT-PROP-008](#)).

2.4 Growing an array: amortized doubling

A contiguous block cannot grow in place: the cells after it may be in use. A **dynamic array** keeps a *capacity* $C \geq n$ and, when `append` finds $n = C$, allocates a new block of $2C$ cells, copies the n elements, and frees the old block. A single `append` can therefore cost $\Theta(n)$. The classical observation is that the *average* over any sequence of appends is constant [Cormen et al. 2022]. With an explicit cost model we can say exactly which constant.

PROPOSITION 2.3 (Exact cost of doubling)

PROVED HERE

Count one step per allocated cell, per element read and per element written, and start from an empty array of capacity 1. Then $m \geq 1$ consecutive appends cost exactly

$$T(m) = m + 4 \left(2^{\lceil \log_2 m \rceil} - 1 \right) < 9m.$$

The ratio $T(m)/m$ does not converge: it is $5 - 4/m$ when m is a power of two and tends to 9 along $m = 2^k + 1$.

Proof.

Each `append` writes one element: m steps. A reallocation happens when an `append` finds the array full, i.e. when $n = C$ for $C = 1, 2, 4, \dots$; the reallocation at capacity C allocates $2C$ cells and reads and writes C elements, costing $4C$. During m appends the array is found full at sizes $C = 2^0, 2^1, \dots, 2^{t-1}$ where $2^{t-1} < m \leq 2^t$, i.e. $t = \lceil \log_2 m \rceil$. Hence $T(m) = m + 4 \sum_{i=0}^{t-1} 2^i = m + 4(2^t - 1)$. Since $2^t < 2m$, $T(m) < m + 8m = 9m$. For $m = 2^k$, $t = k$ and $T/m = 1 + 4(m - 1)/m = 5 - 4/m$; for $m = 2^k + 1$, $t = k + 1$ and $T/m = 1 + 4(2m - 3)/m \rightarrow 9$. ■

The constant 9 is an artefact of charging one step per allocated cell; charging allocation as a single step gives the familiar bound of about 3 per `append`. That sensitivity is exactly why the book insists on stating the cost model. The proposition is easy to check against the SDK, which counts the same steps:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk.structures import ArrayStructure
import math

for m in (1, 16, 17, 1000, 1025, 4096, 4097):
    g = ArrayStructure([]).grammar()
    total = sum(g.execute(f"append({k})", check_adequacy=False).cost.total for k in range(m))
    formula = m + 4 * (2 ** math.ceil(math.log2(m)) - 1)
    print(f"m={m:5d} counted={total:6d} formula={formula:6d} per append={total/m:.3f}")
```

OUTPUT (CHECKED)

m=	1	counted=	1	formula=	1	per append=	1.000
m=	16	counted=	76	formula=	76	per append=	4.750
m=	17	counted=	141	formula=	141	per append=	8.294
m=	1000	counted=	5092	formula=	5092	per append=	5.092
m=	1025	counted=	9213	formula=	9213	per append=	8.988
m=	4096	counted=	20476	formula=	20476	per append=	4.999
m=	4097	counted=	36861	formula=	36861	per append=	8.997

CLAIM BOUNDARY

“Amortized $\mathcal{O}(1)$ append” is a statement about *sequences* of operations. It does not bound any single append, which can cost $\Theta(n)$ — a real concern in latency-sensitive systems — and it assumes the sequence starts from an empty or freshly allocated array.

2.5 Insertion and deletion shift the tail

Inserting at position i must move the $n - i$ elements at positions $i, \dots, n - 1$ one cell to the right, because positions are *defined* by the address formula: the element that ends up at position $i + 1$ has to be at address $b + (i + 1)s$. The same rigidity that makes `get` cheap makes `insert` expensive.

PROPOSITION 2.4

ESTABLISHED

In the contiguous representation, `insert(i, x)` and `delete(i)` cost $\Theta(n - i)$ reads and writes (plus a reallocation for `insert` at full capacity), and this is optimal for any representation in which element j is stored at address $b + js$ before and after the operation.

Proof.

The upper bound is the shift. For the lower bound: after inserting at i , each of the $n - i$ elements formerly at positions $j \geq i$ is at position $j + 1$, so the cell $b + (j + 1)s$ must now hold it; at least one write per moved element is necessary. ■

A linked list makes the opposite trade: insertion after a known node is $\mathcal{O}(1)$ pointer surgery, but finding position i costs i pointer hops. Neither representation dominates the other.

2.6 Arrays and lists on the same workload

Because the SDK gives both structures the same `get` / `set` / `insert` / `delete` moves with the same reference semantics, they can be run on identical workloads with correctness checked automatically:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk.lab import compare_structures, format_table, random_index_workload
from cgtsdk.structures import ArrayStructure, LinkedListStructure

n = 1000
for op in ("get", "delete"):
    work = random_index_workload(n, queries=200, seed=1, op=op)
    rows = compare_structures([ArrayStructure, LinkedListStructure], range(n), work)
    print(f"workload: 200 random {op}(i), n = {n}")
    print(format_table(rows))
    print()
```

OUTPUT (CHECKED)

```
workload: 200 random get(i), n = 1000
implementation      build    per query    space(w) correct
-----
ArrayStructure       1000      3.00        1003 True
LinkedListStructure  1000     551.05       2002 True

workload: 200 random delete(i), n = 1000
implementation      build    per query    space(w) correct
-----
ArrayStructure       1000     742.06       1003 True
LinkedListStructure  1000     418.66       1658 True
```

For random `get`, the array costs exactly 3 steps per query and the list about $n/2$ (here 551 — the 200 random indices happen to average a little above 500). For random `delete` the comparison reverses: the array shifts on average about half of the remaining elements with a read *and* a write each, while the list walks about half of them and splices in constant time — still linear, but cheaper by a factor of about 1.8 under this cost model. The “build” column charges one write per initial element; the space column counts words and shows the list’s two words per node (it shrinks as elements are deleted, while the array keeps its capacity). Some deletions in the random workload hit indices beyond the shrinking end and are undefined; both structures agree on exactly which ones, which is part of what `correct` certifies.

2.7 What the array teaches

The array is the cleanest instance of the first mechanism catalogued in this book ([CGT-DEF-009](#)): **arithmetic addressing**. Its operation `get(i)` is a one-word sentence whose evaluation is arithmetic, and the conditions under which this works — names are numbers, names are dense, locations are an affine function of names — are exactly the conditions we will look for in trees, grids, hypercubes and graphs. Where they hold, the grammar of the structure is executable machinery; where they fail, we pay for pointers, tables or search. The array also teaches the first counter-lesson: the rigidity of the address formula is what makes insertion linear. A representation is a bargain, not a free lunch.

EXERCISE 2.1

A $3 \times 4 \times 5$ array is stored in row-major order. Compute $\nu(2, 1, 3)$ and the three strides. Which single arithmetic operation moves from position $(2, 1, 3)$ to $(2, 2, 3)$?

► Solution to [Exercise 2.1](#)

EXERCISE 2.2

Redo [Proposition 2.3](#) for a growth factor of 3 instead of 2 (the capacity triples). Find the exact total cost and the lim sup of the per-append cost. Which factor minimizes the lim sup under this cost model?

EXERCISE 2.3

Suppose `pop` *shrinks* the array to half its capacity whenever n falls to $C/2$. Exhibit a sequence of m operations costing $\Theta(m^2)$. What shrinking rule avoids this?

► Solution to [Exercise 2.3](#)

EXERCISE 2.4

Run the comparison of [§2.6](#) with `insert(i, x)` workloads, using the SDK. Predict, before running, which representation wins and by roughly what factor; then explain any discrepancy from the counted costs.

EXERCISE 2.5

“An array lookup is $\mathcal{O}(1)$.” Rewrite this statement so that it names the machine model, the parameter that grows, and the assumption about the size of indices under which it is true. Give a setting in which it is false.

SUMMARY

- The address formula $\text{addr}(i) = b + is$ makes indexed access a constant number of word operations: the index is a one-word sentence evaluated by arithmetic.
 - It requires names that are numbers, dense, and mapped to locations by an arithmetic function; each requirement can fail, and later chapters study what replaces it.
 - Row-major numeration extends dense arithmetic addressing to grids; moves are stride additions.
 - Doubling dynamic arrays cost exactly $m + 4(2^{\lceil \log_2 m \rceil} - 1) < 9m$ steps for m appends in this book’s cost model; amortized bounds say nothing about single operations.
 - The rigidity that makes `get` cheap makes `insert` linear; lists make the opposite trade. The SDK runs both on identical workloads and checks them against one semantics.
-

Lists, Stacks, Queues, and Deques

LEARNING OBJECTIVES

- Describe singly and doubly linked lists, and explain why reaching position i costs i pointer hops in a pointer machine.
- Express list navigation as a grammar of cursor moves (`next` , `prev`).
- Prove that the operation sequences a stack can execute from empty form a non-regular language, and count them.
- Explain the circular buffer as arithmetic addressing on the ring $\mathbb{Z}_C$.

3.1 Linked lists

A **singly linked list** stores each element in a record holding the value and the address of the next record; a *head* cell holds the address of the first record. Records can live anywhere in memory. Nothing about record i 's address can be computed from i : it is known only once record $i - 1$ has been read.

PROPOSITION 3.1 (Walking is necessary) ESTABLISHED

Consider a machine that can obtain the address of a record only by reading it from the head cell or from a previously read record. Then reading the element at position i of a singly linked list requires at least i pointer dereferences after reading the head, and i suffice.

Proof.

By induction on i : the address of record i is stored only in record $i - 1$ (or in the head when $i = 0$), so record $i - 1$ must be read first. Following the chain from the head reads records $0, 1, \dots, i$: i dereferences after the head. ■

The hypothesis is the crux. On a word RAM nothing *forbids* computing an address, but for a list nothing makes the computed address *correct*: the representation offers no arithmetic relation between position and location. That is precisely the property the array has and the list lacks.

The list's operations have the same denotations as the array's (`get` , `set` , `insert` , `delete` on an abstract sequence) and different costs. Inserting after a node one already holds is three pointer writes, independent of n ; finding that node is the walk. The SDK's `LinkedListStructure` charges `pointer` for each hop, which is how the comparison at the end of Chapter 2 was counted.

3.1.1 Cursors and the grammar of navigation

A more grammatical view of a list replaces “position i ” by a **cursor** and two moves, `next` and `prev` (the latter requires a *doubly* linked list, whose records also store the previous address). A navigation is then a word over $\{\text{next}, \text{prev}\}$, and its denotation is a partial function on positions:

$$\llbracket \text{next} \rrbracket(i) = i + 1 \text{ if } i + 1 < n, \quad \llbracket \text{prev} \rrbracket(i) = i - 1 \text{ if } i \geq 1.$$

A word w moves the cursor by its *net displacement* $\#_{\text{next}}(w) - \#_{\text{prev}}(w)$, provided every prefix stays within $[0, n)$. Executing w on the list costs $|w|$ pointer hops. But the denotation of w is determined by just three numbers: the net displacement and the minimum and maximum prefix displacements. A word of a million moves could be “compiled” into those three numbers once and then applied in constant time — *if* the representation let us jump by a computed displacement. A doubly linked list does not; an array does. This simple observation reappears in Chapter 13 as a theorem about which structures admit compiled operations.

3.2 Stacks

A **stack** supports `push(x)` , `pop` , `top` and `empty` , with last-in-first-out semantics. Array-backed, every operation is amortized $\mathcal{O}(1)$ (pushes inherit the doubling analysis of Chapter 2). The interesting feature of the stack is not its cost but its **language**.

Ignore the pushed values and record only the sequence of move names. Starting from an empty stack, which words over $\{\text{push}, \text{pop}\}$ are *defined* — never pop an empty stack?

PROPOSITION 3.2 (The defined stack words are not regular) ESTABLISHED

Let $D \subseteq \{\text{push}, \text{pop}\}^*$ be the set of words in which every prefix contains at least as many *push* as *pop*. Then D is exactly the set of words defined on the empty stack, D is context-free, and D is not regular. The number of words of length L in D is $\binom{L}{\lfloor L/2 \rfloor}$.

Proof.

Definedness. A pop is defined iff the stack is non-empty, i.e. iff the number of pushes so far exceeds the number of pops so far; every pop in w is defined iff the prefix condition holds.

Non-regularity. Suppose a DFA with q states accepted D . Among the words $push^0, \dots, push^q$ two, $push^a$ and $push^b$ with $a < b$, lead to the same state. Then $push^a pop^b$ and $push^b pop^b$ are both accepted or both rejected; but the first is not in D and the second is. Contradiction.

Context-freeness. D is generated by $S \rightarrow \varepsilon \mid push\ S \mid push\ S\ pop\ S$ (prefixes of Dyck words), or recognized by a one-counter automaton.

Counting. Words in D are lattice paths of L steps ± 1 that never go below 0; by the reflection principle their number is $\binom{L}{\lfloor L/2 \rfloor}$, the classical ballot-type count. ■

The proposition is old mathematics, but its lesson for this book is new emphasis: **the admissibility language of a structure's operations can be more complex than any finite automaton can check**. A system that wants to validate stack-operation sequences *before* running them — as a Grammar-as-a-Service front end might (Chapter 19) — needs a counter, not a regular expression. The SDK sidesteps the issue by checking definedness in the semantic layer, call by call. We can let it count the defined words by brute force and compare with the formula:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from itertools import product
from math import comb
from cgtsdk.structures import StackStructure

for L in range(9):
    defined = 0
    for w in product(["push(0)", "pop"], repeat=L):
        r = StackStructure().grammar().execute(" ".join(w), check_adequacy=False)
        defined += bool(r.defined)
    print(L, defined, comb(L, L // 2))
```

OUTPUT (CHECKED)

```
0 1 1
1 1 1
2 2 2
3 3 3
4 6 6
5 10 10
6 20 20
7 35 35
8 70 70
```

3.3 Queues and the ring $\mathbb{Z}_C$

A FIFO **queue** supports `enqueue`, `dequeue`, `front` and `len`. The standard array representation is a **circular buffer**: a block of capacity C , a *head* index h , and a count n . The element at queue position i is stored at index $(h + i) \bmod C$, and

enqueue : write at $(h + n) \bmod C$, $n \leftarrow n + 1$; *dequeue* : read at h , $h \leftarrow (h + 1) \bmod C$, $n \leftarrow n - 1$.

The positions of the queue are numbered in the ring $\mathbb{Z}_C$, and both moves are arithmetic on that ring — addition modulo C — so both cost $\mathcal{O}(1)$ (amortized, with doubling when full). The circular buffer is arithmetic addressing ([CGT-DEF-009](#)) with the integers replaced by a cyclic group: the same pattern, and the first hint that *group structure* is what makes some numerations work (Chapter 13 develops this for de Bruijn graphs and hypercubes). A **deque** adds `push_front` and `pop_back` — subtraction modulo C — at the same cost.

The admissibility language of the queue, like that of the stack, is the set of words in which each `dequeue` is preceded by more `enqueue`s than `dequeue`s; ignoring values, it is the *same* language D , because emptiness depends only on the count. What distinguishes the two structures is the semantics — which value comes out — not the syntax. This is a small but genuine instance of the book’s three layers: identical admissibility, different denotations, similar costs.

EXAMPLE 3.1 (Same language, different meaning)

On `push(1) push(2) pop` the stack returns 2; on `enqueue(1) enqueue(2) dequeue` the queue returns 1. Both words are admissible from empty, and both cost a constant number of steps per call.

3.4 When lists still win

Linked structures lose to arrays on indexed access, but they have advantages that the cost columns of this chapter do not show: stable addresses (a record never moves, so external pointers to it stay valid), $\mathcal{O}(1)$ splicing of whole sublists, and no reallocation spikes. These are representation properties, not grammar properties — and they are a reminder that “which representation is best” depends on the workload, a theme made quantitative in Chapter 12.

EXERCISE 3.1

Compile the cursor word $w = next^3 prev^5 next^4$ into (net displacement, minimum prefix displacement, maximum prefix displacement). For which starting positions i in a list of length $n = 10$ is $\llbracket w \rrbracket(i)$ defined?

► Solution to [Exercise 3.1](#)

EXERCISE 3.2

Give a context-free grammar for D different from the one in the proof of [Proposition 3.2](#), and prove that it generates exactly D .

EXERCISE 3.3

Show that the set of words over $\{push, pop\}$ that are defined on a stack of *bounded* capacity K (where `push` on a full stack is undefined) is regular, and give its minimal DFA. How many states does it have?

► Solution to [Exercise 3.3](#)

EXERCISE 3.4

A queue implemented with two stacks (an “inbox” and an “outbox”) gives amortized $\mathcal{O}(1)$ operations. Write its operations as words over the stacks’ moves, and prove the amortized bound with a potential function.

SUMMARY

- In a linked list the location of element i is not a function of i ; reaching it takes i dereferences. Cursor navigation is a grammar of `next / prev` words whose denotation is determined by three numbers, but the list cannot exploit that.
 - The operation words a stack can execute from empty form the non-regular, context-free language D of Dyck prefixes, with $\binom{L}{\lfloor L/2 \rfloor}$ words of length L : admissibility can exceed finite-automaton power.
 - The circular buffer is arithmetic addressing on the ring $\mathbb{Z}_C$; queues and dequeues cost amortized $\mathcal{O}(1)$.
 - Stacks and queues share an admissibility language and differ in semantics — the three layers in miniature.
-

Hash Tables, Sets, Maps, and Tries

LEARNING OBJECTIVES

- Explain hashing as an attempt to recover the array’s address formula for keys that are not dense integers.
- Separate the *expected* cost of hashing (under an explicit randomness assumption) from its *worst-case* cost, and exhibit inputs that realize the worst case.
- Measure probe counts in a linear-probing table and relate them to the load factor.
- Describe a trie as the deterministic automaton of a key set, and say what sharing a minimal automaton would add.

4.1 Recovering an address formula

A **map** (dictionary) associates values with keys drawn from a large universe U — strings, 64-bit identifiers, tuples. A **set** is a map whose values are ignored. Chapter 2 showed what makes arrays fast: names that are dense numbers and an arithmetic map from names to locations. Keys have neither property. A table of capacity C with $n \ll |U|$ keys can only use an array if keys are first turned into slot numbers.

A **hash function** $h : U \rightarrow \{0, \dots, C - 1\}$ does exactly that. The address of key k is “computed” as $h(k)$ — but h cannot be injective when $|U| > C$, so two keys may *collide*. A collision-resolution strategy decides where the second key goes. In **linear probing** (the strategy of the SDK’s `HashTableStructure`), the slots $h(k), h(k) + 1, h(k) + 2, \dots$ (modulo C) are tried in turn until the key or an empty slot is found. Each slot inspection is a **probe**.

The grammar of a map is simple and its semantics is the abstract finite function:

MOVE	ADMISSIBLE WHEN	DENOTATION	COST (LINEAR PROBING)
<code>put(k, v)</code>	always	$f[k \mapsto v]$	1 hash, probes until k or empty, writes; occasional resize

MOVE	ADMISSIBLE WHEN	DENOTATION	COST (LINEAR PROBING)
get(k)	$k \in \text{dom } f$	$f(k)$	1 hash, probes
contains(k)	always	$[k \in \text{dom } f]$	1 hash, probes
delete(k)	$k \in \text{dom } f$	f without k	1 hash, probes, a tombstone write

As always, the first three columns are shared by every representation of maps — sorted arrays with binary search, balanced trees, tries — and only the cost column changes.

4.2 Expected versus worst-case cost

The appeal of hashing is the claim that lookups take “ $\mathcal{O}(1)$ time”. That claim has a hidden hypothesis, and it is false without it.

PROPOSITION 4.1 (Worst case of any fixed hash function) ESTABLISHED

For any fixed function $h : U \rightarrow \{0, \dots, C - 1\}$ with $|U| \geq (n - 1)C + 1$ there are n keys with the same hash value. Inserting them into a linear-probing table and then looking up the last one inserted costs n probes; inserting all of them costs $\Theta(n^2)$ probes in total.

Proof.

By the pigeonhole principle some slot value is the image of at least n keys. Inserting those keys one by one, the j -th inserted key probes the $j - 1$ occupied slots of its predecessors and then an empty one: j probes, $\sum_j j = \Theta(n^2)$ in total. A later lookup of the n -th key repeats its n probes. ■

The “ $\mathcal{O}(1)$ ” therefore requires *randomness*: either the keys are assumed to behave as if random, or h is drawn at random from a suitable family after the keys are fixed (universal hashing). Under such assumptions classical analyses bound the expected number of probes by a function of the **load factor** $\alpha = n/C$ alone [Cormen et al. 2022]. The SDK uses a fixed, deterministic hash (FNV-1a over a canonical encoding) so that every count in this book is reproducible — which means its guarantees are *empirical* for the inputs we test, not worst-case. Both behaviours are easy to exhibit:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import random
from cgtsdk import Cost
```

```

from cgtsdk.structures import HashTableStructure

rng = random.Random(4)
keys = rng.sample(range(10**12), 2000)
for label, hash_fn in (("FNV-1a (fixed)", None), ("adversarial: h(k)=0", lambda k: 0)):
    t = HashTableStructure(capacity=4096, **({} if hash_fn is None else {"hash_fn": hash_fn}))
    g = t.grammar()
    for k in keys[:500]:
        g.execute(f"put({k}, 1)", check_adequacy=False)
    c = Cost()
    for k in keys[:500]:
        c.merge(g.execute(f"get({k})", check_adequacy=False).cost)
    print(f"{label:22s} n=500 load={500/t.cap:.3f} probes per successful get = {c.counts['probe']/500:.2f}")

```

OUTPUT (CHECKED)

FNV-1a (fixed)	n=500 load=0.122	probes per successful get = 1.10
adversarial: h(k)=0	n=500 load=0.122	probes per successful get = 250.50

The same table, the same keys, the same load factor: the only difference is whether the hash function scatters the keys. With every key colliding, the average successful lookup probes $(n + 1)/2 = 250.5$ slots, exactly as [Proposition 4.1](#) predicts.

CLAIM BOUNDARY

An $\mathcal{O}(1)$ claim for hashing is a claim about an *expectation*, under a stated randomness assumption, with the load factor bounded. It is not a worst-case bound, and inputs chosen with knowledge of a fixed hash function defeat it. Treat any “constant-time lookup” claim for a hashing-based index — including any built from a grammar — with this checklist.

4.3 Load factor and resizing

Probing cost grows with the load factor, so tables **resize**: the SDK doubles capacity and rehashes every key whenever more than half the slots are used (live keys plus tombstones). The amortized analysis is the dynamic-array argument of Chapter 2 with “copy” replaced by “rehash”. Measured on random keys, probe counts behave as the theory leads one to expect — a little above one probe per successful lookup at low load, rising as $\alpha \rightarrow 1/2$:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```

import random
from cgtsdk import Cost
from cgtsdk.structures import HashTableStructure

```

```

rng = random.Random(7)
for n in (100, 300, 500, 700, 1000, 2000):
    t = HashTableStructure(capacity=2048)
    g = t.grammar()
    keys = rng.sample(range(10**9), n)
    for k in keys:
        g.execute(f"put({k}, 0)", check_adequacy=False)
    hit, miss = Cost(), Cost()
    for k in keys:
        hit.merge(g.execute(f"get({k})", check_adequacy=False).cost)
    for k in rng.sample(range(10**9, 2 * 10**9), 500):
        miss.merge(g.execute(f"contains({k})", check_adequacy=False).cost)
    print(f"n={n:5d} capacity={t.cap:5d} load={n/t.cap:.3f} hit={hit.counts['probe']/n:.2f} miss=
{miss.counts['probe']/500:.2f}")

```

OUTPUT (CHECKED)

```

n= 100 capacity= 2048 load=0.049 hit=1.00 miss=1.06
n= 300 capacity= 2048 load=0.146 hit=1.11 miss=1.23
n= 500 capacity= 2048 load=0.244 hit=1.18 miss=1.33
n= 700 capacity= 2048 load=0.342 hit=1.24 miss=1.55
n= 1000 capacity= 2048 load=0.488 hit=1.56 miss=2.47
n= 2000 capacity= 4096 load=0.488 hit=1.44 miss=2.31

```

Unsuccessful searches (`contains` on absent keys) must run to an empty slot, so they cost more and grow faster with load; at the same load factor, doubling the table leaves probe counts roughly unchanged (the small differences are between two different random key sets) — the empirical signature of a cost that depends on α and not on n .

4.4 Hashing as a grammar mechanism

In the vocabulary of this book, hashing is **arithmetic addressing with collisions**: the name k is turned into a number by computation, and the number into a location by the array formula. The density problem of Chapter 2 is solved by choosing $C = \Theta(n)$, and the price is that the address is only a *first guess*, corrected by probing. Two consequences are worth stating:

- **The grammar does not change.** A map's operations and their meanings are the same for every representation. What hashing changes is the execution layer, and its guarantees are probabilistic.
- **Perfect hashing is precomputation.** For a *static* key set one can build, in expected linear time, a hash function without collisions on that set — a construction outside this book's scope. In the classification of Chapter 12 this is mechanism M-PRE: work done once on the key set so that later queries need no probing. It inherits all the trade-offs of preprocessing: it must be rebuilt when the set changes.

4.5 Tries: keys as words

A **trie** stores a set of strings by sharing their prefixes: each node is a prefix, and the edge from prefix p to pc is labelled by the character c . Looking up a key of length ℓ follows ℓ edges, independently of how many keys are stored, and $\text{count_prefix}(p)$ — the number of keys beginning with p — is a single walk if each node stores the size of its subtree.

PROPOSITION 4.2 (A trie is a deterministic automaton)

ESTABLISHED

The trie of a finite set K of strings, with each node marked accepting iff it is a key, is a deterministic finite automaton recognizing exactly K ; its states are the prefixes of keys. The minimal DFA of K is obtained by merging states with identical futures, which shares common suffixes as well as prefixes.

Proof.

Reading w from the root follows the unique path labelled w if w is a prefix of some key and falls off otherwise; it ends in an accepting state iff $w \in K$. Two prefixes p, p' have the same future iff $\{x : px \in K\} = \{x : p'x \in K\}$; merging such states (Myhill–Nerode) yields the minimal automaton, in which identical suffix sets are represented once. ■

The trie thus makes literal the idea that a key is a *sentence* read by a machine. The minimal automaton — known in the string-processing literature as a directed acyclic word graph for dictionaries — is the first appearance of the third mechanism of this book, **sharing** (M-SHARE): repeated structure represented once. Chapter 12 measures what sharing buys for trees and what updates cost it.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk.structures import TrieStructure

g = TrieStructure(["car", "cart", "carton", "cat", "dog", "dot"]).grammar()
r = g.execute("contains('car'); contains('ca'); count_prefix('ca'); count_prefix('do')")
print(r.values, r.cost.snapshot())
```

OUTPUT (CHECKED)

```
[True, False, 4, 2] {'pointer': 9, 'read': 4}
```

Each query cost one pointer step per character plus one read: $3 + 2 + 2 + 2 = 9$ pointer steps for the four keys and prefixes of total length 9, whatever the size of the stored set.

EXERCISE 4.1

Prove that with linear probing and no deletions, the set of occupied slots after inserting a set of keys does not depend on the order of insertion (although the slot of each key may).

EXERCISE 4.2

Using the SDK, construct 30 distinct integer keys that all receive the same FNV-1a slot in a table of capacity 64 (search by brute force). What does this exercise show about the guarantees of any fixed, publicly known hash function?

► Solution to [Exercise 4.2](#)

EXERCISE 4.3

Build the minimal DFA of $K = \{\text{tap, top, tip, taps, tops}\}$ over the alphabet of letters. How many states does the trie have, and how many does the minimal DFA have?

EXERCISE 4.4

A sorted array with binary search, a balanced search tree, a hash table and a trie all implement the same map grammar. For each, give the worst-case cost of `get` and of `put` and say which ones support `count_prefix` efficiently.

SUMMARY

- Hashing recovers an address formula for keys that are not dense integers, at the price of collisions resolved by probing.
- “ $\mathcal{O}(1)$ hashing” is an expectation under a randomness assumption with bounded load; any fixed hash function has inputs costing $\Theta(n)$ per lookup, and the SDK demonstrates them.
- Probe counts depend on the load factor, not on n ; resizing keeps the load bounded at amortized constant cost.
- A trie is the deterministic automaton of its key set; the minimal automaton additionally shares suffixes — the first instance of the sharing mechanism.

Trees, Heaps, Search Trees, and Forests

LEARNING OBJECTIVES

- Name every node of a binary tree by its address, a word over $\{0, 1\}$, and evaluate addresses by pointer walks or by arithmetic.
- State and prove the exact scope of arithmetic tree addressing: when the heap numeration gives constant-time navigation and when it does not.
- Compare arithmetic addressing with the preprocessing approach to lowest common ancestors.
- Explain why rotations in balanced search trees invalidate addresses.
- Compress forests by sharing identical subtrees, and prove how adversarial updates destroy that sharing.

5.1 Addresses are words

A **rooted binary tree** is a structure whose elements are nodes, with partial functions *left*, *right* and *parent* and a distinguished root. Write 0 for a step to the left child and 1 for a step to the right child. Every node x is reached from the root by exactly one word over $\{0, 1\}$, its **address** $\text{addr}(x)$; the root's address is the empty word ε .

DEFINITION 5.1 (Grammatical address and numeration)

CGT-DEF-006

Let a structure have a root r and deterministic moves Σ . The **address** of an element x is the shortlex-least word $w \in \Sigma^*$ with $\llbracket w \rrbracket(r) = x$. A **numeration** is an injective map ν from addresses to $\mathbb{N}$; it is **arithmetic** if each move acts on numerals by a map computable in $\mathcal{O}(\lceil b/w \rceil)$ word operations on b -bit numerals. Its **density** is the number of elements divided by $1 + \max \nu$.

For trees the moves are L, R, U (left, right, up), and the address of a node is its unique root-to-node word over $\{L, R\}$ — written over $\{0, 1\}$. The set of addresses of a tree is **prefix-closed**: if w is an address so is every prefix of w . Conversely, every finite prefix-closed set of binary words is the address set of exactly one binary tree. A tree is its language of addresses.

This is the formal counterpart of the observation from which the research program began: an address that identifies a state and determines how to reach it is a word in a small language, and *evaluating* the word is computation.

5.2 The heap numeration

How cheaply can an address be evaluated? In a pointer representation, evaluating w costs $|w|$ pointer hops. The **heap numeration** does better on some trees:

$$\nu(w) = \text{int}(1w), \quad \text{i.e.} \quad \nu(\varepsilon) = 1, \quad \nu(w0) = 2\nu(w), \quad \nu(w1) = 2\nu(w) + 1, \quad \nu(\text{parent}) = \lfloor \nu/2 \rfloor. \quad (5.1)$$

The moves become arithmetic: left child is doubling, right child is doubling plus one, parent is halving. This is the indexing of the binary heap introduced by Williams [Williams 1964], in which a complete binary tree with n nodes occupies exactly the array slots $1, \dots, n$. The SDK represents a tree either way; the costs differ:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk.structures import BinaryTreeStructure

for shape, make in (("complete, n=1023", lambda rep: BinaryTreeStructure.complete(1023, rep)),
                    ("path, n=1000", lambda rep: BinaryTreeStructure.path(1000, "01", rep))):
    for rep in ("pointer", "heap"):
        t = make(rep)
        deepest = max(t.labels, key=len)
        r = t.grammar().execute(f"goto('{deepest}') read")
        print(f"{shape} {rep:7s} depth={len(deepest):4d} cost={r.cost.total:5d} {r.cost.snapshot()} density={t.density:.3g}")
```

OUTPUT (CHECKED)

```
complete, n=1023 pointer depth= 9 cost= 11 {'pointer': 9, 'read': 2} density=1
complete, n=1023 heap depth= 9 cost= 3 {'arith': 1, 'probe': 2} density=1
path, n=1000 pointer depth= 999 cost= 1001 {'pointer': 999, 'read': 2} density=9.33e-299
path, n=1000 heap depth= 999 cost= 18 {'arith': 16, 'probe': 2} density=9.33e-299
```

On the complete tree the heap numeration reaches the deepest node in three steps and has density 1: an array of exactly n slots suffices. On the path-shaped tree the numerals have 1000

bits; each arithmetic operation costs $\lceil 1000/64 \rceil = 16$ word steps, still much better than 999 pointer hops. But the density is 9×10^{-299} : an array indexed by numerals would need 2^{1000} slots, and the SDK has to fall back to a hash table keyed by numerals (the probe steps). Arithmetic navigation has not become free; it has moved the cost into the *size of the address space*.

5.3 The scope of arithmetic tree addressing

The following theorem states exactly when the heap numeration gives constant-time navigation. Its components are classical; the statement of scope is this book's.

THEOREM 5.2 (Scope of arithmetic tree addressing)

ADAPTED

CGT-THM-003

Let T be a binary tree of height h with n nodes, labelled by heap numerals. On a word RAM with a most-significant-bit instruction:

1. (Operations.) Parent, children, depth, the ancestor test and the lowest common ancestor are computable from the numerals alone in $\mathcal{O}(\lceil (h+1)/w \rceil)$ word operations.
2. (Address space.) Any table indexed directly by numerals has $2^{h+1} - 1$ slots; the density is $n/(2^{h+1} - 1)$, which exceeds $1/2$ for complete trees and is $2^{-\Theta(n)}$ for path-shaped trees.
3. (Scope.) Since $h \geq \lfloor \log_2 n \rfloor$, the label-only operations are $\mathcal{O}(1)$ if and only if $h = \mathcal{O}(w)$; with $w = \Theta(\log n)$, if and only if $h = \mathcal{O}(\log n)$.
4. (Instability.) A single rotation at a node v changes the address, hence the numeral, of every node in the subtree of v .

Proof.

(1) The child and parent formulas are (5.1). Node u is an ancestor of v iff $\text{addr}(u)$ is a prefix of $\text{addr}(v)$, iff shifting $\nu(v)$ right by $\text{depth}(v) - \text{depth}(u)$ yields $\nu(u)$, where $\text{depth}(x) = \text{msb}(\nu(x))$. For the LCA, first shift the deeper numeral right until both have the same length; this replaces the deeper node by its ancestor at the other's depth and does not change the LCA. Two numerals of equal length agree exactly on the bits above the most significant bit of their exclusive-or, so the LCA is the common numeral shifted right by $\text{msb}(\nu_u \oplus \nu_v) + 1$ (with $\text{msb}(0) = -1$). Each step is a constant number of operations on $(h + 1)$ -bit integers.

(2) Numerals range over $[1, 2^{h+1} - 1]$. A complete tree has $n \geq 2^h$; a path has $h = n - 1$.

(3) follows from (1) and (2).

(4) Let v have address p and left child ℓ ; a right rotation makes ℓ the root of the subtree. Afterwards ℓ has address p (before $p0$); v has $p1$ (before p); a node of ℓ 's left subtree with address $p00x$ gets $p0x$; a node of ℓ 's right subtree with address $p01x$ gets $p10x$; a node of v 's right subtree with address $p1x$ gets $p11x$. Every word changes. ■

The theorem draws a clean line. Arithmetic addressing wins only where it needs *no* preprocessing and *no* large tables — on dense, shallow shapes such as heaps. Everywhere else, the competitor is **precomputation**: an Euler tour of the tree with a range-minimum structure answers LCA queries in $\mathcal{O}(1)$ for *every* shape after $\mathcal{O}(n)$ preprocessing [Bender & Farach-Colton 2000; Harel & Tarjan 1984; Schieber & Vishkin 1988]. The book's first experiment measured all three approaches ([CGT-EXP-001](#)): pointer walking costs grow with depth, heap arithmetic is constant (4 word operations) as long as $h < 64$ and grows as $\lceil (h + 1)/64 \rceil$ beyond, and the Euler-tour index answers in 3 operations for every shape after $\Theta(n \log n)$ build steps in the simple sparse-table variant.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import Cost
from cgtsdk.algorithms import EulerLCA, NaiveLCA, heap_lca
from cgtsdk.lab import random_parent_array
import random

rng = random.Random(3)
for shape in ("complete", "random", "path"):
    par = random_parent_array(4096, 5, shape)
    b1, b2 = Cost(), Cost()
    naive, euler = NaiveLCA(par, b1), EulerLCA(par, b2)
    q1, q2, q3 = Cost(), Cost(), Cost()
    pairs = [(rng.randrange(4096), rng.randrange(4096)) for _ in range(500)]
    for u, v in pairs:
        a = naive.query(u, v, q1); b = euler.query(u, v, q2)
        assert a == b
    if shape == "complete":
        assert heap_lca(u + 1, v + 1, q3) - 1 == a
```

```

heap = f"{q3.total/500:6.1f}" if shape == "complete" else "  n/a"
print(f"{shape:8s} per query: walk {q1.total/500:7.1f} euler {q2.total/500:4.1f} (build {b2.total}) heap-arith {heap}")

```

OUTPUT (CHECKED)

```

complete per query: walk   19.0 euler  6.0 (build 188419) heap-arith   4.0
random   per query: walk   14.5 euler  6.0 (build 188419) heap-arith  n/a
path     per query: walk 1329.2 euler  6.0 (build 188419) heap-arith  n/a

```

(The Euler query is charged 6 steps here — three arithmetic operations, two reads, one comparison — under the SDK’s finer cost model.) On the complete tree, heap arithmetic needs no build at all; the Euler index needs about 188,000 counted build steps (tour plus sparse table) before its first query. On the other two shapes heap labels would need $h + 1$ bits, which for the path is 4,096 bits per label — and the experiment does not even try.

5.4 Search trees and rotations

A **binary search tree** stores keys so that every key in a node’s left subtree is smaller and every key in its right subtree larger. Search, insertion and deletion follow one root-to-leaf path, so they cost $\Theta(h)$. Insertion order determines h : sorted insertions build a path ($h = n - 1$). **AVL trees** [Adelson-Velsky & Landis 1962] restore balance after each update by *rotations*, guaranteeing $h = \mathcal{O}(\log n)$:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```

from cgtsdk.structures import BSTStructure

keys = list(range(1, 1024))
for balance in ("none", "avl"):
    t = BSTStructure(keys, balance)
    r = t.grammar().execute("contains(1023); contains(512); contains(2000)")
    print(f"{balance:4s} height={t.tree_height():5d} three searches cost {r.cost.total:5d} {r.cost.snapshot()}")

```

OUTPUT (CHECKED)

```

none height= 1023 three searches cost 7670 {'compare': 5114, 'pointer': 2556}
avl  height=   10 three searches cost   59 {'compare': 40, 'pointer': 19}

```

The rotations that make this possible are exactly the operations that part (4) of [Theorem 5.2](#) shows to be fatal for addresses: every node in a rotated subtree changes its address. Balanced

search trees and address-based indexes are therefore in tension. A search tree navigates by *comparison* — its grammar is “go left if smaller” — and does not need stable addresses; an address-based scheme needs stable addresses and cannot afford rotations. The research ledger records this as a counterexample to “address-based indexes are stable” (CGT-NEG-003).

A third family of representations deserves mention: **succinct** trees, which encode the shape of an n -node tree in $2n + o(n)$ bits — within lower-order terms of the information-theoretic minimum — while supporting navigation in constant time [Jacobson 1989]. They achieve for arbitrary shapes what the heap numeration achieves only for complete ones, by storing a small amount of precomputed rank information (M-PRE) next to a compact encoding of the shape.

5.5 Forests and shared structure

A **forest** is a sequence of trees. Forests of syntax trees, XML documents and file systems are often highly repetitive: the same subtree occurs many times. The simplest *structural grammar* of a tree represents each distinct subtree once.

DEFINITION 5.3 (Minimal DAG of a tree)

CGT-DEF-004

The **minimal DAG** of a labelled ordered tree has one node per distinct subtree; the node for a subtree with root label a and children subtrees $t_1, \dots, t_k$ points to the nodes of $t_1, \dots, t_k$. Equivalently, it is a grammar with one nonterminal X_t per distinct subtree and the production $X_t \rightarrow a(X_{t_1}, \dots, X_{t_k})$.

The minimal DAG is computed by *hash-consing*, bottom-up, in expected linear time; its use for common subexpressions goes back at least to Downey, Sethi and Tarjan [Downey et al. 1980], and for compressing XML structure to Buneman, Grohe and Koch [Buneman et al. 2003]. A full binary tree of height h with identical labels has $2^{h+1} - 1$ nodes but a minimal DAG of $h + 1$ nodes: exponential compression. Navigation by preorder rank still works without decompression, by storing each DAG node’s expanded size and descending, at a cost of $\mathcal{O}(\text{depth} \times \text{degree})$ instead of $\mathcal{O}(1)$ in an explicit array (CGT-EXP-004).

Sharing, however, is fragile.

THEOREM 5.4 (Update inflation of shared trees)**PROVED HERE**

CGT-THM-005

Let T_h be the full binary tree of height h with identical labels, $n = 2^{h+1} - 1$; its minimal DAG has $h + 1$ nodes.

1. Relabelling one node at depth d with a fresh label creates exactly $d + 1$ new DAG nodes, and the minimal DAG of the result has $h + d + 1$ nodes.
2. For every $k \leq 2^h$ there are k leaves whose relabelling with pairwise distinct fresh labels yields a minimal DAG with at least $k(h + 1 - \lceil \log_2 k \rceil)$ nodes; since each relabelling adds at most $h + 1$ nodes, the size is $\Theta(k \log n)$ for $k \leq \sqrt{n}$.
3. The unary path of n nodes with one label has a minimal DAG of n nodes, though its label string a^n has a straight-line program with $\mathcal{O}(\log n)$ rules.

Proof.

(1) The relabelled node and its d ancestors are the only nodes whose subtrees contain the fresh label, so their $d + 1$ subtrees are new and pairwise distinct (they have different heights). Every other subtree is a full tree of some height below h , and all heights $0, \dots, h - 1$ still occur; the full tree of height h no longer does. Total: $h + d + 1$.

(2) Choose leaves whose root paths pass through k distinct nodes at every depth $j \geq \lceil \log_2 k \rceil$. Every node on the union U of these paths has a non-empty set of fresh labels below it; nodes of U at the same depth have disjoint such sets, and nodes at different depths have subtrees of different heights, so all $|U| \geq k(h + 1 - \lceil \log_2 k \rceil)$ subtrees are distinct.

(3) Subtrees of the unary path have pairwise distinct sizes. For a^n , the rules $X_0 \rightarrow a$, $X_{i+1} \rightarrow X_i X_i$ together with the binary expansion of n give $\mathcal{O}(\log n)$ rules. ■

Part (3) shows that minimal DAGs are not the most succinct structural grammars: tree straight-line programs can compress repetition that is not a repeated *complete subtree*, and can be exponentially more succinct than DAGs [Lohrey 2015]. The experiment of the ledger (CGT-EXP-004) saw both effects: a full binary tree with 65,535 nodes went from a 16-node DAG to 1,521 live nodes after 200 random relabels, and a unary path of 65,536 nodes did not compress at all.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import Cost
from cgtsdk.sharing import DagGrammar, Explicit, full_binary

F = Explicit(); F.roots.append(full_binary(F, 10))
D = DagGrammar.from_explicit(F)
print("n =", F.n, " minimal DAG nodes =", D.live_size())
c = Cost()
for depth_rank in (0, 1, 1000, 2046):          # preorder ranks of a few nodes
```

```
created = D.relabel(depth_rank, f"fresh{depth_rank}", c)
print(f"relabel rank {depth_rank:4d}: new rules {created:2d}, live DAG nodes {D.live_size()}")
```

OUTPUT (CHECKED)

```
n = 2047  minimal DAG nodes = 11
relabel rank    0: new rules  1, live DAG nodes 11
relabel rank    1: new rules  2, live DAG nodes 12
relabel rank 1000: new rules 11, live DAG nodes 21
relabel rank 2046: new rules 11, live DAG nodes 30
```

Relabelling the root (depth 0) creates one rule and leaves the DAG size at $h + 0 + 1 = 11$; relabelling rank 1 (depth 1) creates two rules. The later relabels act on a tree that already contains fresh labels, so part (1) no longer applies exactly, but each still adds at most $h + 1 = 11$ rules, and the DAG grows towards the $\Theta(k \log n)$ of part (2).

EXERCISE 5.1

Compute the heap numerals of the nodes with addresses ε , 0, 011 and 1101, and use the XOR method of [Theorem 5.2](#) to find the LCA of 011 and 0101.

► [Solution to Exercise 5.1](#)

EXERCISE 5.2

Prove that every finite prefix-closed set of binary words is the address set of exactly one binary tree.

EXERCISE 5.3

A k -ary heap numbers the children of node x as $kx - (k - 2), \dots, kx + 1$ (1-based). Derive the parent formula and the analogue of part (2) of [Theorem 5.2](#).

EXERCISE 5.4

Using the SDK's `BSTStructure`, insert the keys $1, \dots, n$ in sorted order and in random order for $n = 2^{10}$, and compare the total comparison counts of all insertions with and without AVL balancing. Relate the numbers to the height bounds.

EXERCISE 5.5

Give a forest of n nodes whose minimal DAG has $\Theta(n)$ nodes even though every tree in it is isomorphic, as an unlabelled tree, to every other. What property of the labels causes this?

SUMMARY

- A tree is its prefix-closed language of addresses; evaluating an address is computation, by pointer walk or by arithmetic.
 - The heap numeration makes navigation $\mathcal{O}(1)$ exactly when the height is $\mathcal{O}(w)$ (dense, shallow shapes); otherwise numerals are long and tables astronomically sparse (CGT-THM-003).
 - Precomputation (Euler tour + RMQ) gives $\mathcal{O}(1)$ LCA on every shape after linear preprocessing; arithmetic wins only where it needs no preprocessing.
 - Rotations, the tool of balanced search trees, change every address in the rotated subtree.
 - Minimal DAGs compress repeated subtrees exponentially, but k adversarial relabels inflate them to $\Theta(k \log n)$ nodes, and they miss repetition that tree straight-line programs capture (CGT-THM-005).
-

Graphs and Paths

LEARNING OBJECTIVES

- Compare adjacency-matrix and adjacency-list representations by the cost of their basic operations.
- Read a labelled graph as a family of relations, and words of labels as compositions of relations.
- Prove that the meaning of a word of moves is the composition of the meanings of its letters, and that languages of words denote unions.
- Distinguish walks from simple paths, and explain why that semantic choice moves a problem from polynomial time to NP-hardness.
- Relate paths-as-grammar to Tarjan's path expressions and to semiring path problems.

6.1 Graphs and their representations

A **directed graph** $G = (V, E)$ is a structure with one binary relation $E \subseteq V \times V$. A **labelled** graph has one relation E_a per label a in a finite alphabet Σ ; a **weighted** graph attaches a number to each edge. Throughout, $n = |V|$ and $m = |E|$.

Two representations dominate:

OPERATION	ADJACENCY MATRIX (N^2 BITS)	ADJACENCY LISTS ($\mathcal{O}(N + M)$ WORDS)
test whether $(u, v) \in E$	$\mathcal{O}(1)$: one bit at address $un + v$	$\mathcal{O}(\text{outdeg}(u))$, or $\mathcal{O}(1)$ expected with hashing
list the out-neighbours of u	$\Theta(n)$	$\Theta(\text{outdeg}(u))$
breadth-first search from s	$\Theta(n^2)$	$\Theta(n + m)$

OPERATION	ADJACENCY MATRIX (N^2 BITS)	ADJACENCY LISTS ($\mathcal{O}(N + M)$ WORDS)
space	$\Theta(n^2)$ bits	$\Theta(n + m)$ words

The matrix applies the array’s address formula to *pairs* of vertices (row-major numeration of $V \times V$, Chapter 2), so edge tests are arithmetic; but it pays n^2 bits whether or not the graph is sparse, and every traversal touches whole rows. Lists pay only for edges that exist. Neither is better in general — the familiar shape of every comparison in this book.

6.2 Labelled graphs as relations

Read each label a as a **move** whose meaning is the relation E_a : from vertex u , the move a may lead to any v with $(u, v) \in E_a$. A *word* of labels $a_1 a_2 \cdots a_k$ then means “follow an a_1 edge, then an a_2 edge, ...”, and its meaning is a composition of relations.

DEFINITION 6.1 (Relational semantics of move words)

CGT-DEF-002

Given relations $\llbracket a \rrbracket \subseteq V \times V$ for each $a \in \Sigma$, extend $\llbracket \cdot \rrbracket$ to words and languages by

$$\llbracket \varepsilon \rrbracket = \text{id}_V, \quad \llbracket uw \rrbracket = \llbracket u \rrbracket ; \llbracket w \rrbracket, \quad \llbracket \mathcal{L} \rrbracket = \bigcup_{w \in \mathcal{L}} \llbracket w \rrbracket,$$

where $R ; S = \{(x, z) : \exists y. (x, y) \in R \wedge (y, z) \in S\}$ is relational composition (first R , then S). For a vertex x , write $\llbracket w \rrbracket(x) = \{y : (x, y) \in \llbracket w \rrbracket\}$.

PROPOSITION 6.2 (Compositional semantics)

ESTABLISHED

CGT-PROP-001

$\llbracket \cdot \rrbracket$ is a monoid homomorphism from $(\Sigma^*, \cdot, \varepsilon)$ to $(\text{Rel}(V), ;, \text{id}_V)$. For languages, $\llbracket \mathcal{L}_1 \mathcal{L}_2 \rrbracket = \llbracket \mathcal{L}_1 \rrbracket ; \llbracket \mathcal{L}_2 \rrbracket$, $\llbracket \mathcal{L}_1 \cup \mathcal{L}_2 \rrbracket = \llbracket \mathcal{L}_1 \rrbracket \cup \llbracket \mathcal{L}_2 \rrbracket$, and $\llbracket \mathcal{L}^* \rrbracket = \bigcup_{k \geq 0} \llbracket \mathcal{L} \rrbracket^k$, the reflexive-transitive closure of $\llbracket \mathcal{L} \rrbracket$.

Proof.

The extension to words is by composition, and relational composition is associative with identity id_V , so $\llbracket uw \rrbracket = \llbracket u \rrbracket ; \llbracket w \rrbracket$ for all words by induction on $|u|$. Composition distributes over arbitrary unions on both sides, which gives the identities for concatenation and union of languages, and hence for $\mathcal{L}^* = \bigcup_k \mathcal{L}^k$. ■

The proposition turns the recursive slogan “a path from u to v is an edge from u to some w followed by a path from w to v ” into an identity:

$$\llbracket EE^* \rrbracket = \llbracket E \rrbracket ; \llbracket E^* \rrbracket. \quad (6.1)$$

Paths really are grammatical objects: the set of walks is the language EE^* (or Σ^+ for labelled graphs), and its meaning is the transitive closure. This is the relational reading underlying Kleene algebra, and Tarjan's **path expressions** make it algorithmic: a regular expression over the edges that represents *all* paths from a source to each vertex, from which shortest paths, path counts and dataflow facts can be computed by interpreting the expression in a suitable algebra [Tarjan 1981a; Tarjan 1981b]. Tarjan showed that constructing path expressions is, in a precise sense, the most general path problem.

CLAIM BOUNDARY

Proposition 6.2 is a statement about **semantics**. It says the recursive description of paths is *correct*; it says nothing about how *cheaply* a path can be found. Chapter 10 shows that evaluating the recursive description is exactly the work of computing a closure, and that two grammars with the same meaning can differ asymptotically in cost (CGT-THM-004).

The SDK implements this relational semantics directly: the state of `GraphStructure` is a *set* of current vertices, and a label move maps the set to its image.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk.structures import GraphStructure

edges = [(0, "a", 1), (0, "a", 2), (1, "b", 3), (2, "b", 3), (3, "a", 0), (2, "c", 4)]
g = GraphStructure(5, edges).grammar()
r = g.execute("at(0) a here b here a here c")
print(r.values, "| defined:", r.defined, "| undefined at call", r.undefined_at, "-", r.reason)
```

OUTPUT (CHECKED)

```
[None, None, [1, 2], None, [3], None, [0]] | defined: False | undefined at call 7 -  $\llbracket c \rrbracket$  of the current set is empty
```

From $\{0\}$, the move *a* reaches $\{1, 2\}$, then *b* reaches $\{3\}$, then *a* returns to $\{0\}$; the final *c* has no edge from 0, so the expression is undefined *at that call* — the semantic layer again, with its reason.

6.3 Walks, simple paths, and a change of meaning

A **walk** may repeat vertices; a **simple path** may not. The distinction looks minor. It is not.

PROPOSITION 6.3 (Walk semantics versus simple-path semantics) ESTABLISHED

Let $\mathcal{L}$ be a regular language over the labels, given by an automaton with $|Q|$ states. Deciding whether some walk from s to t has its label word in $\mathcal{L}$ takes $\mathcal{O}(|Q|(|\Sigma|n + m))$ time. Deciding whether some simple path from s to t has its label word in $\mathcal{L}$ is NP-complete for some fixed regular languages $\mathcal{L}$ [Mendelzon & Wood 1995], and remains NP-hard on very restricted graph classes [Barrett et al. 2000].

The first half is the product construction of Chapter 8 (CGT-THM-001); the second is a theorem of Mendelzon and Wood. Nothing about the *syntax* of the query changed — the same regular expression, the same graph. What changed is the **semantics**: which sequences of edges count as a path. That one change moves the problem across the boundary between polynomial time and NP-completeness (assuming $P \neq NP$). It is the clearest example in this book of why the semantic layer must be stated precisely before any claim about cost is made.

6.4 Path problems as algebra

Many path problems differ only in *how* the paths in $\llbracket E^* \rrbracket$ are aggregated:

PROBLEM	VALUE OF A PATH	COMBINE ALTERNATIVES	ALGEBRA
reachability	true	or	Boolean semiring ($\{0, 1\}, \vee, \wedge$)
shortest path	sum of weights	min	tropical semiring ($\mathbb{R} \cup \{\infty\}, \min, +$)
widest path	min of capacities	max	($\mathbb{R}, \max, \min$)
number of paths (DAG)	1	+	($\mathbb{N}, +, \times$)

Interpreting the path grammar in each algebra gives each problem; this is the unifying idea behind Tarjan’s path algebra and the closed-semiring treatment of shortest paths in textbooks [Cormen et al. 2022]. For CGT the point is precise: the **grammar** of paths (EE^*) is common to all of them; the **semantics** is chosen by the algebra; the **execution** cost depends on the algebra’s

properties (Dijkstra’s algorithm needs non-negative weights; Bellman–Ford does not). Chapter 10 implements each.

EXERCISE 6.1

For the graph of the SDK example above, compute $\llbracket (ab)^* \rrbracket(0)$ and $\llbracket a(ba)^* \rrbracket(0)$ by hand, using Proposition 6.2.

► Solution to [Exercise 6.1](#)

EXERCISE 6.2

Show that relational composition does *not* distribute over intersection: give relations R, S, T with $R ; (S \cap T) \neq (R ; S) \cap (R ; T)$. What does this imply for a “language” built with intersection?

EXERCISE 6.3

Explain why the walk/simple-path dichotomy does not arise for the language Σ^* (plain reachability): show that if a walk from s to t exists, a simple path exists.

EXERCISE 6.4

In the counting semiring, $\llbracket E^* \rrbracket$ may be infinite on graphs with cycles. Which property of DAGs makes path counting well defined, and how is it computed in $\mathcal{O}(n + m)$ time?

SUMMARY

- Adjacency matrices apply the address formula to vertex pairs; adjacency lists pay only for existing edges. Neither dominates.
 - Labels are moves whose meanings are relations; words mean compositions and languages mean unions ([CGT-PROP-001](#)). “A path is an edge followed by a path” is the identity $\llbracket EE^* \rrbracket = \llbracket E \rrbracket ; \llbracket E^* \rrbracket$.
 - This is a semantic fact, not a cost bound.
 - Changing the semantics from walks to simple paths makes regular path queries NP-complete: the syntax is unchanged, the meaning is not.
 - Path problems share a grammar and differ by the algebra in which it is interpreted.
-

Operational Grammars

LEARNING OBJECTIVES

- Define an operational grammar and its three layers precisely.
- Define what it means for a representation to be *adequate* for a grammar, and why the SDK tests adequacy.
- Distinguish operational grammars (which generate operations over a structure) from structural grammars (which generate the structure).
- Write the grammar of each structure of Part I in one common format.
- State the five mechanisms of advantage and the mechanism thesis, and say why the thesis is a conjecture.
- Locate the definitions with respect to labelled transition systems, Kleene algebra with tests, attribute grammars and automatic structures.

7.1 The object

Part I described structures one at a time. Each had a set of operations (“moves”), a meaning for each move, a notion of which sequences of moves are allowed, and a representation with costs. The following definition collects these components.

An **operational grammar** is a tuple

$$G = (\Sigma, \mathfrak{A}, \llbracket \cdot \rrbracket, \mathcal{L})$$

where $\mathfrak{A}$ is a computational structure with set of states A ; Σ is a finite **move signature**, each move with an arity; $\llbracket a \rrbracket \subseteq A \times A$ is the **relational semantics** of move a (for moves with arguments, one relation per argument tuple); and $\mathcal{L} \subseteq \Gamma^*$, for a subset $\Gamma \subseteq \Sigma$, is the **admissibility language**, given by a finite description (a regular expression, an automaton, a context-free grammar). The class of G is the class of $\mathcal{L}$ (regular, context-free, ...).

A word $w \in \Sigma^*$ (with arguments) is an **operation expression**. It is

- **syntactically admissible** if its moves are in Σ with correct arities and its projection $\pi_\Gamma(w)$ onto Γ lies in $\mathcal{L}$;
- **semantically defined at x** if $\llbracket w \rrbracket(x) \neq \emptyset$, where $\llbracket w \rrbracket$ is the composition of Definition 6.1;
- **executed** by a representation at a cost given by a cost profile (Definition 7.4).

Three remarks fix the reading.

1. **Admissibility is a projection.** The language constrains only the moves it mentions; observer moves such as `here` or `read` are free unless named. This is a definitional choice (CGT-D-015); without it, every constraint would have to enumerate every observer.
2. **Admissibility and definedness are different.** `pop` on an empty stack is admissible under the universal language and undefined; `(a b)* a` may reject `a b` although $\llbracket ab \rrbracket(x)$ is non-empty. Syntax is decided by $\mathcal{L}$ without looking at the state; semantics depends on the state.
3. **The finite description of $\mathcal{L}$ is a conventional formal grammar** (Σ, N, P, S) or an automaton. What the definition adds is the structure-dependent semantics $\llbracket \cdot \rrbracket$ and, in Definition 7.4, a cost model.

7.2 Adequate representations

A representation implements the moves on concrete memory. It is correct when it computes what the semantics says.

DEFINITION 7.2 (Adequacy)

A representation ρ of a structure with grammar G is **adequate** if for every state x and every expression w : ρ executes w to completion iff $\llbracket w \rrbracket(x) \neq \emptyset$, the values it returns at each call are those of the semantics, and when w is undefined, ρ reports undefinedness at the first call i at which the prefix $w_{\leq i}$ becomes undefined.

Adequacy is the standard notion of a correct implementation of an abstract data type (an abstraction function relating concrete and abstract states). The SDK makes it executable: every structure has a pure reference semantics `denote` on its abstract value and an instrumented `apply` on its representation, and `execute` compares them call by call. The test suite checks adequacy for every structure on thousands of random expressions, including undefined ones (CGT-D-014).

Here is the whole interface, used to define a new structure from scratch: a counter that can be incremented and decremented within bounds, with an admissibility language forbidding two decrements in a row.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import Cost, Inadmissible, Move, Structure

class BoundedCounter(Structure):
    """A counter in [0, K]. Abstract value: the integer."""
    name = "counter"
    SIGNATURE = (Move("inc", (), "add one"), Move("dec", (), "subtract one"), Move("value", (), "read"))

    def __init__(self, K, start=0):
        self.K, self.v = K, start

    def abstract(self):
        return (self.K, self.v)

    @classmethod
    def denote(cls, state, call):
        K, v = state
        if call.name == "inc":
            if v == K: raise Inadmissible("counter at its maximum")
            return (K, v + 1), None
        if call.name == "dec":
            if v == 0: raise Inadmissible("counter at zero")
            return (K, v - 1), None
        return state, v

    def apply(self, call, cost):
        cost.add("compare", 1)
        if call.name == "value":
            cost.add("read", 1); return self.v
        step = 1 if call.name == "inc" else -1
```

```

    if not 0 ≤ self.v + step ≤ self.K:
        raise Inadmissible("counter at its maximum" if step > 0 else "counter at zero")
    cost.add("write", 1); self.v += step

for expr in ["inc inc dec value", "inc dec dec", "dec", "inc inc inc inc", "inc value inc dec value"]:
    g = BoundedCounter(K=3).grammar(admissible="(inc | dec inc)* (dec | ε)" # fresh state each time
    r = g.execute(expr)
    status = (f"syntax: {r.syntax.error}" if not r.syntax.ok else
              f"undefined at call {r.undef_at}: {r.reason}" if not r.defined else f"values {r.values}")
    print(f"{expr:24s} → {status} [adequate={r.adequate}]")

```

OUTPUT (CHECKED)

```

inc inc dec value      → values [None, None, None, 1] [adequate=True]
inc dec dec            → syntax: move word inc dec dec is not in the admissibility language [adequate=None]
dec                   → undefined at call 0: counter at zero [adequate=True]
inc inc inc inc        → undefined at call 3: counter at its maximum [adequate=True]
inc value inc dec value → values [None, 1, None, None, 1] [adequate=True]

```

Execution mutates the representation, so each expression starts from a fresh counter at zero. The second expression is rejected *before* execution because `dec dec` is not admissible; the third is admissible and undefined (the counter starts at zero); the fourth runs off the top. The last shows projection at work: `value` is not mentioned by the language, so it does not interfere with it.

7.3 The grammar of each structure

With the definition in hand, the structures of Part I can be written in one format. The table is a summary of Chapters 2–6; the SDK implements each row.

STRUCTURE	STATES	MOVES Σ	DEFINED WHEN	ADMISSIBILITY CLASS (FROM EMPTY)	TYPICAL REPRESENTATION AND MOVE COST
array	sequences	get(i) , set(i,x) , insert , delete , append , pop , len	index in range	universal	contiguous: $\mathcal{O}(1)$ access, $\Theta(n - i)$ shifts
linked list	sequences + cursor	get(i) ... or next , prev	index in range /	universal	records: $\Theta(i)$ access

STRUCTURE	STATES	MOVES Σ	DEFINED WHEN	ADMISSIBILITY CLASS (FROM EMPTY)	TYPICAL REPRESENTATION AND MOVE COST
			neighbour exists		
stack	sequences	push(x) , pop , top , empty	non- empty for pop , top	context-free, not regular (Dyck prefixes)	array: amortized $\mathcal{O}(1)$
queue / deque	sequences	enqueue , dequeue , ...	non- empty	same language as the stack	ring buffer on $\mathbb{Z}_C$: amortized $\mathcal{O}(1)$
map / set	finite functions	put , get , contains , delete	key present for get , delete	universal	hashing: $\mathcal{O}(1)$ <i>expected</i> ; trie: $\mathcal{O}(k)$
heap	multisets	push , pop_min , peek	non- empty	Dyck-type, as for stacks	array with heap numeration: $\mathcal{O}(\log n)$
binary tree	trees + cursor	L , R , U , goto(w) , read	target node exists	universal; addresses form a prefix- closed language	pointers: $\Theta(w)$; heap numerals: $\mathcal{O}(\lceil h/w \rceil)$
search tree	sorted sets	insert , delete , contains , min	key present for delete	universal	$\Theta(h)$; AVL: $\mathcal{O}(\log n)$
labelled graph	vertex sets	one move per label, at(v) , here	image non- empty	any regular / context-free $\mathcal{L}$ by choice	adjacency lists: $\Theta(\text{edges inspected})$
automaton	states	one move per letter	transition defined	—	table: $\mathcal{O}(1)$ per letter

The table makes visible what is common — every structure is a set of states with partial moves — and what is not: the admissibility class (universal, regular, context-free), the dependence of definedness on state, and above all the cost column.

7.4 Structural grammars

An operational grammar generates *operations over* a structure. A second kind of grammar generates *the structure itself*.

DEFINITION 7.3 (Structural grammar)

CGT-DEF-004

A **structural grammar** for a structure $\mathfrak{A}$ is a grammar whose unique derivation (or unique generated object) is $\mathfrak{A}$ or its representation: a minimal DAG of a tree, a straight-line program for a string, a tree straight-line program, a hyperedge-replacement grammar for a graph. Its size is the total length of its productions.

The two kinds combine: navigating a grammar-compressed tree is an operational grammar whose states are given by a structural grammar (Chapter 5, [CGT-THM-005](#)). Grammar-based compression is an established field [Charikar et al. 2005; Lohrey 2015; Bille et al. 2011]; random access into a grammar-compressed string of length N takes $\mathcal{O}(\log N)$ time [Bille et al. 2011], and lower bounds show that for polynomial-space structures this cannot in general be improved to constant time [Verbin & Yu 2013]. This book uses these objects and does not claim new compression results.

7.5 Cost profiles

DEFINITION 7.4 (Resolution scheme and cost profile)

CGT-DEF-005

For a class of structures and a family of queries, a **resolution scheme** is a triple of algorithms (Build, Query, Update) on a fixed machine model. Its **cost profile** is the tuple

$$\mathcal{C} = (T_{\text{build}}, S, T_{\text{query}}, T_{\text{update}}, |\text{out}|),$$

each a function of the structure size n , the grammar size g , the query size q , update parameters u and output size o . A **grammatical index** is a resolution scheme whose Build takes a grammar as input; a **grammar-based algorithm** is one whose control flow follows derivations (product constructions, fixpoint evaluation of productions, descent through a structural grammar).

The notation $T(n, g, q, u, o)$ is a reporting discipline, not yet a theory: whether grammar parameters yield distinctions not already captured by parameterized complexity or by preprocessing–query trade-offs is an open problem (CGT-OPEN-005).

7.6 Five mechanisms

Every advantage observed so far in this research — in the chapters of Part I and in the ten experiments of the ledger — traces to one of five mechanisms.

DEFINITION 7.5 (Mechanisms of advantage)

CGT-DEF-009

CODE	MECHANISM	WHERE IT APPEARS
M-ARITH	an arithmetic numeration of a dense address space replaces stored structure by computation	arrays; ring buffers; heaps (CGT-THM-003); compiled words (CGT-THM-006)
M-PRE	precomputation: answers are stored and later retrieved	closures, LCA tables, perfect hashing — bounded below by information content (CGT-THM-002)
M-SHARE	repeated substructure is represented once	minimal DAGs, minimal automata — fragile under updates (CGT-THM-005)
M-RESTRICT	a restriction of the instance class makes structure exploitable	interval labels on forests; 2-SAT
M-PRUNE	the admissibility language excludes moves before they are executed	product construction — and it can also <i>multiply</i> work (CGT-THM-001)

CONJECTURE 7.6 (Mechanism thesis)

CONJECTURE

CGT-CONJ-001

Every asymptotic advantage of a grammatical resolution scheme over the best conventional scheme for the same queries, machine model and cost component is attributable to a combination of M-ARITH, M-PRE, M-SHARE, M-RESTRICT and M-PRUNE.

The thesis is not yet a mathematical statement: “conventional scheme” and “attributable” are undefined, and making them precise is an open problem (CGT-OPEN-001). It is nevertheless falsifiable in practice — by a grammar-based scheme with an advantage none of the five explains — and it organizes the rest of the book. Chapters 8–14 examine each mechanism with its cost profile and its failure cases.

7.7 Relationship to established theory

The components of an operational grammar are not new, and it is important to say which existing theories each comes from.

- **Labelled transition systems and relational semantics.** $(\Sigma, \mathfrak{A}, \llbracket \cdot \rrbracket)$ is a labelled transition system; the extension to words is the relational model of Kleene algebra. *Kleene algebra with tests* [Kozen 1997] adds Boolean guards and is a natural home for admissibility conditions that depend on state.
- **Formal-language-constrained paths.** With regular or context-free $\mathcal{L}$ over a labelled graph, the grammar is exactly the setting of regular path queries and CFL-reachability [Mendelzon & Wood 1995; Barrett et al. 2000; Yannakakis 1990; Reps 1998].
- **Attribute grammars.** Knuth’s attribute grammars [Knuth 1968] attach semantic functions to the productions of a context-free grammar and evaluate them over derivation trees. They are the classical way of making a grammar *compute*. CGT differs in where the grammar lives: attribute grammars describe strings and their parse trees; operational grammars describe operations over an arbitrary structure. A systematic comparison is a literature debt of this program ([CGT-OPEN-010](#)).
- **Automatic structures and groups.** Structures whose elements are words of a regular language and whose relations are recognized by synchronous automata [Blumensath & Grädel 2000; Khoussainov & Nerode 1995], and groups whose multiplication by generators is so recognized [Epstein et al. 1992], are the closest existing theory to the address-as-sentence idea. Chapter 13’s compiled words are a cost-oriented special case.
- **Abstract data types.** Adequacy is the standard notion of a correct implementation of an abstract type.

What the program adds is not a new object but a *discipline*: every claim filed under one layer, every cost stated as a profile in an explicit model, every advantage attributed to a mechanism and paired with the inputs that defeat it.

EXERCISE 7.1

Extend `BoundedCounter` with a move `reset` and an admissibility language in which `reset` may only occur after at least two `inc`. Write the language as a regular expression and check your answer with the SDK.

EXERCISE 7.2

Prove that adequacy is preserved under sequential composition: if a representation is adequate for every single-call expression from every reachable state, it is adequate for all expressions.

► Solution to [Exercise 7.2](#)

EXERCISE 7.3

Give an operational grammar (states, moves, semantics, admissibility) for a traffic light with moves `next` and `emergency`, in which `emergency` is admissible only from green. Which component of the definition carries the “only from green” condition — syntax or semantics? Argue for both choices.

EXERCISE 7.4

For each mechanism in [Definition 7.5](#), give one example from outside this book (a database index, a compiler optimization, a cache) and say which cost component of the profile it improves and which it worsens.

SUMMARY

- An operational grammar $G = (\Sigma, \mathcal{A}, [\cdot], \mathcal{L})$ consists of moves, relational semantics on a structure, and an admissibility language constraining the projection of move words.
- Syntax (admissibility), semantics (definedness and values) and execution (cost) are separate layers; a representation is adequate when its execution agrees with the semantics, and the SDK tests this.
- Structural grammars generate the structure; operational grammars generate operations over it.
- Costs are reported as profiles $(T_{\text{build}}, S, T_{\text{query}}, T_{\text{update}}, |\text{out}|)$.
- Five mechanisms — arithmetic addressing, precomputation, sharing, restriction, pruning — account for every advantage observed so far; that they account for *all* advantages is a conjecture.
- The components are standard; the program’s contribution is the discipline that keeps them apart.

Admissibility Languages and the Product Construction

LEARNING OBJECTIVES

- Compile a regular expression over move symbols into a minimal deterministic automaton, and know the cost of each step.
- Resolve a language-constrained query on a structure by searching the product of the structure with the automaton.
- Prove that a constraint can multiply the cost of search by exactly $|Q|$ and can also prune it to almost nothing.
- Explain why “search first, filter afterwards” is incorrect, and why enumerating walks is exponential.
- Describe what changes when the admissibility language is context-free.

8.1 From expression to automaton

An admissibility language is written as a regular expression over move symbols, such as $(a \mid b)^*$, $(a \mid \varepsilon)$ or $(L \mid R)^* \cup$. The SDK compiles it by the classical three-stage pipeline:

1. **Thompson’s construction** turns the expression into a nondeterministic automaton with ε -moves, with $\mathcal{O}(r)$ states for an expression of length r .
2. **The subset construction** turns it into a deterministic automaton whose states are sets of NFA states; in the worst case this is exponential in r .
3. **Moore’s partition refinement** merges states with the same future, giving the **minimal** DFA — unique up to renaming (Myhill–Nerode).

PROPOSITION 8.1 (Determinization can be exponential) ESTABLISHED

The language “the k -th letter from the end is a ”, $(a \mid b)^* a (a \mid b)^{k-1}$, has an expression of length $\mathcal{O}(k)$ and a minimal DFA with 2^k states.

Proof.

A DFA must remember the last k letters: two different suffixes of length k differ at some position j , and appending $j - 1$ further letters separates them (one is accepted, the other not). So at least 2^k states are needed, and the automaton tracking the last k letters achieves it.

The SDK's tests check the size 8 for $k = 3$ and check acceptance of every word up to length six against Python's own regular-expression engine for a set of expressions:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtask import compile_regex

for rx in ["a*", "(a b)* (a | ε)", "(L | R)* U", "(a | b)* a (a | b) (a | b)", "(a | b)* a (a | b) (a | b) (a | b) (a | b)":
    d = compile_regex(rx)
    print(f"{rx:44s} minimal DFA states = {d.size:3d}   accepts 'a b a': {d.accepts('a b a'.split())}")
```

OUTPUT (CHECKED)

a^*	minimal DFA states = 1	accepts 'a b a': False
$(a\ b)^* (a\ \ \varepsilon)$	minimal DFA states = 2	accepts 'a b a': True
$(L\ \ R)^* U$	minimal DFA states = 2	accepts 'a b a': False
$(a\ \ b)^* a (a\ \ b) (a\ \ b)$	minimal DFA states = 8	accepts 'a b a': True
$(a\ \ b)^* a (a\ \ b) (a\ \ b) (a\ \ b) (a\ \ b) (a\ \ b)$	minimal DFA states = 32	accepts 'a b a': False

(The automata are *partial*: a missing transition means the word can no longer be accepted, so no “dead” state is counted.) The size $|Q|$ of the minimal DFA is the grammar parameter that governs everything below.

8.2 Searching under a constraint

Let $G = (\Sigma, \mathfrak{A}, \llbracket \cdot \rrbracket, \mathcal{L})$ with $\mathcal{L}$ given by a DFA $D = (Q, \Sigma, \delta, q_0, F)$ with partial δ . The **constrained resolution** problem is: given x , compute $\llbracket \mathcal{L} \rrbracket(x)$, the set of elements reachable from x by some admissible word. For a labelled graph this is the regular path query of Chapter 6.

The answer is found by searching the **product** $A \times Q$: there is a product edge $(y, q) \rightarrow (z, \delta(q, a))$ for each move $(y, z) \in \llbracket a \rrbracket$ with $\delta(q, a)$ defined. Then $y \in \llbracket \mathcal{L} \rrbracket(x)$ iff some (y, q) with $q \in F$ is reachable from (x, q_0) .

THEOREM 8.2 (Product resolution, and the exact cost of a constraint)

PROVED HERE

CGT-THM-001

Let $m = \sum_a \llbracket a \rrbracket$.

1. (Upper bound; standard.) $\llbracket \mathcal{L} \rrbracket(x)$ is computable in $\mathcal{O}(|Q| \cdot (|\Sigma|n + m))$ time and $\mathcal{O}(|Q| \cdot n)$ space.
2. (A constraint can cost a factor $|Q|$ and prune nothing.) For all $k \geq 1$ and all n coprime to k there are a structure with n elements and $m = n$ and a k -state DFA such that unconstrained search from x visits n states, product search visits exactly nk , and $\llbracket \mathcal{L} \rrbracket(x)$ equals the unconstrained answer.
3. (A constraint can prune almost everything.) For every n there are a structure with n elements and a one-state DFA such that product search visits 1 state, unconstrained search visits n , and the answers differ.

Proof.

(1) By induction on $|w|$: $y \in \llbracket w \rrbracket(x)$ and $\delta^*(q_0, w) = q$ iff (y, q) is reachable from (x, q_0) along a product walk spelling w . The product has at most $|Q|n$ vertices; each is expanded once, consulting δ for each letter ($|Q||\Sigma|n$ in total) and inspecting each move of its first component once per state ($|Q|m$ in total). This is the classical construction [Mendelzon & Wood 1995; Barrett et al. 2000].

(2) Take the directed cycle on $\mathbb{Z}_n$ with one label a , $\llbracket a \rrbracket = \{(i, i + 1 \bmod n)\}$, and $\mathcal{L} = \{a^t : t \equiv 0 \pmod{k}\}$, whose DFA is the k -cycle on $\mathbb{Z}_k$. The product move is $(i, j) \mapsto (i + 1 \bmod n, j + 1 \bmod k)$, and the orbit of $(0, 0)$ is $\{(t \bmod n, t \bmod k) : t \geq 0\}$, of size $\text{lcm}(n, k) = nk$ by the Chinese remainder theorem. Unconstrained search visits the n vertices. For every target i some t satisfies $t \equiv i \pmod{n}$ and $t \equiv 0 \pmod{k}$, so $\llbracket \mathcal{L} \rrbracket(0) = \mathbb{Z}_n$, the unconstrained answer.

(3) Take a star with centre x and $n - 1$ edges labelled b to leaves, and $\mathcal{L} = a^*$. The product visits only (x, q_0) and answers $\{x\}$; unconstrained search visits all n vertices. ■

Part 1 is textbook material; parts 2 and 3 are elementary and quite possibly folklore, but together they state something that is rarely made explicit: **a grammatical constraint is not a speedup**. It changes the search space, in either direction, by up to the size of the automaton. Whether it helps depends jointly on the language and on the structure — exactly the claim of hypothesis H-005, which the experiments supported (CGT-H-005).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import Cost, compile_regex
from cgtsdk.algorithms import product_reach
from cgtsdk.structures import GraphStructure
```

```

n = 101
cycle = GraphStructure(n, [(i, "a", (i + 1) % n) for i in range(n)])
for k in (1, 2, 5, 10):
    rx = " ".join(["a"] * k) # words of length divisible by k: (a^k)*
    reach, explored = product_reach(cycle, 0, compile_regex(f"({rx})*"), return_explored=True)
    print(f"k={k:2d}: product states explored = {explored:4d} = n*k = {n*k:4d}; |answer| = {len(reach)}")

```

OUTPUT (CHECKED)

```

k= 1: product states explored = 101 = n*k = 101; |answer| = 101
k= 2: product states explored = 202 = n*k = 202; |answer| = 101
k= 5: product states explored = 505 = n*k = 505; |answer| = 101
k=10: product states explored = 1010 = n*k = 1010; |answer| = 101

```

On random labelled graphs ([CGT-EXP-003](#)) the measured ratio of explored product states to unconstrained states ranged from 0.003 (language a^* on a sparse graph) to exactly 5.000 (walk length divisible by 5): both extremes of the theorem occur in practice, not only on constructed inputs.

8.3 Two incorrect shortcuts

Two tempting alternatives to the product construction fail.

Search, then filter. Plain reachability forgets *which* label words lead to a vertex, so no filtering of its answer can recover $[[\mathcal{L}]](x)$. On the random graphs of the experiment, unconstrained search misreported up to about 15,600 of 20,000 vertices for restrictive languages ([CGT-OBS-007](#)). The constraint must be carried *during* the search.

Enumerate walks, then check the word. This is correct if walks up to length $|Q| \cdot n$ are enumerated — any longer accepted walk revisits a product state and can be shortened — but the number of walks grows exponentially. On a 12-vertex graph the experiment’s walk enumeration hit its cap of two million walks while the product had at most 55 states. The product construction is, in effect, the **sharing** mechanism applied to walks: all walks reaching the same (vertex, state) pair are represented once.

8.4 Context-free admissibility

When $\mathcal{L}$ is context-free — balanced parentheses for matched calls and returns, the Dyck language of Chapter 3 — the product is no longer a finite graph, and constrained reachability becomes **CFL-reachability**, the formulation behind interprocedural program analysis [Reps 1998;

Yannakakis 1990]. It is solved by a dynamic program over pairs of vertices and nonterminals in time cubic in n (with factors depending on the grammar) [Barrett et al. 2000]. Whether the factor- $|Q|$ phenomenon of Theorem 8.2 has a precise context-free analogue — how the size of the grammar enters, and whether the bound is attained — is open ([CGT-OPEN-008](#)).

EXERCISE 8.1

Compile $(L \mid R)^* \cup$ by hand: give the Thompson NFA, the subset DFA, and the minimal DFA. Explain in one sentence what language of tree navigations it describes.

► Solution to [Exercise 8.1](#)

EXERCISE 8.2

In part 2 of [Theorem 8.2](#), what happens when $\gcd(n, k) = g > 1$? Compute the number of explored product states and the answer set.

► Solution to [Exercise 8.2](#)

EXERCISE 8.3

Prove the shortening argument: if some walk from x to y has its label in $\mathcal{L}$, then some such walk has length less than $|Q| \cdot n$.

EXERCISE 8.4

Using the SDK, generate a random labelled graph with $n = 2000$ and average degree 1.5, and compare product-search sizes for a^* , $(a \mid b)^* c$ and “length divisible by 5”. Reproduce the qualitative pattern of [CGT-EXP-003](#).

SUMMARY

- Admissibility languages compile to minimal DFAs (Thompson, subset, Moore); determinization can cost 2^k states.
 - Constrained resolution is search on the product $A \times Q$, in $\mathcal{O}(|Q|(|\Sigma|n + m))$ time.
 - A constraint can multiply the search space by exactly $|Q|$ without changing the answer, or prune it to a single state ([CGT-THM-001](#)); measured ratios on random graphs span both extremes.
 - Filtering an unconstrained search is incorrect; enumerating walks is exponential; the product shares all walks that reach the same (vertex, state) pair.
 - Context-free admissibility leads to CFL-reachability, cubic in general; its cost-of-constraint theory is open.
-

Searching and Sorting Reconstructed

LEARNING OBJECTIVES

- State, implement and prove correct linear search, binary search, insertion sort, merge sort, heapsort, quicksort and counting sort.
- Predict their comparison counts exactly or within stated bounds, and check the predictions with counted costs.
- Prove the $\Omega(n \log n)$ lower bound for comparison sorting, and explain it as a statement about the grammar of comparisons.
- Explain why counting sort escapes the bound — arithmetic addressing — and exactly when it stops helping.

Each algorithm in Part III is presented as an **algorithm card**: the problem, the conventional algorithm with pseudocode and a correctness argument, its costs, a pointer to its SDK implementation, its CGT reading (which grammar it operates on, and how grammar operations map to implementation steps), and — always — where the grammatical view adds nothing. This chapter's verdict is mostly *equivalence*: sorting and searching are the home ground of classical algorithm design, and the grammatical view re-describes them without making them faster. That verdict is itself a result.

9.1 Searching

Problem. Given a sequence $a_0, \dots, a_{n-1}$ and a value x , find an index i with $a_i = x$, or report that none exists.

ALGORITHM 9.1 (Binary search (leftmost occurrence))

TEXT

```
lo ← 0; hi ← n                                invariant: a[lo-1] < x ≤ a[hi] (sentinels a[-1] = -∞, a[n] = +∞)
while lo < hi:
    mid ← ⌊(lo + hi)/2⌋
    if a[mid] < x: lo ← mid + 1 else: hi ← mid
return lo if lo < n and a[lo] = x else -1
```

PROPOSITION 9.1 ESTABLISHED

On a sorted sequence, binary search returns the leftmost index of x or -1 , using at most $\lceil \log_2(n+1) \rceil$ loop comparisons plus one final equality test. Linear search uses at most n comparisons and needs no order.

Proof.

The invariant holds initially and is preserved by both branches because the sequence is sorted; the interval $[lo, hi)$ strictly shrinks and at least halves, so the loop runs at most $\lceil \log_2(n+1) \rceil$ times. At exit $lo = hi$, and the invariant says every element before lo is $< x$ and $a_{lo} \geq x$; hence x occurs iff $a_{lo} = x$, and then lo is its leftmost index. ■

SDK: `cgtsdk.algorithms.binary_search`, `linear_search`. *Grammar:* binary search is navigation in the implicit balanced search tree whose root is the middle element: each comparison is a move L or R , and the comparison outcomes spell the address of the answer. The sorted array is a perfectly balanced BST in disguise, with addresses computed by midpoint arithmetic — arithmetic addressing again (M-ARITH). *Where grammar adds nothing:* the algorithm is optimal among comparison-based searches (an adversary argument gives $\lceil \log_2(n+1) \rceil$), and no re-description changes that.

9.2 Four comparison sorts

Problem. Rearrange a sequence into non-decreasing order.

The four classical comparison sorts, in their SDK form:

ALGORITHM	COMPARISONS (WORST)	COMPARISONS (BEST)	EXTRA SPACE	STABLE	REFERENCE
insertion sort	$n(n-1)/2$	$n-1$ (sorted input)	$\mathcal{O}(1)$	yes	folklore
merge sort	$n\lceil\log_2 n\rceil$	about $\frac{1}{2}n\log_2 n$	$\mathcal{O}(n)$	yes	[Cormen et al. 2022]
heapsort	$\mathcal{O}(n\log n)$	$\mathcal{O}(n\log n)$	$\mathcal{O}(1)$	no	[Williams 1964]
quicksort (random pivot)	$\Theta(n^2)$	$\mathcal{O}(n\log n)$ expected	$\mathcal{O}(\log n)$	no	[Hoare 1962]

ALGORITHM 9.2 (Insertion sort)

TEXT

```

for i ← 1 to n-1:                invariant: a[0..i) is sorted
    x ← a[i]; j ← i - 1
    while j ≥ 0 and a[j] > x: a[j+1] ← a[j]; j ← j - 1
    a[j+1] ← x

```

The invariant gives correctness at once. The comparison count is $n-1$ plus the number of **inversions** (pairs $i < j$ with $a_i > a_j$), capped per element by its position — exactly $n-1$ on sorted input and $n(n-1)/2$ on reversed input. The SDK counts confirm both exactly:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```

import random
from cgtsdk import Cost
from cgtsdk.algorithms import heapsort, insertion_sort, merge_sort, quicksort

n = 512
inputs = {"sorted": list(range(n)), "reversed": list(range(n, 0, -1)),
          "random": random.Random(1).sample(range(10**6), n)}
print(f'{"input":9s} {"insertion":>10s} {"merge":>7s} {"heap":>7s} {"quick":>7s}   n(n-1)/2={n*(n-1)//2}
n*log2(n)={n*9}')
for name, a in inputs.items():
    counts = []
    for f in (insertion_sort, merge_sort, heapsort, quicksort):
        c = Cost(); assert f(a, c) == sorted(a); counts.append(c.counts.get("compare", 0))
    print(f'{"name":9s} {"counts[0]:10d} {"counts[1]:7d} {"counts[2]:7d} {"counts[3]:7d}')

```

OUTPUT (CHECKED)

input	insertion	merge	heap	quick	$n(n-1)/2=130816$	$n \cdot \log_2(n)=4608$
sorted	511	2304	7958	8642		
reversed	130816	2304	7203	8217		
random	69565	3970	7599	7796		

(Quicksort here uses a three-way partition that charges up to two comparisons per element per partitioning step, which is why its counts sit above merge sort’s.) Insertion sort is the best of the four on sorted input and by far the worst on reversed input; merge sort makes exactly $n \log_2 n / 2 = 2304$ comparisons when one half always wins, and fewer than $n \lceil \log_2 n \rceil = 4608$ in every case.

9.3 The comparison lower bound as a statement about grammar

A comparison sort can learn about its input only by comparing elements. Its execution on all inputs of size n is therefore described by a **decision tree**: a binary tree whose internal nodes are comparisons “ $a_i < a_j$?” and whose leaves are output permutations. Each run follows one root-to-leaf path; the path’s address — the word of comparison outcomes — determines the output.

THEOREM 9.2 (Comparison lower bound) ESTABLISHED

Every comparison-based sorting algorithm makes at least $\lceil \log_2 n! \rceil = n \log_2 n - \mathcal{O}(n)$ comparisons on some input of size n .

Proof.

Distinct inputs that are permutations of $\{1, \dots, n\}$ require distinct outputs, so the decision tree has at least $n!$ leaves. A binary tree of height h has at most 2^h leaves, so $h \geq \log_2 n!$, and $\log_2 n! = n \log_2 n - n \log_2 e + \mathcal{O}(\log n)$ by Stirling’s formula. ■

In the language of this book: the **grammar** available to a comparison sort is the binary alphabet of comparison outcomes, and the **information** it must produce is one of $n!$ answers. No grammar over a two-letter alphabet can address $n!$ answers with words shorter than $\log_2 n!$. This is the counting lemma that reappears in Chapter 12 as an information lower bound on *any* precomputed grammar (CGT-THM-002). Re-describing sorting grammatically cannot beat it — and claims to “sort in constant time with a grammar” should be read against it.

9.4 Counting sort: escaping the bound by arithmetic addressing

The lower bound assumes that comparisons are the only way to learn about the input. If the keys are integers in a small range $[0, k)$, a key can be used as an **address**:

ALGORITHM 9.3 (Counting sort)

TEXT

```
count[0..k] ← 0
for x in a: count[x+1] ← count[x+1] + 1      histogram, using x as an address
for i ← 0 to k-1: count[i+1] ← count[i+1] + count[i]  prefix sums: first slot of each key
for x in a (left to right): out[count[x]] ← x; count[x] ← count[x] + 1  stable placement
```

PROPOSITION 9.3 ESTABLISHED

Counting sort sorts n integers in $[0, k)$ stably with no comparisons, in $\Theta(n + k)$ time and space.

This is mechanism M-ARITH in its purest form: the key's value is the location, exactly like an array index (Chapter 2). And it has exactly the array's limitation: the **density** n/k must not be small. With $k = \Theta(n)$ it is linear; with 64-bit keys ($k = 2^{64}$) it is absurd. Radix sort refines the idea by addressing one digit at a time.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import random
from cgtsdk import Cost
from cgtsdk.algorithms import counting_sort, merge_sort

rng = random.Random(2)
for n, k in ((10_000, 100), (10_000, 10_000), (10_000, 1_000_000)):
    a = [rng.randrange(k) for _ in range(n)]
    c1, c2 = Cost(), Cost()
    assert counting_sort(a, k, c1) == merge_sort(a, c2) == sorted(a)
    print(f"n={n} k={k:9d}  counting sort: {c1.total:9d} steps, 0 comparisons  merge sort: {c2.total:7d} steps")
```

OUTPUT (CHECKED)

n=10000 k=	100	counting sort:	50301 steps, 0 comparisons	merge sort:	253841 steps
n=10000 k=	10000	counting sort:	80001 steps, 0 comparisons	merge sort:	254082 steps
n=10000 k=	1000000	counting sort:	3050001 steps, 0 comparisons	merge sort:	254093 steps

At density 100 keys per slot, counting sort is five times cheaper than merge sort in counted steps; at density 10^{-2} it is twelve times *more* expensive. The same algorithm wins or loses according to a single structural parameter — the density of the address space.

9.5 Verdict

For searching and sorting, the grammatical view gives *re-descriptions*: binary search as navigation in an implicit tree, a sorting algorithm as a grammar of comparison outcomes, counting sort as arithmetic addressing. It gives no speedup over the classical algorithms, and the comparison lower bound explains why none is possible within the comparison model. What it does give is a uniform vocabulary in which the one known way around the bound — using keys as addresses — is the same mechanism as array indexing and heap navigation, with the same failure condition.

EXERCISE 9.1

Prove that insertion sort makes exactly $(n - 1) + I - E$ comparisons, where I is the number of inversions and E is the number of elements that are smaller than every element before them (those for which the inner loop runs off the left end). Check the formula against the SDK on random inputs.

EXERCISE 9.2

Give an input of size 8 on which the SDK's quicksort with seed 0 makes more comparisons than heapsort, or prove that none exists. (Use the SDK to search.)

EXERCISE 9.3

Show that $\lceil \log_2 n! \rceil$ comparisons suffice to sort $n = 5$ elements (seven comparisons), or look up and verify a schedule that does.

EXERCISE 9.4

Counting sort uses $\Theta(k)$ memory even when only a few distinct keys occur. Combine it with hashing (Chapter 4) to sort n keys from a huge universe with d distinct values in $\mathcal{O}(n + d \log d)$ time. Which mechanisms does your algorithm use?

SUMMARY

- Binary search is navigation in an implicit balanced tree addressed by midpoint arithmetic; it is optimal among comparison searches.
 - Insertion, merge, heap and quick sort are implemented with counted comparisons that match their analyses exactly where the analysis is exact.
 - The $\log_2 n!$ lower bound is a counting statement: a binary grammar of comparison outcomes cannot address $n!$ answers with shorter words.
 - Counting sort escapes it by using keys as addresses — arithmetic addressing — and is only as good as the density of the key range.
 - For searching and sorting the grammatical view yields equivalence, not improvement; that is a finding, not a failure.
-

Graph Algorithms Reconstructed

LEARNING OBJECTIVES

- Implement and justify BFS, DFS, topological sorting, strongly connected components, Dijkstra, Bellman–Ford, Kruskal, Prim, union–find and Floyd–Warshall, with counted costs.
- Read each algorithm as the evaluation of a path grammar in a particular algebra.
- Prove the exact rule-application count of semi-naïve evaluation of the right-linear path grammar, and compare it with the doubling grammar.
- Explain why two grammars with the same meaning can have asymptotically different costs, trading total work against parallel depth.

10.1 Traversal, order and components

Breadth-first search from s visits vertices in order of hop distance and computes $\text{dist}(s, \cdot)$ in $\Theta(n + m)$; **depth-first search** visits each vertex and edge once and classifies edges by discovery and finishing times. Both are textbook algorithms [Cormen et al. 2022]. Three classical consequences:

- **Topological sorting.** A DAG’s vertices can be ordered so that every edge goes forward; Kahn’s algorithm repeatedly removes a vertex of in-degree zero, in $\Theta(n + m)$, and reports a cycle if it gets stuck [Kahn 1962].
- **Strongly connected components.** Tarjan’s algorithm [Tarjan 1972] finds them in one DFS using low-link values, and numbers them in reverse topological order of the condensation (the SDK’s `tarjan_scc` returns that numbering, so component 0 is a sink).
- **Reachability and closure.** The reflexive–transitive closure — the semantics $\llbracket E^* \rrbracket$ of Chapter 6 — can be computed by condensing SCCs and OR-ing bitset rows in reverse topological order: each OR of n -bit rows costs $\lceil n/w \rceil$ word operations.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import Cost
from cgtsdk.algorithms import bfs, tarjan_scc, topological_sort, transitive_closure
```

```
from cgtsdk.lab import random_digraph

for name, adj in (("random DAG", random_digraph(2000, 2.0, seed=1, dag=True)),
                  ("random digraph", random_digraph(2000, 2.0, seed=1))):
    c1, c2, c3 = Cost(), Cost(), Cost()
    d = bfs(adj, 0, c1)
    comp, k = tarjan_scc(adj, c2)
    order = topological_sort(adj)
    rows = transitive_closure(adj, c3)
    pairs = sum(bin(r).count("1") for r in rows)
    print(f"{name:15s} m={sum(map(len, adj))} BFS edges={c1.counts['edge']} SCCs={k} "
          f"topological order: {'yes' if order else 'no (cycle)'} closure pairs={pairs} closure cost="
          f"{c3.total}")
```

OUTPUT (CHECKED)

```
random DAG      m=4000 BFS edges=20 SCCs=2000 topological order: yes closure pairs=26226 closure
cost=136000
random digraph  m=4000 BFS edges=3240 SCCs=718 topological order: no (cycle) closure pairs=2575275 closure
cost=39584
```

On the random digraph a giant strongly connected component absorbs 1,283 of the 2,000 vertices; condensing it shares one closure row among all its members, which is why the closure of a graph with *more* reachable pairs costs *fewer* steps. This is the sharing mechanism (M-SHARE) at work inside a classical algorithm.

10.2 Shortest paths and spanning trees

ALGORITHM	PROBLEM	REQUIREMENT	COST	SOURCE
Dijkstra	single-source shortest paths	weights ≥ 0	$\mathcal{O}((n + m) \log n)$ with a binary heap	[Dijkstra 1959]
Bellman–Ford	single-source shortest paths, negative-cycle detection	none	$\mathcal{O}(nm)$	[Bellman 1958; Ford 1956]
Floyd–Warshall	all-pairs shortest paths	no negative cycles	$\Theta(n^3)$	[Cormen et al. 2022]

ALGORITHM	PROBLEM	REQUIREMENT	COST	SOURCE
Kruskal	minimum spanning forest	undirected	$\mathcal{O}(m \log m)$ with union-find	[Kruskal 1956; Tarjan 1975]
Prim	minimum spanning forest	undirected	$\mathcal{O}(m \log n)$ with a binary heap	[Prim 1957]

All are implemented in `cgtsdk.algorithms` and tested against one another and against brute force: Dijkstra and Bellman–Ford against Floyd–Warshall on random weighted digraphs, Kruskal against Prim and against exhaustive search over edge subsets on small graphs.

In the path-algebra view of Chapter 6 all three shortest-path algorithms evaluate the *same* path grammar EE^* in the tropical semiring $(\min, +)$; they differ in evaluation *strategy*. Dijkstra exploits a property of the algebra on non-negative weights — once a vertex is settled its distance never improves — to evaluate each vertex once in increasing order of distance. Bellman–Ford makes no such assumption and relaxes every edge up to $n - 1$ times. Floyd–Warshall is Kleene’s algorithm for the closure of a matrix: it allows intermediate vertices $0, 1, \dots, k$ in turn. The grammar is common; the costs come from the semantics’ algebraic properties and the evaluation order. A grammatical description, by itself, does not choose the order.

10.3 Evaluating a path grammar

We now make that last sentence precise. The closure $P^+ = \llbracket EE^* \rrbracket$ (paths of length at least one) is the least relation P satisfying either of two grammars:

$$\text{right-linear: } P \rightarrow E \mid EP \qquad \text{doubling: } P \rightarrow E \mid PP. \qquad (10.1)$$

Both have the same meaning (Chapter 6). Evaluate each **bottom-up**: start from the facts $E(u, v)$, and repeatedly apply the productions to newly derived facts until nothing new appears (*semi-naive evaluation*, standard in Datalog).

THEOREM 10.1 (Same meaning, different grammar, different cost)**PROVED HERE**

CGT-THM-004

Let G have n vertices and m edges, and let ℓ be the largest length of a shortest positive-length path between a pair in P^+ (pairs (u, u) on cycles included).

1. Semi-naive evaluation of the right-linear grammar makes exactly $m + \sum_{(w,v) \in P^+} \text{indeg}(w)$ rule applications, which is at most $m + nm$.
2. Semi-naive evaluation of the doubling grammar makes $\mathcal{O}(n \cdot |P^+|) = \mathcal{O}(n^3)$ rule applications.
3. With synchronous rounds, the right-linear grammar needs $\ell - 1$ productive rounds and the doubling grammar $\lceil \log_2 \ell \rceil$.

Proof.

(1) Initialization attempts each edge once: m . A fact (w, v) enters the work list only when first derived, so each fact of P^+ is processed exactly once; processing it applies $E(u, w) \wedge P(w, v) \Rightarrow P(u, v)$ once per in-edge (u, w) , i.e. $\text{indeg}(w)$ times. Soundness and completeness of semi-naive evaluation follow by induction on path length.

(2) Each fact is processed once, and processing (w, v) joins it with the current $\{u : P(u, w)\}$ and $\{z : P(v, z)\}$, at most $2n$ attempts.

(3) By induction, after i synchronous rounds the right-linear iterate contains exactly the pairs at distance in $[1, i + 1]$ and the doubling iterate those at distance in $[1, 2^i]$. ■

Part 1 says something exact and somewhat deflating: **evaluating the path grammar is computing the closure**, with the work of one reverse search per target. The recursive description of paths, however elegant, does no less work than the classical algorithm. Parts 2 and 3 show that the *shape* of a grammar is a real cost parameter: the doubling grammar does asymptotically more work on long paths but needs exponentially fewer rounds — the trade between total work and parallel depth behind computing transitive closure by repeated squaring. This is well known in Datalog practice (linear versus non-linear recursion); the exact count in part 1 is, as far as this research has found, a convenient statement rather than a new result.

The SDK checks part 1 exactly on every test graph. The experiment of the ledger measured the doubling grammar on directed paths (CGT-EXP-007): its rule applications grew by a factor of about 8 each time n doubled (cubic), while the right-linear grammar's grew by 4 (quadratic).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import Cost
from cgtsdk.algorithms import path_grammar_closure
```

```

for n in (32, 64, 128):
    path = [[i + 1] if i + 1 < n else [] for i in range(n)]
    lin, dbl = Cost(), Cost()
    P1 = path_grammar_closure(path, "linear", lin)
    P2 = path_grammar_closure(path, "doubling", dbl)
    assert P1 == P2
    pairs = sum(map(len, P1))
    formula = (n - 1) + sum(len(P1[w]) * (1 if w > 0 else 0) for w in range(n))
    print(f"n={n:4d} |P+|= {pairs:5d} right-linear rules={lin.counts['rule']:6d} (formula {formula}) doubling
rules={dbl.counts['rule']:8d}")

```

OUTPUT (CHECKED)

```

n= 32 |P+|= 496 right-linear rules= 496 (formula 496) doubling rules= 9516
n= 64 |P+|= 2016 right-linear rules= 2016 (formula 2016) doubling rules= 81500
n= 128 |P+|= 8128 right-linear rules= 8128 (formula 8128) doubling rules= 675004

```

(On a directed path every vertex except the first has in-degree 1, so the formula reduces to $m + \sum_{w>0} |P(w)| = |P^+|$.)

10.4 The mapping, algorithm by algorithm

ALGORITHM	GRAMMAR IT EVALUATES	ALGEBRA	EVALUATION STRATEGY	MECHANISM VISIBLE
BFS	EE^* from one source	Boolean / hop count	frontier by frontier	— (it is the search)
SCC condensation + bitsets	EE^* , all sources	Boolean	reverse topological, sharing rows	M-SHARE
Dijkstra	EE^*	$(\min, +)$, non-negative	settle in distance order	algebraic monotonicity
Bellman–Ford	EE^*	$(\min, +)$	$n - 1$ global rounds	—
Floyd–Warshall	$E \mid PP$ restricted by intermediate vertex	$(\min, +)$	Kleene elimination	—

ALGORITHM	GRAMMAR IT EVALUATES	ALGEBRA	EVALUATION STRATEGY	MECHANISM VISIBLE
semi-naive right-linear	$P \rightarrow E \mid EP$	Boolean	worklist	— (exact work count, THM-004)
semi-naive doubling	$P \rightarrow E \mid PP$	Boolean	worklist / squaring	depth for work
Kruskal / union-find	— (not a path problem)	—	greedy by weight	path compression is precomputation (M-PRE, amortized)

The verdict for graph algorithms is again **equivalence**: each classical algorithm is an evaluation strategy for a path grammar in some algebra, and the grammar alone does not supply the strategy. The grammatical view earns its keep by making the shared structure explicit — which is what made the product construction of Chapter 8 and the constrained random surfer of Chapter 16 straightforward to derive.

EXERCISE 10.1

Verify part 1 of [Theorem 10.1](#) by hand on the graph with edges $0 \rightarrow 1, 1 \rightarrow 2, 0 \rightarrow 2, 2 \rightarrow 0$.

► Solution to [Exercise 10.1](#)

EXERCISE 10.2

Show that Dijkstra's algorithm can return wrong distances with one negative edge and no negative cycle. Which algebraic property used by its evaluation strategy fails?

EXERCISE 10.3

Design a third grammar for P^+ — for example $P \rightarrow E \mid PE$ (left-linear) — and derive its exact rule count under semi-naive evaluation. When is it cheaper than the right-linear grammar?

► Solution to [Exercise 10.3](#)

EXERCISE 10.4

Using the SDK, compare `transitive_closure` (bitsets) with `path_grammar_closure` on random digraphs of increasing density. Explain the result in terms of the word-RAM's word parallelism.

SUMMARY

- Classical graph algorithms — traversal, components, shortest paths, spanning trees, closure — are implemented with counted costs and cross-checked against each other and brute force.
 - Each path algorithm evaluates the grammar EE^* in an algebra; the strategy, not the grammar, determines the cost.
 - Evaluating the right-linear path grammar costs exactly $m + \sum_{(w,v) \in P^+} \text{indeg}(w)$ rule applications: it is closure computation ([CGT-THM-004](#)).
 - Two grammars with the same meaning can differ asymptotically: doubling trades $\Theta(n^3)$ work on paths for $\mathcal{O}(\log \ell)$ rounds.
 - The verdict is equivalence; the benefit of the grammatical view is a shared vocabulary that makes new constructions easy to derive.
-

Range Queries, Dynamic Programming, and Strings

LEARNING OBJECTIVES

- Explain Fenwick trees and segment trees as implicit trees addressed by bit arithmetic, and state their costs.
- Contrast them with sparse tables, which trade preprocessing for constant-time queries.
- Read dynamic programming as the evaluation of a grammar of subproblems with sharing, and connect it to parsing.
- Explain the Knuth–Morris–Pratt algorithm as a deterministic automaton for its pattern, with linear preprocessing.

11.1 Prefix sums by bit arithmetic

Problem. Maintain an array $a_0, \dots, a_{n-1}$ under point updates $a_i \leftarrow a_i + \delta$ and answer prefix sums $\sum_{j < i} a_j$.

A plain array answers updates in $\mathcal{O}(1)$ and sums in $\Theta(n)$; a prefix-sum array does the reverse.

The **Fenwick tree** [Fenwick 1994] answers both in $\mathcal{O}(\log n)$ with a single array $t[1..n]$, where slot i stores the sum of the range $(i - \text{lowbit}(i), i]$ and $\text{lowbit}(i) = i \& (-i)$ is the lowest set bit of i . A prefix sum walks $i \rightarrow i - \text{lowbit}(i)$, an update walks $i \rightarrow i + \text{lowbit}(i)$:

$$\sum_{j < i} a_j = t[i] + t[i - \text{lowbit}(i)] + \dots, \quad \text{update at } i : t[i] \leftarrow t[i] + \delta, t[i + \text{lowbit}(i)] \leftarrow t[i + \text{lowbit}(i)] + \delta, \dots \quad (11.1)$$

Each walk visits at most $\lceil \log_2(n + 1) \rceil$ slots, because each step clears (or carries past) one bit. The Fenwick tree is an implicit tree whose parent pointers are **computed** from the index by bit arithmetic — the same mechanism as the heap numeration (M-ARITH), with density exactly 1: it uses n slots for n elements. The **segment tree** is the heap-numbered binary tree over the array's leaves (leaves at $n, \dots, 2n - 1$, node i with children $2i, 2i + 1$), supporting any associative operation in $\mathcal{O}(\log n)$.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import Cost
from cgtsdk.algorithms import Fenwick, SegmentTree, SparseTable

n = 1 << 16
a = [(7 * i) % 101 for i in range(n)]
fw, sg = Fenwick(a), SegmentTree(a)
st_cost = Cost(); st = SparseTable(a, cost=st_cost)
c1, c2, c3 = Cost(), Cost(), Cost()
l, r = 12345, 54321
assert fw.range_sum(l, r, c1) == sg.query(l, r, c2) == sum(a[l:r])
assert st.query(l, r - 1, c3) == min(a[l:r])
print(f"Fenwick range sum: {c1.total} steps    segment tree: {c2.total} steps    (log2 n = 16)")
print(f"sparse-table range min: {c3.total} steps after a build of {st_cost.total} steps and {st.size_words} words")
```

OUTPUT (CHECKED)

```
Fenwick range sum: 39 steps    segment tree: 46 steps    (log2 n = 16)
sparse-table range min: 6 steps after a build of 1835044 steps and 983058 words
```

The sparse table is the opposite mechanism: **precomputation** (M-PRE). It stores the minimum of every range of length 2^j , after which any range minimum is the minimum of two overlapping stored ranges — $\mathcal{O}(1)$ per query — at the price of $\Theta(n \log n)$ build steps and words, and of total invalidation on any update. This is the LCA index of Chapter 5 in its general form. For idempotent operations (min, max, gcd), both mechanisms are available; which one wins depends on the ratio of queries to updates, exactly as in the break-even analysis of Chapter 12.

11.2 Dynamic programming as shared derivations

A dynamic program defines a quantity by a **recurrence** — a grammar of subproblems — and evaluates each subproblem once. For the longest common subsequence of strings x and y :

$$L(i, j) = \begin{cases} 0 & i = 0 \text{ or } j = 0, \\ L(i - 1, j - 1) + 1 & x_i = y_j, \\ \max(L(i - 1, j), L(i, j - 1)) & \text{otherwise.} \end{cases} \quad (11.2)$$

Read as a grammar, each production rewrites $L(i, j)$ into smaller instances; a naive evaluation expands a derivation *tree* with exponentially many nodes, while the table evaluates the derivation *DAG*, in which identical subproblems are shared. Dynamic programming is therefore the sharing mechanism (M-SHARE) applied to derivations — the same idea as the minimal DAG of a tree (Chapter 5) and the product construction's sharing of walks (Chapter 8). Its cost is the

number of distinct subproblems times the work per production: $\Theta(|x||y|)$ for LCS and edit distance, $\Theta(nW)$ for 0/1 knapsack with capacity W .

The last bound is a warning about what “polynomial” means. $\Theta(nW)$ is polynomial in the *value* W but exponential in its *length* $\log_2 W$: knapsack is NP-hard, and dynamic programming does not contradict that. It is the first instance in this book of a phenomenon Chapter 14 makes systematic — a grammar (here, of subproblems) whose size depends on numbers in the input rather than on the input’s length.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import itertools, random
from cgtsdk import Cost
from cgtsdk.algorithms import edit_distance, knapsack_01, lcs_length

c = Cost()
print("edit distance(kitten, sitting) =", edit_distance("kitten", "sitting", c), " table cells:",
c.counts["write"])
rng = random.Random(5)
w = [rng.randrange(1, 20) for _ in range(12)]; v = [rng.randrange(1, 30) for _ in range(12)]
for W in (10, 100, 1000):
    c = Cost()
    best = knapsack_01(w, v, W, c)
    brute = max(sum(v[i] for i in S) for k in range(13) for S in itertools.combinations(range(12), k) if
sum(w[i] for i in S) ≤ W)
    print(f"knapsack W={W:5d}: best={best} (brute force {brute}) DP cells={c.counts.get('write', 0)}")
```

OUTPUT (CHECKED)

```
edit distance(kitten, sitting) = 3 table cells: 42
knapsack W= 10: best=57 (brute force 57) DP cells=39
knapsack W= 100: best=147 (brute force 147) DP cells=1102
knapsack W= 1000: best=151 (brute force 151) DP cells=11902
```

The number of table cells grows linearly with W — ten times larger W , about ten times more work — even though the input grew by only one decimal digit per item.

Parsing is the original instance. The CYK algorithm decides membership of a string of length ℓ in a context-free language by a dynamic program over substrings and nonterminals in $\mathcal{O}(\ell^3)$ time — the same table-over-pairs pattern as CFL-reachability in Chapter 8 [Yannakakis 1990].

11.3 String matching as an automaton

Problem. Find all occurrences of a pattern p of length k in a text of length n .

The naive algorithm compares p at every shift: $\Theta(nk)$ comparisons in the worst case. The Knuth–Morris–Pratt algorithm [Knuth et al. 1977] precomputes the **failure function** of p in $\mathcal{O}(k)$ — for each prefix, the longest proper border — which is a compact encoding of the deterministic automaton recognizing Σ^*p . Running that automaton over the text never moves backwards in the text, and makes at most $2n$ comparisons.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import Cost
from cgtsdk.algorithms import kmp_find_all, naive_find_all

text, pat = "a" * 5000, "a" * 60 + "b"
c1, c2 = Cost(), Cost()
assert kmp_find_all(text, pat, c1) == naive_find_all(text, pat, c2) == []
print(f"naive: {c2.counts['compare']} comparisons    KMP (including preprocessing): {c1.counts['compare']}
      comparisons    2n = {2 * len(text)}")
```

OUTPUT (CHECKED)

```
naive: 301340 comparisons    KMP (including preprocessing): 10059 comparisons    2n = 10000
```

KMP is M-PRE: a small amount of preprocessing *of the query* (the pattern), producing an automaton — a grammar — that is then executed over the data. The trie of Chapter 4 does the same for a set of keys, and the Aho–Corasick automaton for a set of patterns. String matching is one domain where “compile the query into an automaton, then run it” is the classical algorithm, not a reinterpretation.

EXERCISE 11.1

Prove that $i - \text{lowbit}(i)$ clears the lowest set bit of i , and hence that a Fenwick prefix sum visits at most $\lceil \log_2(n + 1) \rceil$ slots.

EXERCISE 11.2

Write the recurrence for edit distance as a grammar of subproblems and draw the derivation DAG for $x = ab$, $y = ba$. How many nodes would the derivation *tree* have?

EXERCISE 11.3

Compute the KMP failure function of $p = abacabab$ and draw the corresponding automaton for Σ^*p over $\{a, b, c\}$.

► Solution to [Exercise 11.3](#)

EXERCISE 11.4

For range-minimum queries under updates, compare a segment tree with a sparse table rebuilt after each update. Using the counted costs of the SDK, find the number of queries per update at which the sparse table becomes cheaper for $n = 2^{12}$.

SUMMARY

- Fenwick and segment trees are implicit trees addressed by bit arithmetic (M-ARITH): $\mathcal{O}(\log n)$ updates and queries with density-one storage.
 - Sparse tables trade $\Theta(n \log n)$ preprocessing for $\mathcal{O}(1)$ queries (M-PRE) and break on updates.
 - Dynamic programming evaluates a grammar of subproblems on its derivation DAG — sharing (M-SHARE); pseudo-polynomial bounds are polynomial in numbers, not in input length.
 - KMP compiles the pattern into an automaton in $\mathcal{O}(k)$ and matches in at most $2n$ comparisons — preprocessing the query into a grammar.
-

Cost Profiles, Preprocessing, and Information

LEARNING OBJECTIVES

- Account for the total cost of a workload: build, queries, updates and output, in a single formula.
- Compute break-even query volumes, including under invalidating updates.
- Prove that any representation answering reachability on all digraphs without consulting the graph needs at least $\lfloor n/2 \rfloor \lceil n/2 \rceil$ bits.
- Distinguish retrieving an answer from discovering it, and both from producing a witness.
- Read the experimental evidence that no strategy dominates under updates.

12.1 Total cost

A resolution scheme (Definition 7.4) has a cost profile $(T_{\text{build}}, S, T_{\text{query}}, T_{\text{update}}, |\text{out}|)$. For a workload of Q queries and U updates the cost that matters is

$$T_{\text{total}} = T_{\text{build}} + \sum_{i=1}^Q T_{\text{query},i} + \sum_{j=1}^U T_{\text{update},j}, \quad (12.1)$$

plus the cost of writing the outputs, and subject to the space S fitting in memory. Every “fast query” claim must be read against (12.1). A query answered in $\mathcal{O}(1)$ after $\Theta(n^2)$ preprocessing is a statement about the second term only.

PROPOSITION 12.1 (Break-even under invalidating updates)**PROVED HERE**

CGT-PROP-005

Suppose updates invalidate a precomputed index, splitting the query stream into epochs with $Q_1, \dots, Q_k$ queries, and the index is rebuilt (cost B) at the first query of each non-empty epoch. Against a baseline without preprocessing (query cost q_0) and with indexed query cost $q_1 < q_0$, the index is cheaper iff

$$B \cdot |\{j : Q_j > 0\}| < (q_0 - q_1) \sum_j Q_j;$$

a sufficient condition is $Q_j > Q^* = B/(q_0 - q_1)$ for every non-empty epoch.

Proof.

Sum the costs epoch by epoch: the index pays $B + Q_j q_1$ in each non-empty epoch and the baseline $Q_j q_0$. ■

The formula is trivial; its consequences are not. In the experiments on random graphs with $n = 4096$ (CGT-OBS-004), the break-even query volume for a precomputed transitive closure against breadth-first search ranged from about 4 queries on dense graphs (where a giant strongly connected component lets the closure share rows) to about 27,000 on sparse ones (where random queries are mostly negative and search stops early). The same index is an obvious win or an obvious loss depending on a structural property of the input and on the query distribution.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import Cost, break_even
from cgtsdk.algorithms import bfs, transitive_closure
from cgtsdk.lab import random_digraph
import random

rng = random.Random(1)
for deg in (0.8, 3.0):
    adj = random_digraph(1024, deg, seed=2)
    build = Cost(); rows = transitive_closure(adj, build)
    q0 = Cost()
    for _ in range(300):
        bfs(adj, rng.randrange(1024), q0)    # search without an index (full BFS from a random source)
    q1 = 1                                    # one bit test per indexed query
    Qstar = break_even(build.total, q1, q0.total / 300)
    print(f"avg out-degree {deg}: closure build {build.total} steps, BFS {q0.total/300:.1f} steps/query →
break-even after {Qstar:.0f} queries")
```

OUTPUT (CHECKED)

avg out-degree 0.8: closure build 14742 steps, BFS 10.0 steps/query → break-even after 1637 queries
 avg out-degree 3.0: closure build 8208 steps, BFS 4565.1 steps/query → break-even after 2 queries

(Here each BFS runs to completion rather than stopping at the target, so the dense case is even more favourable to the index than in the ledger’s experiment.)

12.2 The information a grammar must contain

The strongest form of the founding conjecture would be a small “grammar” from which every reachability question about a graph can be *read off*, without searching the graph. Information theory forbids it.

LEMMA 12.2 (Counting lemma)

[PROVED HERE](#)

[CGT-THM-002](#)

Let a scheme map each structure $\mathfrak{A}$ in a class $\mathcal{K}$ to a bit string $E(\mathfrak{A})$ from which every query of a family $\mathcal{Q}$ can be answered without access to $\mathfrak{A}$. If the structures of $\mathcal{K}$ induce N distinct answer functions, then some encoding has length at least $\lceil \log_2(N + 1) \rceil - 1$, and fewer than $N \cdot 2^{-c}$ answer functions have encodings shorter than $\log_2 N - c$ bits.

Proof.

Structures with different answer functions need different encodings. There are $2^{L+1} - 1$ strings of length at most L , so $2^{L+1} - 1 \geq N$; and there are fewer than $2^{\log_2 N - c} = N2^{-c}$ strings shorter than $\log_2 N - c$. ■

THEOREM 12.3 (Information lower bound for retrieval-only reachability)

[PROVED HERE](#)

[CGT-THM-002](#)

Any scheme that answers all reachability queries “does s reach t ?” on all digraphs with vertex set $[n]$, without consulting the graph, uses at least $\lfloor n/2 \rfloor \lceil n/2 \rceil$ bits on some digraph; and on all but a fraction smaller than 2^{-c} of the bipartite graphs below it uses at least $\lfloor n/2 \rfloor \lceil n/2 \rceil - c$ bits.

Proof.

Split $[n]$ into X of size $a = \lfloor n/2 \rfloor$ and Y of size $b = \lceil n/2 \rceil$. For each $S \subseteq X \times Y$ let G_S have edge set S . Every path in G_S has length at most one, so s reaches $t \neq s$ iff $(s, t) \in S$. The 2^{ab} graphs have pairwise different answer functions; apply [Lemma 12.2](#) with $N = 2^{ab}$. ■

COROLLARY 12.4**PROVED HERE**

No grammar, index or labeling of $o(n^2)$ bits answers reachability on all digraphs without reading the graph; the transitive closure (n^2 bits) is optimal for this task up to a factor of 4. Sub-quadratic schemes must either restrict the class of graphs or read the graph at query time — that is, search.

The asymptotic version for acyclic graphs follows from the enumeration of partial orders by Kleitman and Rothschild [Kleitman & Rothschild 1975] ($\log_2$ of their number is $\sim n^2/4$); the bipartite family above gives the exact, non-asymptotic bound in one line. The result is elementary and almost certainly folklore. Its role here is to rule out precisely the reading of the founding conjecture that says a sufficiently expressive grammar could make reachability a constant-time lookup *in general*. The experiments agree: a general-purpose compressor could not shrink bipartite-family closures below $n^2/4$ bits (281,960 bits against the bound 262,144 at $n = 1024$, [CGT-OBS-005](#)).

12.2.1 Restricted classes can be cheap

The lower bound applies to *all* digraphs. Restricted classes have less information, and for them the grammar of the class can be the optimal index.

PROPOSITION 12.5**PROVED HERE**[CGT-THM-002](#)

For rooted forests on $[n]$, assigning each vertex its DFS entry and exit times ($\text{in}(v), \text{out}(v)$) answers “ u is an ancestor of v ” by $\text{in}(u) \leq \text{in}(v) \wedge \text{out}(v) \leq \text{out}(u)$, in $\mathcal{O}(1)$, with $\mathcal{O}(n \log n)$ bits after $\mathcal{O}(n)$ preprocessing [Agrawal et al. 1989]. This is optimal up to a constant factor: there are $(n + 1)^{n-1}$ labelled rooted forests, each with a distinct ancestor relation.

Proof.

The nesting property of DFS intervals gives correctness. Cayley’s formula counts rooted forests on $[n]$ as trees on $n + 1$ vertices. The ancestor relation determines each vertex’s parent as its deepest proper ancestor, so distinct forests have distinct answer functions; the counting lemma gives $\log_2(n + 1)^{n-1} = \Theta(n \log n)$ bits. ■

This is mechanism M-RESTRICT in its clearest form: the nesting structure that *defines* forests is itself a compact index, and the advantage comes from the restriction of the class, not from having a grammar. 2-hop labelings extend the idea to graphs whose reachability has small covers [Cohen et al. 2003].

12.3 Retrieval, discovery, witnesses

Three different things are often all called “answering a query”:

1. **Retrieval:** reading an answer that was computed earlier (one bit of a closure).
2. **Discovery:** computing the answer from the structure (a search).
3. **Witness production:** exhibiting *why* — the actual path, of length ℓ .

Retrieval can be $\mathcal{O}(1)$; discovery cannot be cheaper than reading enough of the structure to decide; witness production costs at least the output size. A closure answers “is there a path?” by retrieval but still needs search-like work to produce the path: walking from s along successors that still reach t costs the sum of their out-degrees ([CGT-NEG-011](#)). Storing next-hop pointers for all pairs makes witness production output-linear, at $\Theta(n^2 \log n)$ bits — whether that is optimal is an open question ([CGT-OPEN-009](#)).

12.4 When the structure changes

Precomputation assumes the structure holds still. The ledger’s dynamic-workload experiment ([CGT-EXP-005](#)) ran nine streams of 4,000 operations on a digraph with $n = 1024$, comparing BFS per query, a lazily rebuilt closure, an incrementally maintained closure (insertions handled by OR-ing rows, deletions forcing a rebuild [Italiano 1986]), and memoized BFS. Four different strategies won:

WORKLOAD	BFS	LAZY REBUILD	INCREMENTAL	MEMOIZED BFS	CHEAPEST
queries only, uniform	62,223	21,641	21,641	66,121	rebuild / incremental
queries only, repetitive	30,216	21,641	21,641	4,077	memo
90% queries, 10% insertions, uniform	242,024	6,775,863	602,748	245,653	BFS
90% queries, 10% insertions, adversarial	1,780,567	7,546,028	661,186	1,770,413	incremental

WORKLOAD	BFS	LAZY REBUILD	INCREMENTAL	MEMOIZED BFS	CHEAPEST
50% queries, mixed updates, uniform	26,624	17,535,647	12,550,168	28,608	BFS

(Counted steps; full table in the ledger.) With 10% insertions, epochs contain about ten queries while $Q^* \approx 1,400$, so the lazily rebuilt index costs 28 times as much as plain search — the break-even proposition at work. No strategy dominates; the winner is decided by the query-update ratio, the share of deletions, and the repetitiveness of queries ([CGT-OBS-009](#)).

Sharing suffers from updates in the same way: k adversarial relabellings inflate a minimal DAG of a full binary tree to $\Theta(k \log n)$ nodes (Chapter 5, [CGT-THM-005](#)).

12.5 An accounting of the mechanisms

MECHANISM	WHAT IT REDUCES	WHAT IT COSTS	WHAT DEFEATS IT
M-ARITH	query time, space for addresses	requires dense numerations	sparse or deep shapes (Ch. 5); non-uniform shapes (Ch. 13)
M-PRE	query time	build time, space $\geq$ information content	updates; the counting lemma
M-SHARE	space	navigation overhead	updates; repetition of the wrong kind
M-RESTRICT	everything, on the class	applicability	inputs outside the class
M-PRUNE	search space	up to a factor $ Q $ more work	permissive languages (Ch. 8)

EXERCISE 12.1

Derive a version of [Proposition 12.1](#) for an index that supports *incremental* updates at cost u each instead of rebuilds. When is incremental maintenance better than lazy rebuilding?

► Solution to [Exercise 12.1](#)

EXERCISE 12.2

Use the counting lemma to give a lower bound on the space of any scheme answering “is u adjacent to v ?” on all undirected graphs with n vertices without consulting the graph. Compare it with the adjacency matrix.

EXERCISE 12.3

The counting lemma says nothing about schemes that read the graph at query time. Give a scheme with $\mathcal{O}(n)$ extra bits that answers reachability on DAGs faster than BFS for some queries, and say which mechanism it uses.

SUMMARY

- Total cost = $T_{\text{build}} + \sum T_{\text{query}} + \sum T_{\text{update}}$ plus output; an index pays only after $Q^* = B/(q_0 - q_1)$ queries per epoch ([CGT-PROP-005](#)).
 - Retrieval-only reachability on all digraphs needs $\lfloor n/2 \rfloor \lceil n/2 \rceil$ bits ([CGT-THM-002](#)); no grammar evades the counting lemma. Restricted classes such as forests have compact optimal indexes.
 - Retrieval, discovery and witness production are different costs.
 - Under updates no strategy dominates; the measured winners vary with the workload.
 - Each mechanism has a cost and a class of inputs that defeats it.
-

Compiling Operations: When Grammar Becomes Arithmetic

LEARNING OBJECTIVES

- Define arithmetically compilable structures and explain what compilation buys: one-time cost proportional to the word, then constant cost per application.
- Prove that every move word over $\{L, R, U\}$ in a heap-shaped binary tree compiles to a map $x \mapsto ((x \gg k) \ll j) \mid c$ defined exactly on an interval.
- Prove that no constant-size compiled form exists for arbitrary tree shapes, by a counting argument.
- Recognize compilable structures beyond trees: de Bruijn graphs, hypercubes, grids.
- Read the experimental evidence, including the workload under which compilation barely helps.

13.1 The idea

Chapter 3 made an observation about lists: a navigation word such as $next^3prev^5next^4$ is determined by three numbers — its net displacement and its minimum and maximum prefix displacements — however long it is. If a representation could jump by a computed displacement, a word of a million moves could be *compiled* once into those three numbers and then applied in constant time. A linked list cannot jump; an array can.

This is the founding question of the book in its sharpest form. A move word is a sentence of the structure's operational grammar. Can the *sentence itself* be turned into machinery — a single cheap operation equivalent to the whole sentence — and on which structures?

DEFINITION 13.1 (Arithmetically compilable structure)

CGT-DEF-011

A deterministic operational grammar is **arithmetically compilable** on a class $\mathcal{K}$ of structures if there are a numeration ν of elements and a class $\mathcal{F}$ of partial maps on numerals such that

1. every move acts on numerals as some $f \in \mathcal{F}$;
2. $\mathcal{F}$ is closed under composition, including conjunction of domains;
3. every $f \in \mathcal{F}$ is stored in $\mathcal{O}(1)$ words, and composing, applying and testing the domain of members of $\mathcal{F}$ cost $\mathcal{O}(1)$ word operations.

Then every move word w **compiles** in $\mathcal{O}(|w|)$ word operations into one element of $\mathcal{F}$, which is applied afterwards in $\mathcal{O}(1)$, independently of $|w|$.

The definition is a cost-oriented specialization of an old idea: the meanings of all words form the **transition monoid** of the structure (CGT-DEF-007), and compilation asks for a representation of that monoid with constant-size elements and constant-time operations. Automatic structures [Blumensath & Grädel 2000; Khousainov & Nerode 1995] and automatic groups [Epstein et al. 1992] ask a related question — whether elements and moves are recognized by finite automata — but not this one: there, applying a move to a word-encoded element costs time linear in the element’s encoding.

13.2 The heap-shaped tree

Let T_n be the binary tree whose nodes are exactly the numerals $1, \dots, n$ (the shape of a binary heap with n elements), with moves

$$L : x \mapsto 2x, \quad R : x \mapsto 2x + 1, \quad U : x \mapsto \lfloor x/2 \rfloor.$$

A word $w \in \{L, R, U\}^*$ is **defined at** x if every numeral visited after each step lies in $[1, n]$. Evaluating w step by step costs $\Theta(|w|)$ (or less, when it falls off the tree early).

THEOREM 13.2 (Word compilation in heap-shaped trees)**PROVED HERE**

CGT-THM-006

For every word $w \in \{L, R, U\}^*$ and every $n \geq 1$ there are integers $k, j \geq 0$, $0 \leq c < 2^j$ and lo, hi , computable from (w, n) with $\mathcal{O}(|w|)$ arithmetic operations, such that for every $x \in [1, n]$:

$$\llbracket w \rrbracket(x) \text{ is defined } \iff lo \leq x \leq hi, \quad \text{and then} \quad \llbracket w \rrbracket(x) = \left\lfloor \frac{x}{2^k} \right\rfloor \cdot 2^j + c. \quad (13.1)$$

With early termination (stop as soon as the domain is empty), all intermediate integers have $\mathcal{O}(\log n)$ bits; for $n < 2^w$ compilation costs $\mathcal{O}(|w|)$ word operations and each application costs $\mathcal{O}(1)$.

Proof.

Normal form of prefixes. We show by induction on i that the map p_i computed by the first i letters (ignoring definedness) is $p_i(x) = \lfloor x/2^{k_i} \rfloor \cdot 2^{j_i} + c_i$ with $0 \leq c_i < 2^{j_i}$. The empty word gives $(0, 0, 0)$. Appending L or R gives $(k, j+1, 2c+b)$ with $b \in \{0, 1\}$. Appending U when $j > 0$ gives $(k, j-1, \lfloor c/2 \rfloor)$, because $\lfloor (2y+b)/2 \rfloor = y$: an up-move cancels the last down-move. Appending U when $j = 0$ gives $(k+1, 0, 0)$.

Monotonicity. Each p_i is non-decreasing in x , as a composition of non-decreasing maps.

Domain, lower end. w is defined at x iff $1 \leq p_i(x) \leq n$ for every prefix $i \geq 1$. A prefix value can drop below 1 only at an up-move taken with $j = 0$, which requires $\lfloor x/2^k \rfloor \geq 1$, i.e. $x \geq 2^k$ for the new k . Since k never decreases, these conditions together say $x \geq 2^{k_{\max}} =: lo$ (and $lo \geq 1$).

Domain, upper end. If $c_i > n$, the prefix value exceeds n for every x and the domain is empty. Otherwise, because $c_i < 2^{j_i}$, $p_i(x) \leq n$ iff $\lfloor x/2^{k_i} \rfloor \leq \lfloor (n - c_i)/2^{j_i} \rfloor$, i.e.

$$x \leq T_i := \left(\left\lfloor \frac{n - c_i}{2^{j_i}} \right\rfloor + 1 \right) 2^{k_i} - 1.$$

Take $hi = \min(n, \min_i T_i)$. Each T_i is a threshold because p_i is non-decreasing; the conjunction of all lower and upper conditions is therefore the interval $[lo, hi]$.

Cost. Each letter updates (k, j, c, lo, hi) with a constant number of operations. Once $j > \lfloor \log_2 n \rfloor$ or $c > n$ the domain is empty and compilation may stop; until then all quantities have $\mathcal{O}(\log n)$ bits, and $k \leq |w|$ with 2^k needed only while $2^k \leq n$. ■

The proof is short, but the statement was checked mechanically as well: for all words of length at most 7, all $n \leq 40$ and all x , the compiled map agrees with step-by-step evaluation — more than two million cases — and for 300 random long words on trees with up to 5,000 nodes (`sdk/tests/test_addressing.py`). The browser laboratory's implementation is checked against the SDK's.

```

import random
from cgtsdk import Cost
from cgtsdk.addressing import HeapWord, heap_walk

n = 10**6
rng = random.Random(1)
x, w = 6000, []
for _ in range(2000):          # a 2000-letter walk that stays inside the tree
    opts = [(a, y) for a, y in (("L", 2*x), ("R", 2*x + 1), ("U", x // 2)) if 1 ≤ y ≤ n]
    a, x = rng.choice(opts); w.append(a)
compile_cost = Cost()
f = HeapWord.compile(w, n, compile_cost)
print(f"|w| = {len(w)} → f(x) = ((x >> {f.k}) << {f.j}) | {f.c}, defined on [{f.lo}, {f.hi}]")
apply_cost, walk_cost = Cost(), Cost()
starts = [rng.randrange(4096, 8192) for _ in range(1000)]
assert all(f(s, apply_cost) == heap_walk(w, s, n, walk_cost) for s in starts)
print(f"compile once: {compile_cost.total} ops; per application: compiled {apply_cost.total/1000:.0f}, step-
by-step {walk_cost.total/1000:.0f}")

```

OUTPUT (CHECKED)

```

|w| = 2000 → f(x) = ((x >> 3) << 9) | 486, defined on [8, 7807]
compile once: 8000 ops; per application: compiled 5, step-by-step 3640

```

A word of two thousand moves collapses to “shift right by 3, shift left by 9, set the low nine bits to 486”, defined on the interval $[8, 7807]$. After a one-time cost of 8,000 operations, each application costs 5 instead of about 3,600.

13.3 The lower bound: arbitrary shapes

Why does the heap-shaped tree compile and a linked list or an arbitrary tree not? Because in T_n the semantics of every move is determined by a *single number*, n . In an arbitrary tree, even the domain of a single move carries a lot of information.

THEOREM 13.3 (No constant-size compilation on arbitrary shapes)

[PROVED HERE](#)

CGT-THM-007

Let a scheme encode each binary tree T with n nodes (nodes named by their addresses) as a bit string $E(T)$ from which it can be decided, for every node, whether the move L is defined there, without access to T . Then some tree needs $|E(T)| \geq \lfloor n/2 \rfloor$ bits.

Proof.

Consider caterpillars: a right spine $1^0, 1^1, \dots, 1^{s-1}$ with left leaves attached at the spine positions in a set $S \subseteq \{0, \dots, \lfloor n/2 \rfloor - 1\}$, and $s = n - |S| \geq \lceil n/2 \rceil$ so that every position of S is on the spine. Then $\text{dom}[L] = \{1^i : i \in S\}$. The $2^{\lfloor n/2 \rfloor}$ caterpillars have pairwise different answer functions, and the counting lemma (Lemma 12.2) applies. ■

So for arbitrary shapes the best possible “compiled” form of a word is essentially a table of its values at all n nodes, built by simulating the word from every node at cost $\Theta(n|w|)$. The theorem was also checked by enumerating all binary trees with up to 11 nodes
(`experiments/check_theorems.py`).

Together the two theorems locate the boundary of mechanism M-ARITH for trees precisely: **compilation to constant size is possible exactly when the move semantics is fixed by a constant number of parameters of the structure, and impossible when the shape itself carries the information.** The general form of this statement — a characterization of compilable classes — is open ([CGT-OPEN-012](#)).

13.4 More compilable structures

The pattern is not special to trees. Three further structures compile with density 1, each with a different algebra:

PROPOSITION 13.4 (De Bruijn graphs, hypercubes and grids are compilable)

[PROVED HERE](#)

[CGT-PROP-008](#)

1. **De Bruijn graph** $B(k, m)$: vertices are base- k numerals of m digits; the move “shift in digit a ” is $x \mapsto (kx + a) \bmod k^m$. Affine maps modulo N are closed under composition: $(a_2, b_2) \circ (a_1, b_1) = (a_2a_1, a_2b_1 + b_2) \bmod N$.
2. **Hypercube** Q_d : the move “flip bit i ” is $x \mapsto x \oplus 2^i$; compositions are XORs with the XOR of the masks.
3. **Grid** $\prod_i [0, d_i)$ with moves $\pm e_i$ defined inside the box: a word compiles to a translation t together with the per-axis minimum and maximum prefix displacements (lo, hi) ; it is defined at x iff $x_i + lo_i \geq 0$ and $x_i + hi_i < d_i$ for all i ; composition adds translations and takes the minimum and maximum of shifted prefixes.

Proof.

Each item is a direct calculation of the closure property, as stated; the grid case is the multi-dimensional version of the cursor observation of Chapter 3. Each is checked exhaustively on small instances in the SDK’s tests.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import Cost
from cgtsdk.addressing import (AffineMod, BoxTranslation, XorMask, compile_word,
                               debruijn_moves, grid_moves, hypercube_moves)

import random

rng = random.Random(3)
cases = [("de Bruijn B(2,20)", debruijn_moves(2, 20), AffineMod.identity(2**20)),
         ("hypercube Q_20", hypercube_moves(20), XorMask(0)),
         ("grid 1000 x 1000", grid_moves((1000, 1000)), BoxTranslation.identity((1000, 1000)))]
for name, moves, ident in cases:
    w = [rng.choice(sorted(moves)) for _ in range(10_000)]
    c = Cost(); f = compile_word(moves, w, ident, c)
    print(f"{name:18s} 10,000-move word compiled in {c.total} ops into {f}")
```

OUTPUT (CHECKED)

```
de Bruijn B(2,20)  10,000-move word compiled in 10000 ops into AffineMod(a=0, b=782888, N=1048576)
hypercube Q_20    10,000-move word compiled in 10000 ops into XorMask(c=537303)
grid 1000 x 1000  10,000-move word compiled in 10000 ops into BoxTranslation(dims=(1000, 1000), t=(32, -78),
lo=(-51, -81), hi=(39, 35))
```

The de Bruijn word compiled to a *constant* map ($a = 0$): after more than 20 shifts the starting vertex is entirely forgotten, which the affine form exposes immediately. The grid word compiled to “move by (32, −78)”, defined for start points with coordinates at least (51, 81) and below (1000 − 39, 1000 − 35).

13.5 What the experiments show

The ledger’s experiment ([CGT-EXP-008](#)) measured compilation against step-by-step evaluation under two workloads, a lesson learned from earlier experiments ([CGT-D-017](#)).

WORKLOAD ON T_{10^6}	$ W $	COMPILED OPS / APPLICATION	STEP-BY-STEP OPS / APPLICATION	APPLICATIONS TO BREAK EVEN
words generated as walks (≈90% of starts	8	5	16	2.9

WORKLOAD ON T_{10^6}	$ W $	COMPILED OPS / APPLICATION	STEP-BY-STEP OPS / APPLICATION	APPLICATIONS TO BREAK EVEN
defined)				
	512	5	933	2.2
	4096	5	7,420	2.2
random words (most starts fall off early)	512	5	29	86
	4096	5	130	131

When words stay inside the tree, compilation pays after about two applications. When random words fall off the tree within a few steps, step-by-step evaluation stops early and compilation needs over a hundred applications to pay. On random pointer trees with $n = 20,000$, the same kind of walk-generated word was defined at only 0.1% of nodes for $|w| = 512$ — the non-uniformity that [Theorem 13.3](#) lower-bounds ([CGT-OBS-014](#)).

CLAIM BOUNDARY

Compilation is a statement about *repeated application of the same word*. It does not make a single evaluation cheaper (compiling costs as much as evaluating), and it applies only to structures whose move semantics is uniform. Its novelty as a normal form has not been verified against the literature on implicit tree navigation and on transition monoids; it is presented as proved here, not as new.

EXERCISE 13.1

Compile $w = L R R U U L$ for $n = 100$ by hand, following the proof of [Theorem 13.2](#). Check your (k, j, c, lo, hi) with the SDK or the laboratory.

► [Solution to Exercise 13.1](#)

EXERCISE 13.2

Generalize [Theorem 13.2](#) to heap-shaped k -ary trees with moves “child i ” and “parent”. What replaces shifts?

EXERCISE 13.3

Show that the grid compiled form of [Proposition 13.4](#) is closed under composition by proving the composition formula, and give the compiled form of the reverse of a word in terms of the word's compiled form.

EXERCISE 13.4

Prove that the moves *next* and *prev* on a *circular* doubly linked list of known length n compile into translations modulo n — and explain why this does not make a linked list as fast as an array.

► [Solution to Exercise 13.4](#)

SUMMARY

- A structure is arithmetically compilable when its moves act on numerals through a composition-closed class of constant-size maps; then any word compiles once and applies in constant time.
 - In heap-shaped trees every word over $\{L, R, U\}$ compiles to $x \mapsto ((x \gg k) \ll j) \mid c$ on an interval domain ([CGT-THM-006](#)).
 - For arbitrary shapes no constant-size compiled form exists: one move's domain can carry $\lfloor n/2 \rfloor$ bits ([CGT-THM-007](#)).
 - De Bruijn graphs, hypercubes and grids compile too, via affine maps, XOR masks and bounded translations ([CGT-PROP-008](#)).
 - Measured: compilation pays after about two applications when words stay defined, and after over a hundred when early exits make step-by-step evaluation cheap.
-

The Limits: Advice, Restriction, and P versus NP

LEARNING OBJECTIVES

- State, for each classical boundary of complexity theory touched by this program, the exact theorem or model being questioned, the CGT alternative proposed, and the proof obligation it would have to meet.
- Prove that, within standard models, attaching a grammar to inputs cannot move a problem across the P/NP boundary — and identify exactly the four ways in which claims to the contrary go wrong.
- Distinguish tractability that comes from a restricted class (2-SAT) from tractability that a grammar would have to supply.
- Read the solution-grammar experiment for SAT, and why it does not settle anything asymptotically.

14.1 The challenge, made explicit

The research program began with an ambitious conjecture: if computational structures have sufficiently expressive grammars, certain problems might become dramatically easier — perhaps resolvable in constant or near-constant time. A conjecture of that kind is only worth stating if it is stated against something precise. The table lists each classical boundary this book has touched, what CGT proposed against it, and what a proof would have to show.

ESTABLISHED RESULT OR MODEL	CGT PROPOSAL	REQUIRED ASSUMPTIONS	PROOF OBLIGATION	OUTCOME IN THIS BOOK
Comparison sorting needs $\log_2 n!$ comparisons	grammars of sorted orders	comparison model	a sort with fewer comparisons	impossible in the model; escape only by addressing

ESTABLISHED RESULT OR MODEL	CGT PROPOSAL	REQUIRED ASSUMPTIONS	PROOF OBLIGATION	OUTCOME IN THIS BOOK
				(counting sort), which is a change of model (Ch. 9)
Reachability needs search or stored answers	a reachability grammar answering in $\mathcal{O}(1)$	retrieval- only, all digraphs	encoding of $o(n^2)$ bits	impossible: $\lfloor n/2 \rfloor \lceil n/2 \rceil$ bits needed (CGT-THM-002)
Preprocessing- query trade- offs	grammatical indexes	static data	break-even and space bounds	indexes pay only after Q^* queries per epoch (CGT- PROP-005)
Constant-time navigation needs dense addressing	compiled move words	uniform move semantics	constant-size compiled form	established for heap- shaped trees, grids, hypercubes, de Bruijn graphs; impossible for arbitrary shapes (CGT- THM-006, CGT- THM-007)
Work-depth trade-offs in parallel evaluation	grammar shape as a cost parameter	semi-naive evaluation	exact counts	established for one pair of grammars (CGT-THM-004)
Constraints prune search	admissibility languages	regular languages	a speedup guarantee	false in general: a factor $ Q $ slowdown is

ESTABLISHED RESULT OR MODEL	CGT PROPOSAL	REQUIRED ASSUMPTIONS	PROOF OBLIGATION	OUTCOME IN THIS BOOK
				attained (CGT- THM-001)
P versus NP	expressive grammars resolve NP- hard problems quickly	standard Turing/RAM models	a polynomial algorithm for an NP-complete problem, or a separation proof	no route within standard models (Proposition 14.1)
Parameterized complexity	grammar- parameterized costs $T(n, g, q, u, o)$	—	a distinction not captured by existing parameterizations	open (CGT- OPEN-005)

Several rows are negative. That is the point of making the challenge explicit: a precise “no” is a scientific result, and it sharpens what the positive rows mean.

14.2 Grammars and the P/NP boundary

Let $\Pi \subseteq \{0, 1\}^*$ be a decision problem, and suppose a “grammar” Γ is attached to inputs to help decide Π . What Γ can do depends entirely on where it comes from and how large it is.

PROPOSITION 14.1 (Four ways a grammar can help, and what each implies)

ESTABLISHED

CGT-PROP-006

1. **Uniform and polynomial.** If $\Gamma(x)$ is computable in time $\text{poly}(|x|)$ and $\Pi(x)$ is decidable from $(x, \Gamma(x))$ in time $\text{poly}(|x|)$, then $\Pi \in \text{P}$. If Π is NP-complete, this means $\text{P} = \text{NP}$.
2. **Advice.** If Γ depends only on $|x|$, has size $\text{poly}(|x|)$, and Π is decidable from $(x, \Gamma_{|x|})$ in polynomial time, then $\Pi \in \text{P/poly}$. For NP-complete Π , this would collapse the polynomial hierarchy to its second level [Karp & Lipton 1980] — widely believed false.
3. **Unbounded size.** For every decidable Π , letting Γ_n be the truth table of Π on inputs of length n (2^n bits) answers every query by a single lookup; building Γ_n costs at least 2^n decisions. Nothing follows about P versus NP.
4. **Restriction.** Restricting Π to a class C of inputs may make it polynomial while Π remains NP-complete (2-CNF versus 3-CNF [Aspvall et al. 1979]). The tractability belongs to C .

Proof.

(1) is composition of polynomial-time computations. (2) is the definition of P/poly together with the Karp–Lipton theorem. (3) holds by construction. (4) is an example. ■

Read against the founding conjecture, the proposition says: within the standard models, a grammar that makes an NP-complete problem fast is either an ordinary polynomial-time algorithm in disguise (case 1 — a proof of $\text{P} = \text{NP}$, to be judged as such), or polynomial advice (case 2 — believed impossible), or an exponential lookup table (case 3 — retrieval, not discovery), or a restriction of the problem (case 4). There is no fifth door. The ledger records the strong form of the conjecture as **rejected** (CGT-NEG-009); what survives is the study of which restricted classes and which preprocessing regimes allow fast resolution — the subject of the rest of the book.

CLAIM BOUNDARY

Nothing in this book proves or disproves $\text{P} = \text{NP}$, and no claim is made that Computational Grammar Theory bears on the question beyond Proposition 14.1. A fast benchmark on chosen instances, a new notation, or an algorithm that is fast on a restricted class is not evidence about the unrestricted problem.

14.3 Restriction: 2-SAT

Satisfiability of CNF formulas with at most two literals per clause is solvable in linear time [Aspvall et al. 1979]: each clause $(a \vee b)$ is the pair of implications $\neg a \Rightarrow b$ and $\neg b \Rightarrow a$, and the

formula is satisfiable iff no variable shares a strongly connected component of this **implication graph** with its negation. The implication graph is a structural grammar of the formula, and the decision procedure is the SCC algorithm of Chapter 10. With three literals per clause the problem is NP-complete, and no comparable structure is known. The tractability comes from the restriction, not from having a grammar. (The large random instance below has clause-to-variable ratio 1.5, above the satisfiability threshold of random 2-CNF, and is unsatisfiable; deciding that also takes linear time.)

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import random
from cgtsdk import Cost
from cgtsdk.algorithms import brute_force_count, satisfies, two_sat

rng = random.Random(11)
agree = 0
for trial in range(300):
    n = rng.randrange(2, 13)
    clauses = [tuple(rng.choice([v, -v]) for v in rng.sample(range(1, n + 1), 2)) for _ in
range(rng.randrange(1, 2 * n))]
    a = two_sat(n, clauses)
    agree += (a is not None) == (brute_force_count(n, clauses) > 0) and (a is None or satisfies(a, clauses))
print(f"2-SAT via implication-graph SCCs agrees with brute force on {agree}/300 random formulas")
c = Cost()
big = [tuple(rng.choice([v, -v]) for v in rng.sample(range(1, 100_001), 2)) for _ in range(150_000)]
print("100,000 variables, 150,000 clauses:", "satisfiable" if two_sat(100_000, big, c) else "unsatisfiable",
      f"- {c.counts['edge']} edge inspections (linear in the formula)")
```

OUTPUT (CHECKED)

```
2-SAT via implication-graph SCCs agrees with brute force on 300/300 random formulas
100,000 variables, 150,000 clauses: unsatisfiable - 600000 edge inspections (linear in the formula)
```

14.4 Solution grammars for 3-SAT

The most expressive grammar one can attach to a satisfiability instance is a grammar generating exactly its satisfying assignments — in its canonical form, a layered deterministic automaton over $\{0, 1\}$, essentially an ordered binary decision diagram. Once it exists, satisfiability, model counting and many other queries are linear in its size: retrieval. This is **knowledge compilation**, a field that maps exactly this trade-off between the succinctness of a target representation and the queries it supports in polynomial time [Darwiche & Marquis 2002]. The question for CGT is what building it costs.

The ledger’s experiment ([CGT-EXP-006](#)) built the automaton by deduplicating residual clause sets, under the natural variable order, for random 3-CNF near the satisfiability threshold and random 2-CNF:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import random, math
from cgtsdk.algorithms import solution_automaton

rng = random.Random(6)
for label, width, ratio in (("3-CNF, m/k=4.26", 3, 4.26), ("2-CNF, m/k=1.0", 2, 1.0)):
    row = []
    for k in (8, 12, 16, 20, 24):
        sizes = []
        for _ in range(5):
            cls = [tuple((v + 1) * rng.choice([1, -1]) for v in rng.sample(range(k), width)) for _ in
range(round(ratio * k)))]
            sizes.append(solution_automaton(k, cls)[2])
        row.append(sorted(sizes)[2])
    print(f"{label:16s} median automaton nodes for k=8..24: {row}")
```

OUTPUT (CHECKED)

```
3-CNF, m/k=4.26 median automaton nodes for k=8..24: [47, 212, 875, 2835, 10855]
2-CNF, m/k=1.0 median automaton nodes for k=8..24: [36, 73, 174, 340, 1260]
```

The sizes grow quickly, but over this range an exponential fit and a polynomial fit of high degree describe the medians about equally well ($R^2 \approx 0.99$ and 0.98 in the ledger’s larger run, [CGT-OBS-011](#)). The data **cannot** decide the asymptotics, and the book makes no asymptotic claim from them. Two things can be said. First, 2-CNF automata grow too, more slowly: 2-SAT’s linear-time algorithm does not come from a small solution grammar. Second, if a polynomial-size solution grammar could always be *built* in polynomial time, satisfiability would be in P (case 1 of Proposition 14.1); whatever the asymptotics of these particular automata, the construction cost is where any claim must be checked. Known exponential lower bounds for decision-diagram representations of some formulas are part of the knowledge-compilation literature; connecting them precisely to this experiment is open ([CGT-OPEN-006](#)).

14.5 Grammars whose size depends on numbers

Chapter 11’s knapsack dynamic program is a grammar of subproblems with $\Theta(nW)$ productions, where W is a *number* in the input. It is polynomial in the value W and exponential in its length $\log_2 W$. Any claim that a grammar resolves a numerical NP-hard problem “in polynomial time” must say polynomial in *what*: an input of length ℓ can encode numbers as large as 2^ℓ .

14.6 What survives

Within standard models, grammars do not change complexity classes. What this book has established instead is a set of **precise local results**: where a grammar’s structure *is* a cheap algorithm (dense addressing, compiled words, restricted classes), where it is a disguised precomputation bounded by information content, and where it is a constraint whose cost can go either way. These are the results that a “challenge to the foundations” can honestly claim at present: not a new complexity class, but a sharper map of where grammatical structure becomes computational advantage and where it provably cannot.

EXERCISE 14.1

A colleague claims a “grammar-indexed SAT solver” answers every query in $\mathcal{O}(1)$ after loading a precomputed grammar. Which case of [Proposition 14.1](#) applies? List the questions you would ask to decide.

► [Solution to Exercise 14.1](#)

EXERCISE 14.2

Prove the correctness of the 2-SAT criterion: the formula is unsatisfiable iff some variable x lies in the same strongly connected component as $\neg x$ in the implication graph.

EXERCISE 14.3

For the formula $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$, build the solution automaton by residual deduplication under the order x_1, x_2, x_3 and count its models.

SUMMARY

- Each classical boundary touched by the program has been stated with its proof obligation; several outcomes are negative.
 - Within standard models a grammar helps decide an NP-complete problem only as a polynomial algorithm (a $P = NP$ claim), as polynomial advice (believed impossible), as an exponential table (retrieval), or through a restriction of the problem (CGT-PROP-006).
 - 2-SAT is tractable by restriction; 3-SAT solution grammars grow quickly, and the data cannot decide how quickly.
 - Grammar sizes that depend on numbers rather than lengths yield pseudo-polynomial, not polynomial, bounds.
 - What survives are precise local results about mechanisms, not changes of complexity class.
-

PageRank from First Principles

LEARNING OBJECTIVES

- Build the PageRank model from a directed graph: transition matrix, dangling pages, damping, teleportation.
- Prove that the PageRank vector exists and is unique, and that power iteration converges geometrically with rate α .
- Certify the error of an approximate PageRank vector from the last residual, and bound the number of iterations in advance.
- Compare power iteration and Gauss–Seidel, and explain why the vertex order matters for one and not the other.
- State the cost of one iteration in time, memory and communication, and the effect of graph updates and personalization.

15.1 The ranking problem

A collection of hyperlinked pages forms a directed graph: an edge $u \rightarrow v$ means page u links to page v . **PageRank** assigns each page a score meant to capture how much attention the link structure directs to it. It was introduced by Page, Brin, Motwani and Winograd [Page et al. 1999] and described as part of the prototype of the Google search engine by Brin and Page [Brin & Page 1998]. The mathematics is now standard [Langville & Meyer 2006; Langville & Meyer 2004; Bianchini et al. 2005].

CLAIM BOUNDARY

This chapter and the next treat PageRank as a published algorithm on mathematical graphs. They say nothing about any search engine’s current ranking systems, which combine many signals and are not public. No result here should be read as an improvement to, or an endorsement by, any company’s production search.

15.2 The random surfer

Let $G = (V, E)$ have n vertices, out-degrees $o(u)$, and **dangling** vertices $D = \{u : o(u) = 0\}$. Fix a **damping factor** $\alpha \in [0, 1)$ and a **teleportation distribution** v on V (uniform unless stated otherwise). A random surfer at u :

- with probability α follows one of u 's out-links uniformly at random — or, if u is dangling, jumps to a page drawn from v ;
- with probability $1 - \alpha$ jumps to a page drawn from v .

Write M for the $n \times n$ **column-stochastic** matrix of the link step with the dangling fix: $M_{wu} = 1/o(u)$ if $u \rightarrow w$, $M_{wu} = v_w$ if $u \in D$, and 0 otherwise. The surfer's distribution evolves by

$$x_{k+1} = T(x_k), \quad T(x) = \alpha Mx + (1 - \alpha) v. \quad (15.1)$$

DEFINITION 15.1 (PageRank vector)

The **PageRank vector** π is a probability vector with $T(\pi) = \pi$, i.e.

$$\pi = \alpha \left(P^\top \pi + \left(\sum_{u \in D} \pi_u \right) v \right) + (1 - \alpha) v, \quad (15.2)$$

where $P^\top$ distributes each non-dangling page's score equally over its out-links.

Three modelling choices are hidden in (15.2) and each matters in practice. **Dangling pages** (no out-links: images, PDFs, pages not yet crawled) would otherwise leak probability; here their mass is redistributed by v , one of several conventions [Langville & Meyer 2004]. **Damping** α (classically 0.85) controls how far the surfer follows links before teleporting. **Teleportation** v makes the chain irreducible and is the lever for personalization (§§15.7).

15.3 Existence, uniqueness, convergence

THEOREM 15.2 (Contraction of the PageRank map)

ESTABLISHED

CGT-THM-008

For probability vectors x, y , $\|T(x) - T(y)\|_1 \leq \alpha \|x - y\|_1$. Hence T has a unique fixed point π in the probability simplex; for every starting distribution x_0 , the iterates satisfy $\|x_k - \pi\|_1 \leq \alpha^k \|x_0 - \pi\|_1 \leq 2\alpha^k$; and, with residual $r_k = \|x_k - x_{k-1}\|_1$,

$$\|x_k - \pi\|_1 \leq \frac{\alpha}{1 - \alpha} r_k. \quad (15.3)$$

Proof.

$T(x) - T(y) = \alpha M(x - y)$. A column-stochastic matrix has non-negative entries with columns summing to one, so for any vector z , $\|Mz\|_1 = \sum_w |\sum_u M_{wu} z_u| \leq \sum_u |z_u| \sum_w M_{wu} = \|z\|_1$. Thus T is an α -contraction in L_1 on the simplex, which T maps to itself; Banach's fixed-point theorem gives existence, uniqueness and $\|x_k - \pi\| \leq \alpha^k \|x_0 - \pi\|$, and $\|x_0 - \pi\|_1 \leq 2$ for probability vectors. For (15.3):

$$\begin{aligned} \|x_{k-1} - \pi\|_1 &\leq \|x_{k-1} - x_k\|_1 + \|x_k - \pi\|_1 \leq r_k + \alpha \|x_{k-1} - \pi\|_1, \\ \text{so } \|x_{k-1} - \pi\|_1 &\leq \frac{r_k}{1-\alpha}, \quad \|x_k - \pi\|_1 \leq \alpha \|x_{k-1} - \pi\|_1 \leq \frac{\alpha}{1-\alpha} r_k. \end{aligned}$$

In spectral terms, $\alpha M + (1 - \alpha)v\mathbf{1}^\top$ is a column-stochastic matrix with eigenvalue 1, and its second eigenvalue has modulus at most α [Haveliwala & Kamvar 2003]; the contraction argument gives the rate directly without eigenvalues.

PROPOSITION 15.3 (A-priori iteration bound)

ESTABLISHED

CGT-PROP-009

Power iteration from $x_0 = v$, stopped as soon as $\frac{\alpha}{1-\alpha} r_k \leq \tau$, stops after at most

$$k^* = \left\lceil \frac{\log(\tau(1-\alpha)/2)}{\log \alpha} \right\rceil$$

iterations, and the returned vector is within τ of π in L_1 .

Proof.

$r_k = \|T(x_{k-1}) - T(x_{k-2})\| \leq \alpha^{k-1} r_1 \leq 2\alpha^{k-1}$, so $\frac{\alpha}{1-\alpha} r_k \leq \frac{2\alpha^k}{1-\alpha} \leq \tau$ once $\alpha^k \leq \tau(1-\alpha)/2$. The error claim is (15.3).

For $\alpha = 0.85$ and $\tau = 10^{-10}$, $k^* = 158$; for $\alpha = 0.99$, $k^* = 2,819$. The bound is a worst case. The SDK's tests check the per-step contraction against an exact linear solve, and the experiments found it respected on every instance:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk.algorithms import pagerank_exact, pagerank_power

adj = [[1, 2], [2], [0, 3], [], [0, 3, 2]] # page 3 is dangling
pi = pagerank_exact(adj, 0.85) # Gaussian elimination, the oracle
print("PageRank (exact):", [round(p, 6) for p in pi])
for k in (1, 2, 5, 10, 20, 40):
    x = pagerank_power(adj, 0.85, tol=0, max_iter=k)
    err = sum(abs(a - b) for a, b in zip(x.scores, pi))
    print(f"k={k:2d} true error {err:.2e} certified bound {x.error_bound:.2e} 2*alpha^k {2 * 0.85**k:.2e}")
```

OUTPUT (CHECKED)

```
PageRank (exact): [0.223817, 0.163171, 0.321147, 0.223817, 0.068049]
k= 1 true error 1.09e-01 certified bound 2.12e+00 2*alpha^k 1.70e+00
k= 2 true error 7.58e-02 certified bound 8.90e-01 2*alpha^k 1.44e+00
k= 5 true error 8.59e-03 certified bound 1.15e-01 2*alpha^k 8.87e-01
k=10 true error 4.02e-04 certified bound 6.80e-03 2*alpha^k 3.94e-01
k=20 true error 7.14e-07 certified bound 8.54e-06 2*alpha^k 7.75e-02
k=40 true error 1.23e-12 certified bound 2.15e-11 2*alpha^k 3.00e-03
```

The certified bound stays within a factor of about 10–20 of the true error and, after the first few iterations, is far tighter than the a-priori $2\alpha^k$ — this graph converges faster than the worst case, and the residual detects it.

15.4 Solvers

Equation (15.2) is also a linear system: $(I - \alpha M)\pi = (1 - \alpha)v$. Power iteration is the Jacobi method for it. **Gauss–Seidel** updates π_i using the newest values of the other entries within a sweep:

$$\pi_i \leftarrow \frac{(1 - \alpha)v_i + \alpha \left(\sum_{u \rightarrow i, u \neq i} \pi_u / o(u) + v_i \sum_{u \in D, u \neq i} \pi_u \right)}{1 - \alpha M_{ii}}.$$

The SDK implements both, plus an exact solver by Gaussian elimination used as an oracle on small graphs. The experiments ([CGT-EXP-009](#)) show that neither iterative method dominates. On a graph with host-like blocks and many cycles, Gauss–Seidel needed fewer sweeps than power iteration needed iterations (56 versus 104 at $\alpha = 0.85$). On an acyclic “copying-model” graph whose links all point to *earlier* vertices, sweeping in the natural order works *against* the direction of information flow, and Gauss–Seidel was dramatically slower (1,561 sweeps versus 37 iterations at $\alpha = 0.99$). Reversing the vertex order cut its sweeps from 110 to 34. Power iteration, by contrast, is invariant under reordering: the same linear map, the same iteration count and the same number of edge operations in any order ([CGT-OBS-016](#)). The ledger records “Gauss–Seidel is faster” as a rejected general hypothesis ([CGT-REJ-003](#)).

The convergence data also contain a surprise that this book cannot yet explain. On the acyclic copying-model graph, power iteration stopped after 17–37 iterations for every α up to 0.99, while k^* grew to 2,819; on the cyclic block graph, iterations grew from 27 to 485 as α went from 0.5 to 0.99 ([CGT-OBS-015](#)). A tempting explanation — without cycles, probability cannot circulate, so the iteration settles after about the length of the longest path — is contradicted by the next example, in which the acyclic graph converges *more slowly* than the cyclic one: dangling pages send their mass back through teleportation, which re-creates circulation.

```

from cgtsdk.algorithms import pagerank_gauss_seidel, pagerank_power
import math, random

rng, n = random.Random(4), 400
cyclic = [[(u + 1) % n, rng.randrange(n)] for u in range(n)] # a ring plus one random link each
acyclic = [[w for w in nbrs if w > u] for u, nbrs in enumerate(cyclic)] # keep only forward links
print("alpha    k*    power: cyclic acyclic    Gauss-Seidel: cyclic acyclic")
for alpha in (0.5, 0.85, 0.99):
    kstar = math.ceil(math.log(1e-10 * (1 - alpha) / 2) / math.log(alpha))
    p = [pagerank_power(g, alpha, tol=1e-10).iterations for g in (cyclic, acyclic)]
    s = [pagerank_gauss_seidel(g, alpha, tol=1e-10 * (1 - alpha)).iterations for g in (cyclic, acyclic)]
    print(f"{alpha:5}  {kstar:5d}          {p[0]:5d}  {p[1]:6d}          {s[0]:5d}  {s[1]:6d}")

```

OUTPUT (CHECKED)

alpha	k*	power: cyclic	acyclic	Gauss-Seidel: cyclic	acyclic
0.5	36	22	29	12	5
0.85	158	48	104	37	10
0.99	2819	78	393	523	78

Three observations, each consistent with the theory and none implied by α alone. First, every count is far below k^* : the bound is a worst case. Second, the acyclic graph needs *more* power iterations than the cyclic one here, the opposite of the copying graph. Third, on the acyclic graph every link points *forward*, so Gauss–Seidel’s natural sweep order follows the flow of probability and it needs a small fraction of power iteration’s work; on the cyclic graph at $\alpha = 0.99$ it needs several times more. Which structural parameters govern these counts is an open question for this program; that the *order* of evaluation relative to the link direction matters for Gauss–Seidel and not for power iteration is established ([CGT-OBS-016](#)).

15.5 Cost of an iteration: memory and communication

One power iteration reads every edge once and every vertex a constant number of times: $\Theta(n + m)$ operations. In memory, the standard representation is **compressed sparse rows**: an array of m target indices, an array of $n + 1$ offsets, and two vectors of n floating-point scores — about m words plus $3n$. This is the adjacency-list representation of Chapter 6 laid out contiguously, so that the edge array is read sequentially (arithmetic addressing of edges by offset).

At web scale the graph does not fit in one machine’s memory. Partitioning the vertices across machines makes each iteration a local multiply plus an exchange of the scores flowing along *cut* edges; the communication per iteration is proportional to the number of cut edges, which is why host-based partitioning helps — most links stay within a host [Kamvar et al. 2003]. These are

system-level costs that the word-RAM model does not capture; the book states them qualitatively and makes no measured claim about them.

15.6 Updates and incremental ranking

When the graph changes, the old vector is a good starting point. Since Theorem 15.2 holds from any start, a **warm start** from the previous π converges to the new fixed point in about $\log(\|\pi_{\text{old}} - \pi_{\text{new}}\|_1 / \tau) / \log(1/\alpha)$ iterations instead of $\log(2/\tau) / \log(1/\alpha)$. The saving therefore shrinks as the update moves the fixed point further. In the experiment ([CGT-0BS-017](#)), on a 3,000-page graph a single inserted link cut iterations from 38 to 27; after 10,000 random insertions, which moved the ranking by 0.51 in L_1 , there was no saving at all. Incremental ranking is precomputation with invalidation — the break-even logic of Chapter 12 in a numerical setting [Langville & Meyer 2004].

15.7 Personalization

Replacing the uniform v by a distribution concentrated on pages of interest gives **personalized PageRank**; a set of t topic-specific vectors precomputed and mixed at query time gives **topic-sensitive PageRank** [Haveliwala 2002]. The mixing works because π depends *linearly* on v : from (15.1), $\pi = (1 - \alpha)(I - \alpha M)^{-1}v$ when the dangling redistribution uses a fixed vector, so a convex combination of teleportation vectors yields the same combination of PageRank vectors.

Precomputing t vectors costs t PageRank computations and tn words of storage; each query-time mixture is $\mathcal{O}(tn)$. This is again M-PRE, and again the break-even depends on query volume. (With the convention used here — dangling mass redistributed by v itself — M depends on v and exact linearity requires redistributing dangling mass by a fixed vector instead. The SDK computes each personalized vector directly.)

15.8 Evaluating a ranking

Convergence and certified error say how accurately the PageRank *vector* has been computed. They say nothing about whether the ranking is *useful*. Usefulness is measured against relevance judgments on real queries with standard metrics, and requires real corpora and human or behavioural evidence. This book has no such data and makes no claim about ranking quality, here or in the next chapter.

EXERCISE 15.1

Show that if G is d -regular (every vertex has in- and out-degree d) and v is uniform, then π is uniform for every α , and power iteration from v stops after one step. Why does this make regular graphs useless as benchmarks for PageRank solvers?

► Solution to [Exercise 15.1](#)

EXERCISE 15.2

Reverse every link of the acyclic graph in the example (so links point backward) and rerun Gauss–Seidel. Predict the result from the discussion of sweep order before running it, and compare with [CGT-OBS-016](#).

EXERCISE 15.3

Prove that the dangling-mass convention matters: give a three-vertex graph on which redistributing dangling mass uniformly and redistributing it by a personalized v produce different rankings.

EXERCISE 15.4

Derive the warm-start iteration estimate stated in §15.6 from [Theorem 15.2](#).

SUMMARY

- PageRank is the stationary distribution of a damped random surfer with dangling pages and teleportation; it exists, is unique, and attracts every start (CGT-THM-008).
 - Power iteration contracts the L_1 error by α per step; the residual certifies the error, and $k^* = \lceil \log(\tau(1 - \alpha)/2) / \log \alpha \rceil$ iterations always suffice (CGT-PROP-009).
 - Gauss–Seidel can be faster or much slower depending on vertex order relative to link direction; power iteration is order-invariant.
 - One iteration costs $\Theta(n + m)$ operations and about $m + 3n$ words in compressed sparse rows; distribution costs communication on cut edges.
 - Warm starts save iterations in proportion to how little the update moves the fixed point; personalization is linear in v under a fixed dangling convention.
 - Accuracy of the vector is not usefulness of the ranking; the latter needs relevance data this book does not have.
-

PageRank Through the Lens of Grammar

LEARNING OBJECTIVES

- Read the random surfer as an operational grammar with probabilistic semantics.
- Define the grammar-constrained surfer, prove its basic properties, and state its cost.
- Interpret its measured effect on rankings, and say what would be needed to show that it is useful.
- Place host/block structure, ordering and incremental updates in the mechanism taxonomy.
- List what the grammatical view does not provide for PageRank.

16.1 The surfer as a grammar

The random surfer of Chapter 15 has two moves, *follow* and *teleport*, applied at random. As an operational grammar its states are pages, its moves are relations — follow maps u to its out-neighbours, teleport maps u to every page — and its semantics is **probabilistic**: each move carries a probability distribution over successors instead of a set. The PageRank vector is the long-run frequency with which the moves' composition visits each page. Nothing is gained by renaming the model; the reason to read it this way is that the operational-grammar toolkit of Part II — and in particular admissibility languages and the product construction — applies to it directly.

16.2 Constraining the surfer

On a graph whose links carry labels (a citation that *supports* or *refutes*, a link between people that is *colleague* or *family*, a hyperlink in *navigation* or *content*), one may want a ranking driven only by certain *patterns* of links. Admissibility languages express such patterns, and the product construction of Chapter 8 turns them into a Markov chain.

DEFINITION 16.1 (Grammar-constrained random surfer)

CGT-PROP-010

Let G be an edge-labelled graph on n vertices, $D = (Q, \Sigma, \delta, q_0, F)$ a DFA over labels (all states treated as live; the language is effectively prefix-closed), and $\alpha \in [0, 1)$. The constrained surfer lives on $V \times Q$. From (u, q) :

- with probability α it follows a uniformly random edge (u, a, w) among those with $\delta(q, a)$ defined, moving to $(w, \delta(q, a))$; if there is no such edge, it jumps as in the next case;
- with probability $1 - \alpha$ it jumps to (w, q_0) with w uniform.

The **constrained rank** of w is $\sum_q \pi(w, q)$, where π is the stationary distribution.

PROPOSITION 16.2 (Basic properties of the constrained surfer)

PROPOSED

CGT-PROP-010

1. Only states reachable from $V \times \{q_0\}$ carry probability; there are at most $|Q| \cdot n$ of them.
2. One power iteration costs at most $|Q| \cdot m$ edge operations plus $\mathcal{O}(|Q|n)$.
3. The stationary distribution exists and is unique, and power iteration converges with rate α with the certified error bound of [Theorem 15.2](#).
4. For the one-state universal DFA, constrained rank equals PageRank of the underlying multigraph.

Proof.

(1) and (2) hold by the product construction ([Theorem 8.2](#)): each product state is expanded once per iteration, inspecting the edges of its vertex whose label the DFA admits. (3) is [Theorem 15.2](#) applied to the product chain with teleportation vector $\frac{1}{n} \mathbf{1}_{V \times \{q_0\}}$, a probability distribution. (4) With one state that admits every label, the product chain is the surfer chain itself. ■

The status badge reads *proposed*: the model is defined here, its properties are elementary, and **its novelty is unknown**. Products of Markov chains with automata are standard in probabilistic model checking [Baier & Katoen 2008], and similarity measures driven by label patterns (“meta-paths”) are established for heterogeneous information networks [Sun et al. 2011]; whether this exact ranking model has been published is an open literature question ([CGT-OPEN-011](#)).

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import random
from cgtsdk import DFA, Cost, compile_regex
from cgtsdk.algorithms import constrained_pagerank, pagerank_power

rng = random.Random(10)
```

```

n = 2000
E = [(u, rng.choice("abc"), int(rng.random() ** 2 * n)) for u in range(n) for _ in range(rng.randrange(0, 7))]
E = [(u, a, w) for (u, a, w) in E if u != w]
adj = [[] for _ in range(n)]
for u, _, w in E:
    adj[u].append(w)
plain_cost = Cost()
plain = pagerank_power(adj, 0.85, tol=1e-10, cost=plain_cost)
top = lambda s: [i for i, _ in sorted(enumerate(s), key=lambda t: (-t[1], t[0]))[:10]]
for name, dfa in [("universal", DFA.universal("abc")), ("no c: (a | b)*", compile_regex("(a | b)*")),
                  ("alternate: (a b)* (a | ε)", compile_regex("(a b)* (a | ε)"))]:
    c = Cost()
    scores, res, N = constrained_pagerank(n, E, dfa, 0.85, 1e-10, cost=c)
    l1 = sum(abs(x - y) for x, y in zip(scores, plain.scores))
    print(f"{name:26s} |Q|={dfa.size} product states={N:5d} cost ratio={c.counts['edge']} /
plain_cost.counts['edge']:.2f}"
        f" top-10 overlap={len(set(top(scores)) & set(top(plain.scores)))} L1 distance={l1:.3f}")

```

OUTPUT (CHECKED)

universal	Q =1	product states= 2000	cost ratio=1.03	top-10 overlap=10	L1 distance=0.000
no c: (a b)*	Q =1	product states= 2000	cost ratio=0.80	top-10 overlap=7	L1 distance=0.338
alternate: (a b)* (a ε)	Q =2	product states= 3118	cost ratio=0.67	top-10 overlap=8	L1 distance=0.411

The universal language reproduces PageRank exactly, at a small overhead for building the product. A one-state language that forbids `c` links is *cheaper* than PageRank (fewer admissible edges), is at L_1 distance 0.34 from it — so about 17% of the probability mass moves, since L_1 distance is twice the total-variation distance — and changes three of the top ten pages; the alternating language uses about $1.6n$ product states, moves about 21% of the mass and changes two of the top ten. The ledger’s experiment ([CGT-EXP-010](#)) found the same pattern on other graphs and languages, with costs from $0.46\times$ to $6.4\times$ PageRank, and validated each constrained ranking against a direct linear solve of the materialized product chain ([CGT-OBS-018](#)).

CLAIM BOUNDARY

These experiments show that constrained rankings *differ* from PageRank and what they *cost*. They do not show that the differences are *improvements* for any retrieval task. That requires labelled corpora and relevance judgments, which this research does not have. The proposal is a model with known costs, not a demonstrated improvement.

16.3 Block structure as a structural grammar

Web graphs are not random: most links stay within a host. BlockRank exploits this by computing local rankings inside hosts, combining them by host importance, and using the result to start a global power iteration, reporting a speed-up by a small factor [Kamvar et al. 2003]. In this book’s taxonomy the host partition is a **structural grammar** of the graph (a two-level grammar: hosts, then pages), and the method combines M-SHARE (local computations reused as a starting vector), ordering, and M-PRE (warm starts). The general lesson of Chapter 15 applies: power iteration’s cost per iteration is invariant under reordering, and what structure buys is a *better starting point* or a *better sweep order* for Gauss–Seidel-like methods — and fewer cut edges when the graph is distributed.

16.4 What the grammatical view does not provide

It is important to be explicit:

- **No faster PageRank.** Every iteration still reads every admissible edge. The constrained surfer can be cheaper than PageRank only because it ranks a different, smaller chain.
- **No change to convergence theory.** Rates, bounds and solver behaviour are those of Chapter 15.
- **No evidence about ranking quality.** See the boundary above.
- **No claim about production search.** Modern search engines rank with many signals beyond link analysis; nothing here addresses them.

What it does provide is a uniform way to *state* constrained and structured link analyses — as languages over link labels and grammars over graph structure — together with cost bounds that follow from the product construction rather than having to be derived afresh for each variant. Related ranking problems — personalized and topic-sensitive rankings [Haveliwala 2002], hub-and-authority methods, ranking in heterogeneous networks [Sun et al. 2011] — fit the same pattern, and are directions rather than results of this book.

EXERCISE 16.1

Show that the constrained surfer with a DFA that admits a label only from state q_0 and then has no further transitions (the language “at most one link”) produces a ranking computable without iteration. What is it?

► Solution to Exercise 16.1

EXERCISE 16.2

Prove part 2 of Proposition 16.2 precisely, and give a family of graphs and languages on which the bound $|Q| \cdot m$ per iteration is attained.

EXERCISE 16.3

Design an evaluation that could show whether a constrained ranking is better than PageRank for a concrete task. What data, metric and baseline would you need, and what would count as a negative result?

SUMMARY

- The random surfer is an operational grammar with probabilistic semantics; admissibility languages constrain it through the product construction.
 - The grammar-constrained surfer has at most $|Q|n$ states, costs at most $|Q|m$ edge operations per iteration, converges at rate α with certified error, and reduces to PageRank for the universal language ([CGT-PROP-010](#)).
 - Measured: constrained rankings differ materially from PageRank at costs between about half and six times PageRank's; their usefulness is untested and their novelty unverified.
 - Host structure is a structural grammar of the graph; what it buys is starting points, sweep orders and fewer cut edges, not cheaper iterations.
 - The grammatical view gives a uniform statement of constrained link analysis with derived costs, not a faster PageRank.
-

Computation Graphs, Retrieval, and Agent Systems

LEARNING OBJECTIVES

- Model a computation graph as a structural grammar and identify the mechanisms used by common compiler optimizations.
- Read attention masks as admissibility relations, and separate cost reductions from changes of the computed function.
- Model retrieval pipelines and agent workflows as transition systems with admissibility languages, and validate plans in the syntax layer before execution.
- Explain why approximate vector search is navigation without the guarantees of the earlier chapters.
- State precisely what a symbolic grammar does not capture about a learned model.

17.1 Ground rules

Modern AI systems invite loose analogies with grammars, and loose analogies invite exaggerated claims. This chapter therefore begins each section with an **explicit computational model** — what the elements, moves, semantics and costs are — and keeps three distinctions throughout:

1. **Exact versus approximate.** A method that computes the same function as the original is exact; one that computes a different function, however close, is approximate, and its error must be measured, not assumed.
2. **Symbolic structure versus learned semantics.** A grammar can describe *which* operations a system performs and in *what order*. It does not describe what a trained network's weights compute; nothing here claims that it does.
3. **Model results versus system results.** Word-RAM costs do not account for accelerators, memory bandwidth or batching; no claim about real inference speed is made.

17.2 Computation graphs

Model. A numerical program (a neural network’s forward pass, say) is a DAG whose nodes are operations — matrix products, additions, nonlinearities — and whose edges carry tensors. Executing it means evaluating nodes in a topological order (Chapter 10).

In this book’s terms a computation graph is a **structural grammar** of the computation: a straight-line program in which each node is a production combining earlier results. Familiar compiler optimizations then fall into the mechanism taxonomy:

OPTIMIZATION	WHAT IT DOES	MECHANISM	EXACT?
common-subexpression elimination	merges identical subgraphs	M-SHARE (minimal DAG [Downey et al. 1980])	yes
constant folding	evaluates subgraphs with constant inputs at compile time	M-PRE	yes
operator fusion	merges adjacent nodes to avoid materializing intermediates	representation change (memory traffic)	yes
memory planning	reuses buffers whose lifetimes do not overlap	interval scheduling on the topological order	yes
quantization, pruning	changes the numerical operations	—	no: approximate

The table’s last row is the important one: some “optimizations” change the function computed. They are legitimate engineering, but they are approximations, and their effect on outputs must be evaluated, never assumed from the structure of the graph.

17.3 Attention as a relation

Model. In self-attention [Vaswani et al. 2017], each of n token positions computes a weighted combination of values at other positions, with weights derived from query–key similarities. Structurally, attention evaluates a relation $R \subseteq [n] \times [n]$ — “position i may attend to position j ” —

and costs $\Theta(|R|)$ similarity computations (times the head dimension). Full attention uses $R = [n] \times [n]$: n^2 pairs.

An **attention mask** is an admissibility relation. A causal mask admits $j \leq i$, halving the pairs; a local window of width k admits $|i - j| < k$, about $2kn$ pairs:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
n, k = 4096, 128
full = n * n
causal = n * (n + 1) // 2
window = sum(min(n, i + k) - max(0, i - k + 1) for i in range(n))
causal_window = sum(min(i + 1, k) for i in range(n))
for name, pairs in [("full", full), ("causal", causal), ("window |i-j|<128", window), ("causal window",
causal_window)]:
    print(f"{name:18s} {pairs:10d} pairs ({pairs / full:6.2%} of full)")
```

OUTPUT (CHECKED)

full	16777216 pairs	(100.00% of full)
causal	8390656 pairs	(50.01% of full)
window i-j <128	1028224 pairs	(6.13% of full)
causal window	516160 pairs	(3.08% of full)

The cost reduction is real and exactly proportional to $|R|$ — this is pruning (M-PRUNE) or restriction (M-RESTRICT) in its simplest form. But the boundary is sharp: **restricting the relation changes the function**. A causal mask is part of the model's definition (it is how autoregressive models are specified and trained); a sparse mask imposed on a model trained with full attention computes something else, and how much the outputs change is an empirical question about the model, not a property of the mask.

17.4 Retrieval pipelines and vector search

Model. Retrieval-augmented generation [Lewis et al. 2020] composes a retriever (which returns documents relevant to a query) with a generator conditioned on them. As a transition system its states are the pipeline stages and its moves are calls — embed, retrieve, rerank, generate — with a cost per call.

Retrieval is often **approximate nearest-neighbour search** over vector embeddings. Hierarchical navigable small-world graphs [Malkov & Yashunin 2020] answer queries by greedy routing: starting from an entry point, repeatedly move to the neighbour closest to the query, descending through layers of a hierarchy. In this book's vocabulary a search is a *navigation word* on a proximity graph. But unlike every navigation in Part I, its result is **not guaranteed**: greedy

routing can stop at a local minimum, and the quality of the answer (recall) is measured empirically. The grammatical reading describes the procedure; it supplies no correctness guarantee, and none should be claimed for it.

17.5 Agent workflows and admissible plans

Model. An agent that uses tools — search, read a file, write a file, run a command, send a message — is a transition system over a workspace state. A **plan** is a word over tool calls. Some plans are dangerous or meaningless: deleting before confirming, writing a file never read, sending a message before the draft is reviewed. Such policies are naturally **admissibility languages**, and they can be checked in the *syntax* layer — before any tool runs. This is the one place in this chapter where the book’s machinery applies directly and exactly, and it can be run:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import Inadmissible, Move, Structure

class Workspace(Structure):
    """Files as a set of names; tools as moves. Abstract value: (files, opened)."""
    name = "workspace"
    SIGNATURE = (Move("search", ("q",)), Move("open", ("f",)), Move("edit", ("f",)),
                 Move("confirm", ()), Move("delete", ("f",)))

    def __init__(self, files):
        self.files, self.opened = set(files), set()

    def abstract(self):
        return (frozenset(self.files), frozenset(self.opened))

    @classmethod
    def denote(cls, s, c):
        files, opened = s
        if c.name in ("open", "edit", "delete") and c.args[0] not in files:
            raise Inadmissible(f"no file {c.args[0]!r}")
        if c.name == "open":
            return (files, opened | {c.args[0]}), None
        if c.name == "edit" and c.args[0] not in opened:
            raise Inadmissible(f"{c.args[0]!r} edited before being opened")
        if c.name == "delete":
            return (files - {c.args[0]}, opened - {c.args[0]}), None
        return s, None

    def apply(self, c, cost):
        cost.add("rule", 1)
        state, v = self.denote(self.abstract(), c)
        self.files, self.opened = set(state[0]), set(state[1])
        return v
```

Policy (syntax layer): every delete must be immediately preceded by confirm.

```

policy = "(search | open | edit | confirm)* ((confirm delete) (search | open | edit | confirm))*"
plans = ["search('x'); open('a'); edit('a')",
        "open('a'); delete('a')",
        "open('a'); confirm; delete('a')",
        "edit('a')",
        "confirm; delete('zzz')"]
for p in plans:
    r = Workspace(["a", "b"]).grammar(policy).execute(p)
    verdict = ("REJECTED before execution (policy)" if not r.syntax.ok else
              f"admissible, but fails at step {r.undefined_at}: {r.reason}" if not r.defined else "admissible
and executes")
    print(f"{p:36s} → {verdict}")

```

OUTPUT (CHECKED)

search('x'); open('a'); edit('a')	→ admissible and executes
open('a'); delete('a')	→ REJECTED before execution (policy)
open('a'); confirm; delete('a')	→ admissible and executes
edit('a')	→ admissible, but fails at step 0: 'a' edited before being opened
confirm; delete('zzz')	→ admissible, but fails at step 1: no file 'zzz'

The three layers separate cleanly. The policy (syntax) rejects a plan before any tool runs, at the cost of running a DFA over the plan — linear in its length. The workspace semantics rejects plans that are admissible but impossible in the current state, at the step where they fail. And the costs of the tools themselves are a separate matter entirely. Two cautions apply. Policies that depend on counts or nesting (for example, “every acquired lock is released”) are not regular: they need the context-free machinery of Chapter 3 or a counter. And a policy checks the *plan*, not the model that produced it: it constrains what is executed, not what the model “intends”.

17.6 Persistent histories

The motivating work behind this research program concerned persistent addresses and state histories: a system whose state is reached by a recorded sequence of transitions. In operational-grammar terms, a history is a move word; **replaying** it re-executes the word from a recorded initial state; **provenance** is the word itself. Chapter 13 says exactly when such a history can be *compiled* into a constant-size summary: when the moves act through a composition-closed class of constant-size maps (counters, translations, affine maps, bounded tree navigation). For general state — files, documents, model contexts — the semantics of a move depends on the state in ways that carry unbounded information, and the counting argument of [Theorem 13.3](#) applies in spirit: no constant-size summary can stand in for the history. Snapshots plus logs (precomputation plus replay) remain the general answer, with the break-even trade-offs of Chapter 12.

17.7 What a grammar does not capture

A trained network computes a function determined by its weights. A grammar over its computation graph describes the order and kinds of operations, not that function. Claims that a symbolic grammar “explains”, “compresses” or “accelerates” a learned model must be backed by the same evidence as any other claim in this book: an exact equivalence proof, or a measured approximation error under a stated workload. This book offers neither for any learned model, and claims nothing further.

EXERCISE 17.1

Write a regular admissibility language for a deployment agent with moves `build`, `test`, `deploy` and `rollback`, in which every `deploy` must be preceded by a `test` after the most recent `build`. Check it with the SDK on five plans of your choice.

EXERCISE 17.2

Show that the policy “every `lock(x)` is eventually followed by `unlock(x)`, with locks properly nested” is not regular when the number of locks is unbounded, and regular when at most K locks may be held at once.

EXERCISE 17.3

For an attention mask R , the cost is $\Theta(|R|)$. Give a mask with $\Theta(n \log n)$ pairs in which every position can influence every other within $\mathcal{O}(\log n)$ layers. Which earlier construction in this book does it resemble?

► [Solution to Exercise 17.3](#)

SUMMARY

- Computation graphs are structural grammars; CSE, constant folding and fusion are sharing, precomputation and representation changes — and quantization or pruning are approximations.
 - Attention masks are admissibility relations: cost is proportional to the mask, and imposing a mask changes the function unless the model is defined with it.
 - Retrieval and vector search are pipelines and navigations whose quality is empirical; the grammatical reading adds no guarantee.
 - Agent plans can be validated against admissibility languages in the syntax layer before execution, exactly and cheaply; non-regular policies need more than a DFA.
 - Histories compile to constant size only for uniform move semantics; otherwise snapshots and replay remain necessary.
-

The CGT SDK

LEARNING OBJECTIVES

- Install the SDK and run its test suite.
- Execute operation expressions and read each of the three layers of the result.
- Report the same execution under different cost models.
- Compare implementations on identical workloads against an independent oracle, separating build and query cost.
- Record an experiment with a deterministic fingerprint.
- Extend the SDK with a new structure and know which tests it must pass.

18.1 Installation and layout

The SDK is a Python package, `cgtsdk`, with no dependencies outside the standard library (Python 3.10 or later). From a clone of the repository:

TEXT

```
cd sdk
python3 -m pip install -e .          # or simply: export PYTHONPATH=sdk/src
python3 -m unittest discover -s tests # the full test suite
```

MODULE	CONTENTS
<code>cgtsdk.grammar</code>	Move , Call , Structure , OperationalGrammar , ExecutionResult , Inadmissible , parse_expression
<code>cgtsdk.cost</code>	Cost (counter with optional trace), CostModel , Report , break_even
<code>cgtsdk.automata</code>	DFA , compile_regex (Thompson → subset → Moore)

MODULE	CONTENTS
<code>cgtsdk.structures</code>	arrays, lists, stacks, queues, dequeues, hash tables, heaps, tries, binary trees (pointer/heap), BST/AVL, labelled graphs, automata
<code>cgtsdk.algorithms</code>	search, sorting, union-find, DP, string matching, graph algorithms, range queries, LCA, PageRank solvers, SAT baselines, product reachability, path-grammar closure
<code>cgtsdk.addressing</code>	compiled move words: <code>HeapWord</code> , <code>AffineMod</code> , <code>XorMask</code> , <code>BoxTranslation</code>
<code>cgtsdk.sharing</code>	minimal-DAG forests with rank access and path-copying updates
<code>cgtsdk.lab</code>	generators, <code>compare_structures</code> , <code>compare_functions</code> , <code>ExperimentRecord</code>
<code>cgtsdk.service</code>	the Grammar-as-a-Service contract, local server and client (Chapter 19)

The complete, generated API reference is on the SDK reference page.

18.2 Three layers in one call

Every structure exposes `grammar()` , and `execute` returns an `ExecutionResult` with one field per layer:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk.structures import QueueStructure

g = QueueStructure([1, 2]).grammar()
for expr in ["enqueue(3); dequeue; front", "dequeue; dequeue; dequeue", "enqueue()", "push(1)":
    r = g.execute(expr)
    print(f"{expr:28s} syntax_ok={r.syntax.ok!s:5s} defined={r.defined!s:5s} values={r.values} cost={r.cost.total}"
          + (f" [{r.syntax.error or r.reason}]" if (not r.syntax.ok or not r.defined) else ""))
```

OUTPUT (CHECKED)

```
enqueue(3); dequeue; front    syntax_ok=True  defined=True   values=[None, 1, 2] cost=10
dequeue; dequeue; dequeue    syntax_ok=True  defined=False  values=[2, 3] cost=11 [dequeue on empty queue]
enqueue()                    syntax_ok=False defined=None   values=[] cost=0 [call 0: enqueue takes 1
argument(s) ('x',), got 0]
```

```
push(1)                                syntax_ok=False defined=None values=[] cost=0 [call 0: unknown move 'push';
signature is ['dequeue', 'enqueue', 'front', 'len']]
```

Notice that the structure is *stateful*: the second expression continues from the state left by the first (the queue then held 2, 3), which is why its third `dequeue` is undefined. The reason for each failure is reported — `r.reason` for the semantic layer, `r.syntax.error` for an arity error and an unknown move. (The first expression’s ten steps are 3 for `enqueue` — two arithmetic operations for the ring index and one write — 5 for `dequeue` and 2 for `front`.)

18.3 Cost models

A `Cost` records counts by category (`read`, `write`, `compare`, `arith`, `pointer`, `hash`, `probe`, `alloc`, `edge`, `rule`). A `CostModel` weights categories, so one execution can be reported under several models without rerunning it. The SDK ships `UNIT` (every step costs one) and `MEMORY` (only memory traffic: reads, writes, pointer hops, probes):

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk import MEMORY, UNIT, CostModel
from cgtsdk.structures import ArrayStructure, LinkedListStructure

CACHE_MISS = CostModel("pointer-heavy", 64, (("pointer", 100.0), ("read", 1.0), ("write", 1.0)))
for S in (ArrayStructure, LinkedListStructure):
    r = S(range(1000)).grammar().execute("get(600)")
    print(f"{S.name:12s} unit={r.cost.weighted(UNIT):6.0f} memory={r.cost.weighted(MEMORY):6.0f} pointer-
heavy={r.cost.weighted(CACHE_MISS):8.0f}")
```

OUTPUT (CHECKED)

array	unit=	3	memory=	1	pointer-heavy=	3
linked_list	unit=	603	memory=	602	pointer-heavy=	60003

The third model is a caricature in which a pointer hop costs a hundred times a sequential read — a crude stand-in for cache misses. Cost models are assumptions, and the SDK makes them explicit and swappable instead of hiding them in a single “time”.

18.4 Comparing implementations

`compare_functions` runs implementations on identical inputs, checks every output against an independent oracle, and reports build and query cost separately. Here are three ways to answer

LCA queries on a random tree, with the Euler-tour index's preprocessing reported as build cost:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import random
from cgtsdk.algorithms import EulerLCA, NaiveLCA
from cgtsdk.lab import compare_functions, format_table, random_parent_array

par = random_parent_array(3000, seed=1, shape="random")
oracle = NaiveLCA(par)
rng = random.Random(2)
queries = [(rng.randrange(3000), rng.randrange(3000)) for _ in range(400)]
def scheme(cls):
    # build once (cost reported as build), then query
    return lambda c: (lambda u, v, cost, _i=cls(par, c): _i.query(u, v, cost))
rows = compare_functions({"walk up (depths only)": None, "Euler tour + sparse table": None},
                        queries, oracle=lambda u, v: oracle.query(u, v),
                        builds={"walk up (depths only)": scheme(NaiveLCA),
                               "Euler tour + sparse table": scheme(EulerLCA)})
print(format_table(rows))
```

OUTPUT (CHECKED)

implementation	build	per query	space(w)	correct
walk up (depths only)	3000	14.77	-	True
Euler tour + sparse table	133619	6.00	-	True

Both schemes preprocess: the walk-up scheme only computes depths (one write per vertex), the Euler scheme builds its tour and sparse table. The richer index pays for its extra build only after $(B_{\text{Euler}} - B_{\text{walk}})/(q_{\text{walk}} - q_{\text{Euler}})$ queries — `cgtsdk.break_even` computes this kind of number (Chapter 12).

18.5 Recording experiments

An `ExperimentRecord` stores the hypothesis, the seed, the parameters and the results, together with the environment. Its **fingerprint** is a SHA-256 hash of everything deterministic — excluding the environment and any field whose name mentions wall-clock time — so two runs of the same experiment on different machines can be compared by a single string:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
import random
from cgtsdk import Cost
from cgtsdk.algorithms import heapsort, merge_sort
from cgtsdk.lab import ExperimentRecord
```

```
def run(seed):
    rng = random.Random(seed)
    rec = ExperimentRecord("demo-sorting", "merge sort makes fewer comparisons than heapsort on random input",
                           seed, {"n": 1000}, falsification="heapsort makes fewer comparisons on some seed")
    a = rng.sample(range(10**6), 1000)
    for f in (merge_sort, heapsort):
        c = Cost(); f(a, c)
        rec.results.append({"algorithm": f.__name__, "compare": c.counts["compare"]})
    return rec

r1, r2 = run(7), run(7)
print(r1.results)
print("fingerprints equal:", r1.fingerprint() == r2.fingerprint(), r1.fingerprint()[:16])
```

OUTPUT (CHECKED)

```
[{'algorithm': 'merge_sort', 'compare': 8697}, {'algorithm': 'heapsort', 'compare': 16846}]
fingerprints equal: True b1dc23626e0eb76a
```

18.6 Extending the SDK

A new structure subclasses `Structure` and provides four things (the `BoundedCounter` of Chapter 7 and the `Workspace` of Chapter 17 are complete examples):

1. `SIGNATURE` — the moves, as `Move(name, params, doc, mutates);`
2. `abstract()` — the abstract value the representation stands for;
3. `denote(state, call)` — the pure reference semantics, returning `(new_state, value)` or raising `Inadmissible`;
4. `apply(call, cost)` — the instrumented representation, counting every step it performs.

The contract a new structure must satisfy is the **adequacy property** of Chapter 7: on every expression, `apply` and `denote` agree on values and on the first undefined call. The SDK’s tests check this for every built-in structure on thousands of random calls, including undefined ones (`tests/test_grammar_structures.py`); a new structure should be added to that test. New algorithms should be added with an independent oracle test — brute force on small inputs, or a second algorithm — as every algorithm in `tests/test_algorithms.py` is.

REMARK

The book’s example checker found a real defect during writing: the binary search tree’s recursive insertion overflowed Python’s recursion limit on 1,023 sorted keys. Insertion and deletion are now iterative, and a regression test inserts 5,000 sorted keys ([CGT-D-018](#)). The checker exists precisely to catch this kind of error.

EXERCISE 18.1

Define a cost model in which hash probes cost ten units and everything else one, and use it to compare a hash table with a sorted array plus binary search for 10,000 random lookups. At what load factor does the comparison change?

EXERCISE 18.2

Implement a `DequeStructure` backed by a doubly linked list, add it to the adequacy test, and compare it with the ring-buffer deque using `compare_structures`.

EXERCISE 18.3

Write an `ExperimentRecord` for the break-even analysis of the LCA comparison above, including a falsification criterion, and check that its fingerprint is stable across two runs.

SUMMARY

- The SDK is a standard-library Python package mirroring the book: grammar, cost, automata, structures, algorithms, addressing, sharing, lab and service modules.
 - `execute` reports syntax, semantics and execution separately; structures are stateful.
 - Costs are counted by category and weighted by explicit, swappable cost models.
 - `compare_functions` and `compare_structures` check every output against an oracle and separate build from query cost.
 - Experiment records have deterministic fingerprints; new structures must satisfy adequacy, and new algorithms need oracle tests.
-

Grammar as a Service

LEARNING OBJECTIVES

- State the request/response contract `cgt-gaas/0.1` and the reasons for each of its parts.
- Explain layered rejection — syntax, semantics, execution — as a service-level consequence of the three layers.
- Explain why quotas are stated in counted cost, and why expressing a computation grammatically never makes it free.
- Decide which work belongs locally, ahead of time, in a cache, or remotely.
- Use the reference implementation in-process and over HTTP.

19.1 Why a service at all

The SDK runs locally, and everything in this book can be done without accounts, keys or a network. A service becomes useful when several clients want to apply the *same* versioned grammars and algorithms to their structures, with uniform validation, cost accounting and provenance — for instance, a team validating agent plans against shared policies (Chapter 17), or a pipeline that needs reproducible, auditable ranking computations. The design question is not “how do we put grammars behind an API” but **what contract makes such a service honest**: a request whose cost the client can bound in advance, a rejection that says which layer failed, and a result whose provenance can be checked.

19.2 The contract

A request is a JSON object. It either names a **grammar** — a structure kind and a version — together with an initial state and an operation expression, or names an **algorithm** with declared capabilities, an input and parameters. Both may lower the service’s limits.

TEXT

```
{ "contract": "cgt-gaas/0.1",
  "grammar": { "structure": "graph", "version": "1", "admissible": "(a b)*" },
  "init": { "n": 4, "edges": [[0,"a",1], [1,"b",2], [2,"a",3]] },
  "expression": "at(0) a b here",
  "limits": { "max_cost": 10000, "timeout_ms": 500 } }
```

A response always contains a status, a cost report and provenance:

FIELD	MEANING
status	ok , undefined (semantic layer), rejected (syntax or execution layer, with layer and an error code), or error (internal; no details leaked)
values / result	per-call values of an expression, or an algorithm's result
exact	whether the result is exact or approximate with a stated bound (PageRank returns its certified L_1 bound)
cost	counted steps by category, under the SDK's cost conventions
provenance	SDK version, SHA-256 of the canonical request and of the result, determinism flag, wall time

The reference implementation is a pure function, `cgtsdk.service.handle(request)` → response , wrapped by a small HTTP server. Calling the function in-process and calling the server return the same result hashes.

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk.service import Client

svc = Client() # in-process; Client("http://host:port") talks to a server
base = {"grammar": {"structure": "stack", "version": "1"}, "init": {"values": []}}
for expr in ["push(1); push(2); pop", "push(1); pop; pop", "push(1); fly"]:
    r = svc.execute(dict(base, expression=expr))
    print(f"{expr:24s} → {r['status']:9s} layer={r.get('layer', '-'):9s} values={r.get('values')} cost={r['cost']['total']}")
    + (f" error={r['error']['code']} " if 'error' in r else "")
r = svc.execute({"grammar": {"structure": "stack", "version": "2"}, "expression": "pop"})
print("version mismatch →", r["status"], r["error"]["code"], "|", r["error"]["message"])
```

OUTPUT (CHECKED)

```
push(1); push(2); pop → ok          layer=-          values=[None, None, 2] cost=8
push(1); pop; pop    → undefined layer=semantics values=[None, 1] cost=4
```

```
push(1); fly          → rejected layer=syntax    values=None cost=0 error=inadmissible
version mismatch → rejected version_mismatch | stack is at version 1, request asked 2
```

19.3 Layered rejection

The three layers of Chapter 1 become three kinds of “no”:

1. **Syntax** — the request is malformed, names an unknown structure or move, has the wrong arity, asks for the wrong version, or its move word is not in the admissibility language. Nothing is executed and nothing is charged.
2. **Semantics** — the expression is admissible but undefined in the current state. The response reports the index of the first undefined call and the reason; values computed before that call are returned, and their cost is charged.
3. **Execution** — the computation would exceed a quota: too many calls, too much counted cost, too large an output, too much time.

Separating them matters to clients. A syntax rejection is a bug in the request, and retrying will not help. A semantic “undefined” is a fact about the state and may be the expected answer. An execution rejection says the request may succeed with larger limits — but never larger than the service’s hard limits.

19.4 Quotas in counted cost

Every request is bounded by `max_calls`, `max_cost` (counted steps), `max_output_items`, `timeout_ms` and `max_n`. A request may lower any limit and never raise one. Stating the main quota in **counted cost** rather than time has two advantages: it is deterministic, so a request that fits its quota on one machine fits it on every machine; and it forces the cost model to be explicit. It also makes the central point of this book operational: **expressing a computation grammatically does not make it cheap**. A `get(4999)` on a linked list of 5,000 elements costs about 5,000 steps whether it is sent as code, as an expression or as a service request, and the quota sees exactly that:

PYTHON · EXECUTED BY THE BOOK'S EXAMPLE CHECKER

```
from cgtsdk.service import handle

req = {"grammar": {"structure": "linked_list", "init": {"values": list(range(5000))},
               "expression": "; ".join(["get(4999)"] * 50), "limits": {"max_cost": 100000}},
      "r": handle(req)
print(r["status"], r["layer"], r["error"]["code"], "-", r["error"]["message"])
```

```
r = handle(dict(req, grammar={"structure": "array"}))
print(r["status"], "cost", r["cost"]["total"], "for the same 50 calls on an array")
```

OUTPUT (CHECKED)

```
rejected execution quota - cost exceeded max_cost=100000 at call 19
ok cost 150 for the same 50 calls on an array
```

The reference server enforces cost and time limits *between* calls of an expression. Algorithm requests are bounded by input size and iteration limits and checked after the fact; a production service would need preemptive enforcement (separate processes or cooperative cancellation), which the reference implementation does not provide ([CGT-D-016](#)).

19.5 Determinism, provenance and caching

Every result carries the SHA-256 of the canonical request and of the result. Because the contract is a deterministic function of the request, **a cached result is valid exactly when the request hash matches and the grammar version is unchanged**; the reference server caches on the request hash, and its tests check that a cache hit returns the same result hash as a fresh evaluation. Cache validity becomes conditional when any of the following holds, and a production design must then include them in the key or invalidate explicitly:

- a grammar or algorithm is updated *without* a version change (forbidden by the contract: semantic changes require a new version);
- results depend on external state (a mutable stored structure rather than one sent in `init`);
- results are approximate with non-deterministic algorithms (randomized solvers must take the seed as a parameter).

19.6 What should run where

The mechanisms of Chapter 7 suggest a placement rule:

WORK	WHERE	WHY
syntax checks (parsing, arity, admissibility DFA)	client or edge	cheap, deterministic, needs no data; rejects bad requests before any transfer

WORK	WHERE	WHY
compiling admissibility languages, compiling move words (Ch. 13)	ahead of time, cached by grammar version	M-PRE on the <i>query</i> side: pay once per version
building indexes (closures, labelings)	where the data lives, with break-even accounting	M-PRE on the <i>data</i> side: only worthwhile past Q^* queries per epoch (Ch. 12)
executing expressions on large structures	where the structure lives	moving the structure costs more than moving the request
small structures sent in <code>init</code>	anywhere; the service is stateless	determinism and caching are then unconditional

The reference implementation is deliberately stateless: structures arrive in `init`, so every request is self-contained, reproducible and cacheable. Stateful services — structures stored server-side and mutated by requests — need access control, isolation between clients and versioned snapshots, and are outside the current contract.

19.7 Security boundaries

The reference server binds to `127.0.0.1` by default, has no authentication, refuses bodies larger than 1 MiB, never executes client-supplied code (only moves of registered structures and registered algorithms), and returns no tracebacks. It is for local study. Exposing it to untrusted networks would require authentication, rate limiting per client, preemptive resource limits and audit logging — none of which is claimed here.

EXERCISE 19.1

Write a request that asks the service for language-constrained reachability on a graph you choose, and a second request whose answer you can predict to be `undefined` in the semantic layer. Verify both with `Client()`.

EXERCISE 19.2

Extend the contract with a `compile` operation that returns the compiled form of a move word for heap-shaped trees (Chapter 13). Which layer rejects a word with an unknown move? Is the compiled form cacheable, and under which key?

EXERCISE 19.3

Design the cache key and invalidation rule for a *stateful* variant of the service in which structures are stored server-side and mutated by requests.

SUMMARY

- `cgt-gaas/0.1` requests name a versioned grammar or a declared algorithm; responses carry status, values, exactness, counted cost and provenance hashes.
 - Rejections are layered — syntax, semantics, execution — and mean different things to clients.
 - Quotas are stated in counted cost, deterministic across machines; a grammatical request costs what its execution costs.
 - Results are cacheable by request hash because the contract is deterministic and grammar versions are immutable.
 - Syntax checks and compilation belong ahead of the data; index building belongs with the data and must pass break-even; the reference server is local, stateless and not hardened.
-

What Computational Grammar Theory Has Established

LEARNING OBJECTIVES

- Summarize, for each of the four founding questions, what this edition has established and how firmly.
 - Separate candidate contributions from restatements of known results.
 - List the negative results that bound the theory.
 - Identify the research that most deserves independent review, collaboration and publication.
-

20.1 The four questions

The research program set itself four questions. Here are the answers this edition can defend.

1. What is the object? An *operational grammar* $G = (\Sigma, \mathfrak{A}, [\cdot], \mathcal{L})$: moves with relational semantics on a computational structure, constrained by an admissibility language over the projection of move words, executed by representations whose adequacy is checkable and whose costs are reported as profiles in an explicit machine model (CGT-DEF-003, CGT-DEF-005). Alongside it, *structural grammars* generate the structure itself (CGT-DEF-004). The components are standard — labelled transition systems, relational semantics, formal languages, abstract data types — and the book says so (Chapter 7). The definition’s value is that it covers every structure of Part I in one format while keeping syntax, semantics and execution apart.

2. What can it do? Its semantics is compositional, so paths, navigations, plans and histories really are grammatical objects (CGT-PROP-001). Grammars drive computation through product constructions for constrained search (CGT-THM-001), fixpoint evaluation of path grammars (CGT-THM-004), arithmetic evaluation of addresses (CGT-THM-003), compilation of move words (CGT-THM-006), sharing (CGT-THM-005), and validation of plans before execution (Chapter 17). They also define new models — the grammar-constrained random surfer — with costs derived from the general constructions (CGT-PROP-010).

3. What does it cost? Every capability above has a cost profile and a failure family:

CAPABILITY	BEST CASE	FAILURE FAMILY	LEDGER
arithmetic addressing	$\mathcal{O}(1)$ navigation, no index	deep or sparse shapes; rotations relabel subtrees	THM-003, NEG-001-003
compiled move words	$\mathcal{O}(w)$ once, then $\mathcal{O}(1)$ per application	arbitrary shapes need $\lfloor n/2 \rfloor$ bits per move	THM-006, THM-007
precomputed answers	$\mathcal{O}(1)$ retrieval	$\lfloor n/2 \rfloor \lceil n/2 \rceil$ bits for reachability; break-even Q^* ; updates	THM-002, PROP-005
sharing	exponential compression	$\Theta(k \log n)$ inflation after k updates; wrong kind of repetition	THM-005
admissibility constraints	prune to one state	multiply work by exactly $ Q $	THM-001
path grammars	correct by construction	evaluation is closure computation; shape changes cost	THM-004

4. What has actually been discovered? Separated honestly:

- *Candidate contributions, novelty not yet verified.* The constant-size normal form for move words in heap-shaped trees with its interval domain, together with the matching impossibility result for arbitrary shapes ([CGT-THM-006](#), [CGT-THM-007](#)). The grammar-constrained random surfer as a ranking model ([CGT-PROP-010](#)). The unified discipline — operational grammars, three layers, cost profiles, and the five-mechanism account of where grammatical advantage comes from — together with the *mechanism thesis* that these five account for all of it ([CGT-CONJ-001](#), a conjecture).
- *Elementary results proved here that are probably folklore.* The tight factor- $|Q|$ cost of constraints, the exact information bound for retrieval-only reachability, the exact work count of right-linear path-grammar evaluation, update inflation of shared trees, break-even under invalidating updates, the exact cost of doubling arrays under this book's cost model.
- *Known results restated in CGT terms.* Compositional semantics, product constructions, heap numbering and LCA, PageRank's convergence theory, the comparison lower bound, the P/NP boundary for grammar-based claims.

- *Empirical findings.* Eighteen reproducible observations, among them the break-even range from 4 to 27,000 queries, four different winning strategies across nine dynamic workloads, Gauss–Seidel’s dependence on vertex order, and solution-grammar growth for SAT that the data cannot classify.

20.2 What did not survive

The research began with an ambitious conjecture: that sufficiently expressive grammars might make problems dramatically easier, approaching constant-time resolution. In its strong form, this is rejected ([CGT-NEG-009](#)). Within standard models a grammar helps decide a hard problem only as an ordinary algorithm, as advice, as an exponential table or through a restriction of the problem ([CGT-PROP-006](#)). Several weaker hypotheses also failed and are kept in the ledger: address-word LCA as a competitive method, Gauss–Seidel as a generally faster PageRank solver, and constraints as a generally pruning device. Each failure sharpened a boundary that now appears as a theorem or a counterexample.

20.3 Where grammar becomes advantage

Gathering the positive results, one pattern recurs across arrays, ring buffers, heaps, Fenwick trees, grids, hypercubes, de Bruijn graphs and compiled words: **grammatical structure becomes computational advantage when the meaning of a move is determined by a small number of parameters of the structure, so that sentences of the grammar can be evaluated — or compiled — by arithmetic instead of by traversal.** When the structure itself carries the information, as in arbitrary shapes, the advantage must instead be bought with stored information, and the counting lemma prices it. This is the most precise answer this edition can give to the question with which the program began. Turning it into a theorem is open problem [CGT-OPEN-012](#).

20.4 Research agenda

In order of priority:

1. **Independent review** of the proofs marked “proved here”, starting with [CGT-THM-006](#) and [CGT-THM-007](#).
2. **Literature review for novelty** of the compiled-word normal form (implicit tree navigation, transition monoids, automatic structures) and of the constrained surfer (probabilistic model checking, meta-path ranking) ([CGT-OPEN-010](#), [CGT-OPEN-011](#)).

3. **Characterize compilable classes** ([CGT-OPEN-012](#)): Cayley graphs of groups with efficient word problems are the natural next family.
4. **Context-free admissibility**: the cost-of-constraint theory for CFL-reachability ([CGT-OPEN-008](#)).
5. **Formalize the mechanism thesis** ([CGT-OPEN-001](#)) so that it becomes a theorem or is refuted.
6. **Evaluate constrained rankings** on corpora with relevance judgments, which requires collaboration and data this program does not have.

Results that pass review would be candidates for publication: the compilation theorems as a short note on implicit navigation, and the constrained surfer, if novel and useful, as an applied paper. Until then, the book's status remains what its front page says: an unreviewed research manuscript, with every claim labelled by the evidence behind it.

EXERCISE 20.1

Pick one result marked “proved here” and review its proof as an independent referee would: check every step, look for missing hypotheses, and search for prior work. Write a short referee report.

EXERCISE 20.2

Propose a sixth mechanism of advantage not covered by [Definition 7.5](#), or argue that a candidate (caching, partial evaluation, parallelism, incremental maintenance) reduces to the five. What would count as a counterexample to the mechanism thesis?

SUMMARY

- The object is the operational grammar with three separated layers and explicit cost profiles; its components are standard and its discipline is the contribution.
- Capabilities — addressing, compilation, precomputation, sharing, constraints, path grammars — each come with a cost profile and a proven failure family.
- Candidate contributions: compiled move words with a matching impossibility result, the grammar-constrained surfer, and the mechanism account; novelty awaits review.
- The strong founding conjecture is rejected; the surviving answer is that grammar becomes advantage when move semantics is fixed by few parameters, so sentences can be evaluated by arithmetic.
- The agenda: review, novelty search, characterization of compilable classes, context-free constraints, the mechanism thesis, and real-data evaluation.

Notation and Conventions

A.1 Sets, relations, words

NOTATION	MEANING
$[n]$	$\{1, \dots, n\}$ (or $\{0, \dots, n - 1\}$ where stated)
$\Sigma, \Sigma^*, \varepsilon$	an alphabet of moves; all finite words over it; the empty word
$ w $	length of a word w
$\pi_\Gamma(w)$	projection of w onto the letters in Γ
$R ; S$	relational composition: first R , then S
$\text{Rel}(A)$	the relations on A
$\llbracket a \rrbracket, \llbracket w \rrbracket, \llbracket \mathcal{L} \rrbracket$	meaning of a move, a word, a language (Definition Definition 6.1)
$\llbracket w \rrbracket(x) = \emptyset$	w is undefined at x
$\text{dom } f$	domain of a partial map
$\mathcal{L}$	an admissibility language
$D = (Q, \Sigma, \delta, q_0, F)$	a deterministic automaton with partial δ ; $ Q $ its number of states
$\mathfrak{A} = (A; R_1, \dots)$	a computational structure
$G = (\Sigma, \mathfrak{A}, \llbracket \cdot \rrbracket, \mathcal{L})$	an operational grammar (Definition Definition 7.1)

A.2 Numbers, bits, asymptotics

NOTATION	MEANING
w (in cost statements)	the machine word size in bits; context distinguishes it from a word $w \in \Sigma^*$
$\lfloor x \rfloor, \lceil x \rceil$	floor, ceiling
$x \gg k, x \ll k$	shift right / left by k bits
$x \oplus y$	bitwise exclusive or
$\text{msb}(x)$	index of the most significant set bit ($\text{msb}(0) = -1$)
$\text{lowbit}(i)$	$i \& (-i)$, the lowest set bit
$\nu(w) = \text{int}(1w)$	the heap numeration of a binary address
$\mathcal{O}, \Omega, \Theta, o, \omega$	asymptotic notation; the growing parameters are always named
$ x _1$	L_1 norm $\sum_i x_i $

A.3 Cost conventions

Costs are counted elementary steps in categories `read`, `write`, `compare`, `arith`, `pointer`, `hash`, `probe`, `alloc`, `edge`, `rule`. An arithmetic operation on a b -bit integer costs $\lceil b/64 \rceil$ steps. The **unit** cost model weighs every category by one; other models are stated where used. A **cost profile** is $(T_{\text{build}}, S, T_{\text{query}}, T_{\text{update}}, |\text{out}|)$.

A.4 Evidence labels

Proved here · Established · Adapted · Empirical · Conjecture · Rejected · Definition · Proposed — see §§1.8. Ledger identifiers: CGT-DEF definition, CGT-THM / PROP / COR results, CGT-CONJ conjectures, CGT-OBS observations, CGT-EXP experiments, CGT-NEG / REJ negative results, CGT-OPEN open problems, CGT-D decisions, CGT-H hypotheses.

Reproducing This Book

B.1 What is reproducible, and how exactly

ARTIFACT	COMMAND (FROM THE REPOSITORY ROOT)	REPRODUCIBILITY
theorem checks (small-case exhaustive)	<code>python3 experiments/check_theorems.py</code>	exact
research experiments EXP-001 ... EXP-010	<code>sh experiments/run_all.sh</code>	counted steps bit-for-bit; wall-clock fields vary
comparison with committed results	<code>python3 experiments/verify_reproduction.py</code>	exits non-zero if any counted field differs
SDK test suite	<code>cd sdk && python3 -m unittest discover -s tests</code>	exact
book examples	<code>python3 book/tools/check_examples.py</code>	every printed output must match exactly
laboratory-SDK agreement	<code>cd book && npm test</code>	exact (floating point to 10^{-14})
site build with integrity checks	<code>cd book && npm install && npm run build</code>	deterministic HTML; fails on any integrity error
print edition (PDF)	<code>cd book && npm run pdf</code>	generated from the same sources via headless Chromium

Requirements: Python 3.10 or later (standard library only) and, for the book build, Node.js 18 or later with the pinned packages of `book/package.json`. All random inputs are generated from fixed seeds.

B.2 What reproducibility does not cover

Wall-clock times depend on the machine and are reported as secondary data only. The book's arguments rest on counted steps under stated cost models, which are machine-independent. The environment in which the recorded results were produced is listed in

`REPRODUCIBILITY_REPORT.md` in the repository, together with the outcome of each command.

Bibliography

55 entries: 46 metadata verified, 8 secondary sources only, 1 unverified.

Metadata verified means authors, title, venue and year were checked against publisher, DBLP or institutional records; it does *not* mean the full text was read. Each entry states what was consulted.

Adelson-Velsky & Landis 1962

G. M. Adelson-Velsky, E. M. Landis (1962). *An algorithm for the organization of information*. Doklady Akademii Nauk SSSR (English translation in Soviet Mathematics Doklady).

SECONDARY SOURCES ONLY

Journal details from general knowledge; not verified.

Agrawal et al. 1989

R. Agrawal, A. Borgida, H. V. Jagadish (1989). *Efficient management of transitive relationships in large data and knowledge bases*. Proc. ACM SIGMOD 1989, pp. 253–262. doi:10.1145/67544.66950.

METADATA VERIFIED

Aspvall et al. 1979

B. Aspvall, M. F. Plass, R. E. Tarjan (1979). *A linear-time algorithm for testing the truth of certain quantified Boolean formulas*. Information Processing Letters 8(3):121–123. doi:10.1016/0020-0190(79)90002-4. METADATA VERIFIED

Baier & Katoen 2008

C. Baier, J.-P. Katoen (2008). *Principles of Model Checking*. MIT Press (ISBN 978-0-262-02649-9).

METADATA VERIFIED

Cited for the general product of Markov chains with automata; the specific section was not located.

Barrett et al. 2000

C. Barrett, R. Jacob, M. Marathe (2000). *Formal-language-constrained path problems*. SIAM Journal on Computing 30(3):809–837. doi:10.1137/S0097539798337716. [METADATA VERIFIED](#)

Bellman 1958

R. Bellman (1958). *On a routing problem*. Quarterly of Applied Mathematics 16(1):87–90. [METADATA VERIFIED](#)

Bender & Farach-Colton 2000

M. A. Bender, M. Farach-Colton (2000). *The LCA problem revisited*. Proc. LATIN 2000, LNCS 1776, pp. 88–94. [METADATA VERIFIED](#)

Bianchini et al. 2005

M. Bianchini, M. Gori, F. Scarselli (2005). *Inside PageRank*. ACM Transactions on Internet Technology 5(1):92–128. doi:10.1145/1052934.1052938. [METADATA VERIFIED](#)

Bille et al. 2011

P. Bille, G. M. Landau, R. Raman, K. Sadakane, S. R. Satti, O. Weimann (2011). *Random access to grammar-compressed strings*. Proc. 22nd ACM-SIAM SODA, pp. 373–389. [link](#). [METADATA VERIFIED](#)
A journal version (SIAM J. Comput.) exists; its volume and pages were not verified.

Blumensath & Grädel 2000

A. Blumensath, E. Grädel (2000). *Automatic structures*. Proc. 15th IEEE LICS, pp. 51–62. [METADATA VERIFIED](#)

Brin & Page 1998

S. Brin, L. Page (1998). *The anatomy of a large-scale hypertextual Web search engine*. Computer Networks and ISDN Systems 30:107–117. [METADATA VERIFIED](#)

Buneman et al. 2003

P. Buneman, M. Grohe, C. Koch (2003). *Path queries on compressed XML*. Proc. 29th VLDB, pp. 141–152. doi:10.1016/B978-012722442-8/50021-5. [METADATA VERIFIED](#)

Charikar et al. 2005

M. Charikar, E. Lehman, D. Liu, R. Panigrahy, M. Prabhakaran, A. Sahai, A. Shelat (2005). *The smallest grammar problem*. IEEE Transactions on Information Theory 51(7):2554–2576. doi:10.1109/TIT.2005.850116. [METADATA VERIFIED](#)

Cohen et al. 2003

E. Cohen, E. Halperin, H. Kaplan, U. Zwick (2003). *Reachability and distance queries via 2-hop labels*. SIAM Journal on Computing 32(5):1338–1355. doi:10.1137/S0097539702403098. [METADATA VERIFIED](#)

Cormen et al. 2022

T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein (2022). *Introduction to Algorithms, 4th edition*. MIT Press (ISBN 978-0-262-04630-5). [METADATA VERIFIED](#)

Darwiche & Marquis 2002

A. Darwiche, P. Marquis (2002). *A knowledge compilation map*. Journal of Artificial Intelligence Research 17:229–264. doi:10.1613/jair.989. METADATA VERIFIED

Dijkstra 1959

E. W. Dijkstra (1959). *A note on two problems in connexion with graphs*. Numerische Mathematik 1:269–271. doi:10.1007/BF01386390. METADATA VERIFIED

Downey et al. 1980

P. J. Downey, R. Sethi, R. E. Tarjan (1980). *Variations on the common subexpression problem*. Journal of the ACM 27(4):758–771. METADATA VERIFIED

Epstein et al. 1992

D. B. A. Epstein, J. W. Cannon, D. F. Holt, S. V. F. Levy, M. S. Paterson, W. P. Thurston (1992). *Word Processing in Groups*. Jones and Bartlett (later A K Peters). METADATA VERIFIED

Fenwick 1994

P. M. Fenwick (1994). *A new data structure for cumulative frequency tables*. Software: Practice and Experience 24(3):327–336. doi:10.1002/spe.4380240306. METADATA VERIFIED

Ford 1956

L. R. Ford Jr. (1956). *Network flow theory*. RAND Corporation Paper P-923.

SECONDARY SOURCES ONLY

Its role as an origin of the Bellman–Ford algorithm is commonly stated but was not verified here.

Harel & Tarjan 1984

D. Harel, R. E. Tarjan (1984). *Fast algorithms for finding nearest common ancestors*. SIAM Journal on Computing 13(2):338–355. doi:10.1137/0213024. METADATA VERIFIED

Haveliwala & Kamvar 2003

T. Haveliwala, S. Kamvar (2003). *The second eigenvalue of the Google matrix*. Technical report, Stanford University. SECONDARY SOURCES ONLY

Haveliwala 2002

T. H. Haveliwala (2002). *Topic-sensitive PageRank*. Proc. 11th International World Wide Web Conference (WWW 2002). link. METADATA VERIFIED

Hoare 1962

C. A. R. Hoare (1962). *Quicksort*. The Computer Journal 5(1):10–16. doi:10.1093/comjnl/5.1.10. METADATA VERIFIED

Sources disagree on the last page (15 or 16).

Italiano 1986

G. F. Italiano (1986). *Amortized efficiency of a path retrieval data structure*. Theoretical Computer Science 48:273–281. METADATA VERIFIED

Jacobson 1989

G. Jacobson (1989). *Space-efficient static trees and graphs*. Proc. 30th IEEE FOCS, pp. 549–554.

METADATA VERIFIED

Kahn 1962

A. B. Kahn (1962). *Topological sorting of large networks*. Communications of the ACM 5(11):558–562.

doi:10.1145/368996.369025. SECONDARY SOURCES ONLY

Kamvar et al. 2003

S. Kamvar, T. Haveliwala, C. Manning, G. Golub (2003). *Exploiting the block structure of the Web for computing PageRank*. Technical Report 2003-17, Stanford University. SECONDARY SOURCES ONLY

Karp & Lipton 1980

R. M. Karp, R. J. Lipton (1980). *Some connections between nonuniform and uniform complexity classes*. Proc. 12th ACM STOC, pp. 302–309. SECONDARY SOURCES ONLY

The improvement of the collapse to the second level is commonly credited to Sipser (secondary sources).

Khoussainov & Nerode 1995

B. Khoussainov, A. Nerode (1995). *Automatic presentations of structures*. Logic and Computational Complexity, LNCS 960. UNVERIFIED

Known through citing papers only; pages not confirmed.

Kleitman & Rothschild 1975

D. J. Kleitman, B. L. Rothschild (1975). *Asymptotic enumeration of partial orders on a finite set*.

Transactions of the AMS 205:205–220. METADATA VERIFIED

Knuth 1968

D. E. Knuth (1968). *Semantics of context-free languages*. Mathematical Systems Theory 2(2):127–145.

doi:10.1007/BF01692511. METADATA VERIFIED

Knuth et al. 1977

D. E. Knuth, J. H. Morris Jr., V. R. Pratt (1977). *Fast pattern matching in strings*. SIAM Journal on Computing 6(2):323–350. doi:10.1137/0206024. METADATA VERIFIED

Kozen 1997

D. Kozen (1997). *Kleene algebra with tests*. ACM Transactions on Programming Languages and Systems 19(3):427–443. doi:10.1145/256167.256195. METADATA VERIFIED

Kruskal 1956

J. B. Kruskal (1956). *On the shortest spanning subtree of a graph and the traveling salesman problem*. Proceedings of the AMS 7:48–50. doi:10.1090/S0002-9939-1956-0078686-7. METADATA VERIFIED

Langville & Meyer 2004

A. N. Langville, C. D. Meyer (2004). *Deeper inside PageRank*. Internet Mathematics 1(3):335–380.

doi:10.1080/15427951.2004.10129091. METADATA VERIFIED

Langville & Meyer 2006

A. N. Langville, C. D. Meyer (2006). *Google's PageRank and Beyond: The Science of Search Engine Rankings*. Princeton University Press (ISBN 978-0-691-12202-1). [METADATA VERIFIED](#)

Lewis et al. 2020

P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, D. Kiela (2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks*. Advances in Neural Information Processing Systems 33. [link](#). [METADATA VERIFIED](#)
Page numbers not verified.

Lohrey 2015

M. Lohrey (2015). *Grammar-based tree compression*. Proc. DLT 2015 (LNCS). doi:10.1007/978-3-319-21500-6_3. [METADATA VERIFIED](#)

Malkov & Yashunin 2020

Yu. A. Malkov, D. A. Yashunin (2020). *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence. doi:10.1109/TPAMI.2018.2889473. [SECONDARY SOURCES ONLY](#)
DOI and arXiv verified; the journal issue year was not confirmed.

Mendelzon & Wood 1995

A. O. Mendelzon, P. T. Wood (1995). *Finding regular simple paths in graph databases*. SIAM Journal on Computing 24(6):1235–1258. [METADATA VERIFIED](#)

Page et al. 1999

L. Page, S. Brin, R. Motwani, T. Winograd (1999). *The PageRank citation ranking: Bringing order to the Web*. Technical Report 1999-66, Stanford InfoLab. [link](#). [SECONDARY SOURCES ONLY](#)

Prim 1957

R. C. Prim (1957). *Shortest connection networks and some generalizations*. Bell System Technical Journal 36(6):1389–1401. doi:10.1002/j.1538-7305.1957.tb01515.x. [METADATA VERIFIED](#)

Reps 1998

T. Reps (1998). *Program analysis via graph reachability*. Information and Software Technology 40(11–12):701–726. doi:10.1016/S0950-5849(98)00093-7. [METADATA VERIFIED](#)

Schieber & Vishkin 1988

B. Schieber, U. Vishkin (1988). *On finding lowest common ancestors: simplification and parallelization*. SIAM Journal on Computing 17(6):1253–1262. doi:10.1137/0217079. [METADATA VERIFIED](#)

Sun et al. 2011

Y. Sun, J. Han, X. Yan, P. S. Yu, T. Wu (2011). *PathSim: Meta path-based top-k similarity search in heterogeneous information networks*. Proceedings of the VLDB Endowment 4(11):992–1003.

[METADATA VERIFIED](#)

Tarjan 1972

R. E. Tarjan (1972). *Depth-first search and linear graph algorithms*. SIAM Journal on Computing 1(2):146–160. doi:10.1137/0201010. [METADATA VERIFIED](#)

Tarjan 1975

R. E. Tarjan (1975). *Efficiency of a good but not linear set union algorithm*. Journal of the ACM 22(2):215–225. [METADATA VERIFIED](#)

Tarjan 1981a

R. E. Tarjan (1981). *A unified approach to path problems*. Journal of the ACM 28(3):577–593. [METADATA VERIFIED](#)

Tarjan 1981b

R. E. Tarjan (1981). *Fast algorithms for solving path problems*. Journal of the ACM 28(3):594–614. doi:10.1145/322261.322273. [METADATA VERIFIED](#)

Vaswani et al. 2017

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin (2017). *Attention is all you need*. Advances in Neural Information Processing Systems 30, pp. 5998–6008. [METADATA VERIFIED](#)

Verbin & Yu 2013

E. Verbin, W. Yu (2013). *Data structure lower bounds on random access to grammar-compressed strings*. Proc. CPM 2013 (LNCS). link. [METADATA VERIFIED](#)
Venue confirmed via arXiv and citing papers.

Williams 1964

J. W. J. Williams (1964). *Algorithm 232: Heapsort*. Communications of the ACM 7(6):347–348. [METADATA VERIFIED](#)

Yannakakis 1990

M. Yannakakis (1990). *Graph-theoretic methods in database theory*. Proc. 9th ACM PODS, pp. 230–242. doi:10.1145/298514.298576. [METADATA VERIFIED](#)